

UNIVERSIDAD INTERCONTINENTAL DE LA EMPRESA



FACULTAD DE A CORUÑA

**TRABAJO FIN DE GRADO EN
INGENIERÍA DE SISTEMAS INTELIGENTES**

**Análisis y Mitigación de Vulnerabilidades Cuánticas en
Redes Blockchain (EVM): Una Aproximación a la Criptografía
Basada en Retículos**

Autor:

Manuel Mateo Delgado-Gambino López

Director(es):

Isaac Iglesias González



Esta obra se encuentra sujeta a la licencia *Creative Commons* (CC-BY-NC-ND)

Compartir-Reconocimiento-No Comercial-Sin Obra Derivada

RESUMEN

El algoritmo de Shor, ejecutable en un computador cuántico criptográficamente relevante (CRQC), es capaz de resolver el problema del logaritmo discreto en curvas elípticas en tiempo polinómico, comprometiendo la seguridad de ECDSA/secp256k1, el esquema de firma empleado en todas las transacciones de Ethereum. Este trabajo analiza las vulnerabilidades cuánticas de la pila criptográfica de la Máquina Virtual de Ethereum (EVM) y propone una mitigación concreta: un nuevo tipo de transacción (0x50) basado en ML-DSA-65 (CRYSTALS-Dilithium, FIPS-204), el primer esquema de firma post-cuántica estandarizado por NIST, fundamentado en la dureza del problema Module-LWE sobre retículos algebraicos.

La solución se materializa en un *fork* del cliente de ejecución *reth*, donde se reemplaza ECDSA por ML-DSA-65 en la firma y verificación de transacciones, se introduce una derivación de direcciones basada en SHAKE-256 en lugar de Keccak-256, se implementa un nuevo *opcode* EVM (PQHASH, 0x21) y un precompilado para verificación ML-DSA-65, y se deshabilitan trece precompilados clásicos vulnerables a ataques cuánticos. El consenso se implementa como Proof of Authority (PoA) con sellado ML-DSA-65 y rotación round-robin de validadores.

Como componente motivacional, se desarrolla una API de simulación cuántica con Qiskit que demuestra la recuperación de claves privadas mediante el algoritmo de Shor sobre una curva elíptica reducida y la búsqueda de preimágenes hash mediante el algoritmo de Grover.

La validación se realiza mediante una suite de pruebas extremo a extremo con quince tests que verifican identidad de cadena, consenso, modelo de comisiones EIP-1559, transacciones post-cuánticas, despliegue de contratos inteligentes y consistencia entre tres nodos validadores. Todos los tests pasan satisfactoriamente. El trabajo concluye con una propuesta de EIP (*Ethereum Improvement Proposal*) para la estandarización del tipo de transacción post-cuántica.

Palabras clave:

blockchain, computación cuántica, criptografía post-cuántica, Ethereum, EVM, FIPS-204, ML-DSA-65, reth, retículos, SHAKE-256

ABSTRACT

Shor's algorithm, executable on a cryptographically relevant quantum computer (CRQC), can solve the elliptic-curve discrete logarithm problem in polynomial time, compromising the security of ECDSA/secp256k1—the signature scheme used in every Ethereum transaction. This work analyses the quantum vulnerabilities of the Ethereum Virtual Machine (EVM) cryptographic stack and proposes a concrete mitigation: a new transaction type (0x50) based on ML-DSA-65 (CRYSTALS-Dilithium, FIPS-204), the first post-quantum digital signature scheme standardised by NIST, grounded on the hardness of the Module-LWE problem over algebraic lattices.

The solution is implemented as a fork of the *reth* execution client, where ECDSA is replaced by ML-DSA-65 for transaction signing and verification, address derivation is changed from Keccak-256 to SHAKE-256, a new EVM opcode (PQHASH, 0x21) and an ML-DSA-65 verification precompile are introduced, and thirteen classical precompiles vulnerable to quantum attacks are disabled. Consensus is implemented as Proof of Authority (PoA) with ML-DSA-65 block sealing and round-robin validator rotation.

As a motivational component, a quantum simulation API built with Qiskit demonstrates private key recovery via Shor's algorithm on a reduced elliptic curve and hash preimage search via Grover's algorithm.

Validation is performed through an end-to-end test suite comprising fifteen tests that verify chain identity, consensus, the EIP-1559 fee model, post-quantum transactions, smart contract deployment, and state consistency across three validator nodes. All tests pass successfully. The work concludes with an EIP (Ethereum Improvement Proposal) draft for the standardisation of the post-quantum transaction type.

Keywords:

blockchain, Ethereum, EVM, FIPS-204, lattices, ML-DSA-65, post-quantum cryptography, quantum computing, reth, SHAKE-256

DEDICATORIA

*A mis padres,
por su apoyo incondicional
durante toda la carrera.*

AGRADECIMIENTOS

En primer lugar, quiero agradecer a mi director de TFG, Isaac Iglesias González, por su orientación, disponibilidad y rigor durante el desarrollo de este trabajo. Sus comentarios y revisiones han sido fundamentales para dar forma a la memoria que aquí se presenta.

A Eladio Dapena González, mi mentor durante toda la carrera. Más allá de las clases, su forma de enseñar, su honestidad académica y la confianza que ha depositado en mí han marcado profundamente mi manera de entender la ingeniería.

A todos los profesores y compañeros de carrera, por las clases, los proyectos compartidos, las largas tardes de laboratorio y las conversaciones que han hecho de estos años una etapa que llevaré siempre conmigo.

A la Universidad Intercontinental de la Empresa y, en particular, a la Facultad de A Coruña, por proporcionar el marco académico y los recursos necesarios para completar este grado.

A mis padres, por su apoyo constante, su paciencia y por creer en mí en cada etapa de este camino.

Finalmente, a nivel técnico, este trabajo no habría sido posible sin las comunidades de código abierto que lo sustentan. En particular, agradezco al equipo de Paradigm por el desarrollo de *reth*, un cliente de ejecución Ethereum modular y de alta calidad en Rust; a IBM y a la comunidad Qiskit, cuyo *framework* de computación cuántica ha permitido las simulaciones de Shor y Grover presentadas en este trabajo; y, en general, a todos los mantenedores de los *crates* de Rust, las herramientas de $\text{\LaTeX}$ y las bibliotecas criptográficas sobre las que este proyecto se apoya.

ÍNDICE DE CONTENIDOS

RESUMEN	II
ABSTRACT	III
DEDICATORIA	V
AGRADECIMIENTOS	VII
ÍNDICE DE FIGURAS	XIV
ÍNDICE DE TABLAS	XV
1 INTRODUCCIÓN	1
1.1 Motivación del trabajo	1
1.2 Planteamiento del problema	1
1.3 Objetivos	2
1.3.1 Objetivo general	2
1.3.2 Objetivos específicos	2
1.4 Antecedentes	3
1.5 Metodología	4
1.6 Organización del documento	4
2 MARCO TEÓRICO	6
2.1 Criptografía de curva elíptica y ECDSA	6
2.1.1 La curva secp256k1	6
2.1.2 Esquema de firma ECDSA	7
2.1.3 Recuperación de clave pública (ecrecover)	7
2.2 Computación cuántica	8
2.2.1 Modelo de circuito cuántico	8
2.3 Algoritmo de Shor	8
2.3.1 Estructura del algoritmo	8
2.3.2 Complejidad y recursos	9
2.3.3 Simulación sobre curva reducida	9
2.4 Algoritmo de Grover	9
2.4.1 Estructura del algoritmo	10
2.4.2 Implicaciones para funciones hash	10
2.5 Criptografía basada en retículos	11
2.5.1 Problemas computacionales duros	11
2.5.2 Learning With Errors (LWE)	13
2.5.3 Module-LWE	13
2.5.4 Justificación de la elección de retículos	13

2.6	ML-DSA (FIPS-204)	14
2.6.1	Generación de claves	14
2.6.2	Firma	14
2.6.3	Verificación	15
2.6.4	Niveles de seguridad	15
2.7	ML-KEM (FIPS-203)	15
2.7.1	Generación de claves	15
2.7.2	Encapsulación	16
2.7.3	Desencapsulación	16
2.7.4	Niveles de seguridad	16
2.8	Ethereum y la Máquina Virtual de Ethereum (EVM)	17
2.8.1	Arquitectura de capas	17
2.8.2	Modelo de cuentas y estado mundial	17
2.8.3	Merkle Patricia Trie	18
2.8.4	La EVM como máquina de pila	18
2.8.5	Modelo de mensajes y contratos inteligentes	19
2.8.6	Formalización como autómata finito determinista	19
2.8.7	Cómo la migración post-cuántica preserva los contratos	20
2.8.8	Transacciones tipadas (EIP-2718)	21
2.8.9	Precompilados	22
2.8.10	Modelo de comisiones EIP-1559	22
2.9	Reth como cliente de ejecución	22
2.10	SHAKE-256 y la familia SHA-3	23
2.10.1	Construcción esponja	23
2.10.2	SHAKE-256 como XOF	23
2.10.3	Justificación del uso de SHAKE-256	24
3	ANÁLISIS DE LA SITUACIÓN	25
3.1	Descripción detallada del problema	25
3.1.1	Superficie de ataque cuántico en Ethereum	25
3.1.2	Análisis temporal: CRQC vs. complejidad de migración	25
3.1.3	Escenarios de fallo	26
3.1.4	Plan de migración para usuarios	26
3.2	Requisitos	28
3.2.1	Requisitos funcionales	28
3.2.2	Requisitos no funcionales	28
3.2.3	Matriz de trazabilidad	29
3.3	Solución propuesta	29
3.3.1	Visión general de la arquitectura	29
3.3.2	Componentes del sistema	30
3.3.3	Cobertura de los puntos de vulnerabilidad	31
3.4	Alternativas descartadas	31
3.4.1	Falcon-512 como esquema de firma	31

3.4.2	SLH-DSA (SPHINCS+) como esquema de firma	32
3.4.3	Abstracción de cuentas (ERC-4337 / EIP-7702)	32
3.4.4	Proof of Stake post-cuántico	32
3.4.5	Geth como cliente base	33
4	DESARROLLO	34
4.1	Simulación cuántica (qiskit-api)	34
4.1.1	Curva elíptica reducida	34
4.1.2	Algoritmo de Shor para ECDLP	34
4.1.3	Keccak reducido y algoritmo de Grover	35
4.1.4	Endpoints REST	35
4.2	Biblioteca criptográfica (ml-lattice-rs)	36
4.2.1	ML-DSA (FIPS-204)	36
4.2.2	ML-KEM (FIPS-203)	36
4.2.3	SHAKE-256	36
4.3	Tipo de transacción PQ (0x50)	36
4.3.1	Hash de firma	37
4.3.2	Hash de transacción	37
4.3.3	Derivación de direcciones	37
4.3.4	Codificación RLP	37
4.3.5	Recuperación del remitente	38
4.4	EVM post-cuántica	39
4.4.1	Opcode PQHASH (0x21)	39
4.4.2	Precompilado ML-DSA-65 (0x0100)	39
4.4.3	Precompilados deshabilitados	40
4.4.4	Decisión de diseño: simplificación de precompilados	40
4.5	Consenso Proof of Authority	41
4.5.1	Conjunto de validadores	41
4.5.2	Rotación round-robin	41
4.5.3	Sellado de bloques	42
4.5.4	Verificación de bloques	42
4.5.5	Flujo de minado	42
4.6	Red y propagación de bloques	43
4.7	Pool de transacciones	43
4.8	Wallet post-cuántico	44
4.8.1	Biblioteca núcleo (pq-wallet-core)	44
4.8.2	Keystore cifrado	44
4.8.3	Interfaz CLI	44
4.8.4	Interfaz TUI	45
4.9	Suite de pruebas E2E	45
4.10	Infraestructura de despliegue	46
4.10.1	Docker Compose	46
4.10.2	Kubernetes	46

4.10.3	Génesis	47
4.10.4	Nodo binario (pq-reth)	47
4.11	Propuesta de EIP	47
5	RESULTADOS	49
5.1	Simulaciones cuánticas	49
5.1.1	Algoritmo de Shor: recuperación de clave privada	49
5.1.1.1	Ejecución detallada paso a paso	49
5.1.2	Algoritmo de Grover: búsqueda de preimágenes	50
5.2	Pruebas unitarias y de integración	51
5.3	Validación de extremo a extremo	51
5.3.1	Configuración single-node	51
5.3.2	Configuración multi-nodo	51
5.4	Análisis de rendimiento	53
5.4.1	Operaciones criptográficas	53
5.4.2	Funciones hash	53
5.4.3	Tamaño de transacciones	54
5.4.4	Impacto en gas y throughput	54
5.4.5	Calibración de gas del precompilado	56
5.5	Comparación con el estado del arte	56
5.6	Análisis de seguridad	57
5.6.1	Protección frente a replay	57
5.6.2	Resistencia a DoS por inflación de transacciones	57
5.6.3	Maleabilidad de firma y unicidad de transacción	58
5.6.4	Resistencia frente a oráculos cuánticos de hashing	58
5.7	Análisis de resultados	59
6	CONCLUSIONES Y TRABAJOS FUTUROS	60
6.1	Cumplimiento de objetivos	60
6.2	Conclusiones generales	61
6.3	Limitaciones	61
6.4	Trabajos futuros	62
7	BIBLIOGRAFÍA	64
8	ANEXOS	65
8.1	Anexo 1: Declaración de uso de IAG	65
8.2	Anexo 2: Especificación EIP — Post-Quantum Transaction Type	66
8.3	Anexo 3: Estructura de repositorios	68
8.4	Anexo 4: Resultados de Aprendizaje	70

ÍNDICE DE FIGURAS

2.1	Circuito cuántico del algoritmo de Shor para ECDLP. El oráculo U_f evalúa $f(a, b) = aG + bQ$ de forma coherente. Tras la medición del registro de salida y la QFT, los resultados (s, t) permiten recuperar k	9
2.2	Circuito de Grover para $n = 4$ qubits. Cada iteración aplica el oráculo de fase O_f (invierte la fase de las soluciones) y el operador de difusión $D = 2 s\rangle\langle s - I$ (amplifica la amplitud de los estados marcados).	10
2.3	Retículo bidimensional $\mathcal{L}(\mathbf{B}) \subset \mathbb{R}^2$. El mismo retículo admite dos bases: una «buena», casi ortogonal (sólida, azul), y otra «mala», casi degenerada (punteada, roja). El problema SVP consiste en encontrar el vector no nulo más corto del retículo; el problema CVP, dado un punto objetivo t (rojo) fuera del retículo, consiste en encontrar el punto del retículo más cercano (verde). La dureza de ambos crece exponencialmente con la dimensión n cuando solo se dispone de una base mala.	12
2.4	Autómata finito determinista $\mathcal{M}_{\text{EVM}}$. La ejecución comienza en q_0 , evoluciona por aplicaciones sucesivas de δ dentro del estado agregado EJECUTANDO (cada opcode consume gas y modifica la configuración), y converge a uno de los estados terminales: STOP/RETURN comprometen los cambios al estado mundial; REVERT los descarta pero devuelve gas; OOG/INVALID descartan los cambios y consumen todo el gas.	21
2.5	Construcción esponja: los bloques del mensaje m_i se absorben mediante XOR con la tasa r del estado seguido de la permutación f . En la fase de extracción, se extraen r bits por aplicación de f . La capacidad c nunca se expone directamente.	23
3.1	Flujo de migración de un usuario desde una cuenta clásica A a una cuenta post-cuántica A' . La transacción puente (paso 3) es la última firma ECDSA que el usuario produce; los pasos 1, 2 y 4 son completamente PQ. El paso 5 lo dispara el protocolo, no el usuario.	27
3.2	Arquitectura general del sistema PostQuantumEVM.	30
4.1	Disposición de bytes de una transacción 0x50 mínima (transferencia nativa con input vacío). El byte de tipo va fuera del sobre RLP (regla EIP-2718). El 99 % del espacio lo ocupan la firma ML-DSA-65 (3.309 B) y la clave pública (1.952 B). El esquema solo representa los primeros 32 bytes y un fragmento de los campos largos; el resto se denota con elipsis. . . .	38
4.2	Flujo de una transacción post-cuántica: desde la generación de claves y firma en el wallet hasta la verificación e inclusión en un bloque en el nodo.	39
4.3	Rotación round-robin de 3 validadores: cada bloque h es producido por el validador $h \bmod 3$, garantizando una distribución equitativa.	42
4.4	Propagación de bloques PoA: el nodo productor anuncia el bloque vía P2P; el nodo receptor lo importa, verifica el sello ML-DSA-65 y lo inyecta en su cadena canónica mediante la engine API.	43

5.1	Distribución de probabilidad empírica de la medición s del primer registro tras la QFT modular, con $k = 7$, $n = 18$ y 4.096 <i>shots</i> . La distribución es aproximadamente uniforme sobre $\mathbb{Z}_{18}$ porque $\gcd(k, n) = 1$, lo que garantiza que cualquier muestra (s, t) con $\gcd(t, n) = 1$ permite recuperar k unívocamente.	51
5.2	Latencia de Keccak-256 frente a SHAKE-256 en función del tamaño de entrada (escala log-log). Para entradas pequeñas SHAKE-256 introduce un sobrecoste constante de ~ 250 ns por el padding diferente, pero a partir de 4 KB ambas curvas convergen al mismo régimen asintótico lineal dictado por la permutación $f = \text{Keccak-}f[1600]$ compartida.	54
5.3	Comparativa de tamaño de transacción por componente. La firma ML-DSA-65 (3.309 B) y la clave pública (1.952 B) constituyen el 99 % del incremento, multiplicando por 46 el tamaño total respecto a una transacción ECDSA clásica.	55
5.4	Timeline cuántico: la ventana de migración (6–13 años) se solapa con la estimación de disponibilidad de un CRQC (10–15 años, NIST). La amenaza «harvest now, decrypt later» comienza hoy y la zona crítica (rayado rojo) representa los años en que la migración aún no estará completa pero el atacante cuántico ya podría descifrar.	59

ÍNDICE DE TABLAS

2.1	Comparativa de familias PQC para firmas digitales.	14
2.2	Parámetros y tamaños de ML-DSA por nivel de seguridad.	15
2.3	Parámetros y tamaños de ML-KEM por nivel de seguridad.	16
2.4	Tipos de transacción Ethereum asignados (mayo 2026).	22
3.1	Superficie de ataque cuántico en Ethereum por componente.	25
3.2	Matriz de trazabilidad: requisitos, objetivos y componentes.	29
3.3	Cobertura de los puntos de vulnerabilidad por la solución propuesta.	31
4.1	Parámetros del Keccak reducido para la simulación de Grover.	35
4.2	Endpoints de la API de simulación cuántica.	35
4.3	Especificación del opcode PQHASH.	39
4.4	Módulos de pq-wallet-core.	44
4.5	Subcomandos de pq-wallet CLI.	45
4.6	Fases de la suite de pruebas E2E.	46
4.7	Servicios del despliegue Docker Compose.	46
5.1	Pruebas unitarias por <i>crate</i> post-cuántico.	52
5.2	Resultados E2E en configuración single-node (12/12 tests).	52
5.3	Resultados E2E adicionales en configuración multi-nodo (3/3 tests).	53
5.4	Comparativa de latencia: ECDSA/secp256k1 vs. ML-DSA-65.	53
5.5	Comparativa de latencia: Keccak-256 vs. SHAKE-256.	53
5.6	Comparativa de tamaño de transacción: clásica vs. post-cuántica.	54
5.7	Impacto en gas y capacidad de la red.	55
5.8	Comparación con trabajos relacionados.	56
8.1	Resultados de Aprendizaje de tipo Conocimiento.	70
8.2	Resultados de Aprendizaje de tipo Habilidad.	71
8.3	Resultados de Aprendizaje de tipo Competencia.	71

1 INTRODUCCIÓN

1.1 Motivación del trabajo

La seguridad de las redes blockchain públicas descansa sobre primitivas criptográficas cuya solidez se asume computacionalmente intratable para las computadoras clásicas. En particular, Ethereum —la segunda red blockchain por capitalización de mercado, con un volumen diario de transacciones que supera los cien mil millones de dólares— emplea el esquema de firma digital ECDSA (*Elliptic Curve Digital Signature Algorithm*) sobre la curva secp256k1 para autenticar cada transacción enviada a la red (Wood, 2014).

La seguridad de ECDSA se fundamenta en la dureza computacional del Problema del Logaritmo Discreto en Curvas Elípticas (ECDLP): dado un punto $Q = k \cdot G$ en la curva, donde G es el generador y k la clave privada, no existe un algoritmo clásico eficiente para recuperar k . Los mejores ataques clásicos (Pollard's rho, baby-step giant-step) requieren $O(\sqrt{n})$ operaciones, donde n es el orden del grupo, lo que para secp256k1 ($n \approx 2^{256}$) implica aproximadamente 2^{128} operaciones: una barrera computacional infranqueable con la tecnología actual.

Sin embargo, en 1994 Peter Shor demostró que un computador cuántico de escala suficiente puede resolver el ECDLP en tiempo polinómico $O(n^3)$ (Shor, 1994). Esto significa que la aparición de un computador cuántico criptográficamente relevante (CRQC, *Cryptographically Relevant Quantum Computer*) reduciría la seguridad de secp256k1 de 2^{128} operaciones clásicas a un número polinómico de operaciones cuánticas, haciendo viable la falsificación de firmas digitales y, por extensión, el robo de fondos de cualquier cuenta cuya clave pública haya sido expuesta.

Las estimaciones actuales sitúan la disponibilidad de un CRQC en un horizonte de 10 a 15 años (National Institute of Standards and Technology, 2024c). Si bien este plazo puede parecer lejano, dos factores exigen una acción preventiva inmediata:

1. **Amenaza «harvest now, decrypt later»:** Un adversario puede registrar hoy las transacciones firmadas que se propagan por la red —incluyendo las claves públicas que se derivan durante la verificación— y almacenarlas hasta que disponga de un CRQC capaz de recuperar las claves privadas correspondientes. Dado que la blockchain es un registro público e inmutable, todas las transacciones históricas están disponibles para este tipo de ataque (Fernández-Caramés & Fraga-Lamas, 2023).
2. **Complejidad de la migración:** Ethereum es un sistema distribuido con cientos de miles de nodos, múltiples implementaciones de clientes (geth, Nethermind, Besu, Erigon, reth), y un ecosistema de contratos inteligentes valorado en miles de millones. Una migración criptográfica requiere coordinación a escala global y años de planificación, desarrollo, pruebas y despliegue progresivo.

Estas consideraciones motivan el presente trabajo: demostrar la viabilidad técnica de migrar la capa de transacciones de Ethereum a criptografía post-cuántica basada en retículos, proporcionando una implementación funcional y una propuesta de estandarización.

1.2 Planteamiento del problema

El esquema ECDSA/secp256k1 no es un componente aislado de Ethereum, sino que está profundamente integrado en múltiples capas del protocolo. Se identifican cinco puntos de vulnerabilidad cuántica:

- P1. Firma de transacciones:** Cada transacción incluye una firma ECDSA (v, r, s) que autentica al remitente. Un CRQC podría falsificar firmas para cualquier cuenta cuya clave pública sea conocida.
- P2. Recuperación del remitente:** Ethereum no incluye la dirección del remitente en la transacción. En su lugar, la función `recover` recupera la clave pública a partir de la firma (v, r, s) y el hash del mensaje. Esta propiedad de «recuperabilidad» es exclusiva de ECDSA y no existe en los esquemas de firma post-cuánticos.
- P3. Derivación de direcciones:** Las direcciones Ethereum se derivan como $\text{keccak256}(pk)[12:32]$, donde pk es la clave pública no comprimida de 64 bytes (sin el prefijo `0x04`). Este esquema está acoplado al formato de clave `secp256k1`.
- P4. Precompilados criptográficos:** La EVM incluye precompilados para operaciones sobre curvas elípticas (BN254: `0x06–0x08`; BLS12-381: `0x0b–0x13`) y evaluación de puntos KZG (`0x0a`). Todas estas primitivas se basan en el problema del logaritmo discreto o en emparejamientos bilineales, vulnerables al algoritmo de Shor.
- P5. Consenso (Proof of Stake):** La capa de consenso de Ethereum utiliza firmas BLS12-381 para la atestación y propuesta de bloques por parte de los validadores. BLS12-381 es igualmente vulnerable a ataques cuánticos, aunque su migración es un problema separado que requiere agregación de firmas post-cuántica —una capacidad que aún no existe de forma estandarizada.

En el momento de redacción de este trabajo, no existe ninguna propuesta de estandarización (EIP) que defina un tipo de transacción nativo post-cuántico. Las propuestas existentes —EIP-7932 (Ethereum Community, 2025a) (registro de algoritmos de firma secundarios) y EIP-8051 (Ethereum Community, 2025b) (precompilado ML-DSA)— abordan la verificación a nivel de contrato inteligente pero no la autenticación a nivel de protocolo.

1.3 Objetivos

1.3.1 Objetivo general

Demostrar la viabilidad de migrar la capa de transacciones de un cliente de ejecución Ethereum a criptografía post-cuántica basada en retículos, mediante la implementación de un tipo de transacción nativo con ML-DSA-65 y su validación en un entorno multi-nodo.

1.3.2 Objetivos específicos

- OE1.** Simular los ataques de Shor (recuperación de clave privada ECDLP) y Grover (búsqueda de preimagen hash) sobre modelos reducidos de las primitivas criptográficas de Ethereum mediante circuitos cuánticos implementados con Qiskit.
- OE2.** Diseñar un formato de transacción post-cuántica (`0x50`) compatible con la especificación EIP-2718 (*Typed Transaction Envelope*), incluyendo la codificación RLP, el esquema de firma y el hash de transacción.
- OE3.** Implementar la derivación de direcciones post-cuánticas basada en SHAKE-256 como alternativa a Keccak-256, garantizando la separación de dominios con las direcciones clásicas.

- OE4.** Implementar un *opcode* EVM (PQHASH, 0x21) para computación SHAKE-256 dentro de contratos inteligentes, y un precompilado (0x0100) para verificación ML-DSA-65.
- OE5.** Diseñar e implementar un mecanismo de consenso Proof of Authority (PoA) con sellado de bloques mediante ML-DSA-65 y rotación round-robin de validadores, como alternativa al Proof of Stake que requiere agregación BLS.
- OE6.** Desarrollar un *wallet* post-cuántico con interfaz de línea de comandos (CLI) y terminal interactiva (TUI) que permita la generación de claves, firma de transacciones y consulta de saldos.
- OE7.** Validar la solución completa mediante una suite de pruebas extremo a extremo (E2E) ejecutada sobre un clúster de tres nodos validadores, verificando identidad de cadena, consenso, comisiones, transacciones, contratos y consistencia de estado.
- OE8.** Redactar una propuesta de EIP (*Ethereum Improvement Proposal*) que formalice la especificación del tipo de transacción post-cuántica para su eventual presentación a la comunidad Ethereum.

1.4 Antecedentes

Estandarización post-cuántica del NIST

En 2016, el Instituto Nacional de Estándares y Tecnología de Estados Unidos (NIST) inició un proceso de selección de algoritmos criptográficos resistentes a ataques cuánticos. Tras tres rondas de evaluación pública, en agosto de 2024 se publicaron los tres primeros estándares finales (National Institute of Standards and Technology, 2024a, 2024b, 2024d):

- **FIPS-203 (ML-KEM):** Mecanismo de encapsulación de claves basado en Module-LWE (anteriormente CRYSTALS-Kyber).
- **FIPS-204 (ML-DSA):** Esquema de firma digital basado en Module-LWE (antes CRYSTALS-Dilithium). Es el esquema empleado en este trabajo.
- **FIPS-205 (SLH-DSA):** Esquema de firma digital basado en funciones hash (antes SPHINCS+), sin asunciones sobre retículos.

El corpus bibliográfico de este trabajo incluye las especificaciones completas de los tres estándares, así como las implementaciones de referencia de los candidatos originales del NIST (Dilithium, Falcon, Kyber, SPHINCS+).

Propuestas existentes en Ethereum

EIP-2718 (Zoltu, 2020) estableció en 2020 el marco de transacciones tipadas (*Typed Transaction Envelope*), que permite añadir nuevos formatos de transacción sin romper la compatibilidad con los existentes. Actualmente se han asignado los tipos 0x00–0x04.

EIP-7932 propone un registro de algoritmos de firma secundarios con un precompilado *sigrecover*. EIP-8051 propone precompilados específicos para verificación ML-DSA. Ambas propuestas abordan la verificación a nivel de contrato inteligente, pero ninguna define un tipo de transacción nativo, que es la contribución principal de este trabajo.

Trabajos relacionados en blockchain post-cuántica

Varios trabajos académicos han explorado la integración de criptografía post-cuántica en redes blockchain. Kiktenko et al. (2018) presentan un esquema de blockchain protegida cuánticamente. Fernández-Caramés y Fraga-Lamas (2023) analizan la resistencia cuántica en redes blockchain desde una perspectiva general. Berdik et al. (2023) ofrecen una comparativa exhaustiva de blockchains post-cuánticas y cuánticas. Lee et al. (2024) implementan protección post-cuántica en Hyperledger Fabric. El presente trabajo se diferencia al abordar específicamente la EVM de Ethereum con una implementación nativa a nivel de protocolo.

1.5 Metodología

El desarrollo del proyecto sigue una metodología iterativa basada en *sprints* semanales, con las siguientes prácticas:

- **Control de versiones:** Repositorios Git con flujo de ramas por funcionalidad (*feature branches*), integración mediante *pull requests* con *squash and merge*.
- **Integración continua:** Cada *pull request* se valida con `cargo test` (tests unitarios e integración), `cargo clippy` (análisis estático) y `cargo +nightly fmt` (formato).
- **Pila tecnológica:**
 - Rust 1.95 (nodo de ejecución, *wallet*, librería criptográfica)
 - Python 3.13 + Qiskit 2.3 + Qiskit Aer 0.17 (simulación cuántica)
 - Docker + docker-compose (despliegue multi-nodo)
 - ratatui 0.29 + crossterm 0.28 (interfaz TUI)
- **Estructura de repositorios:**
 - PostQuantumEVM/ — repositorio raíz (orquestración, infraestructura, E2E)
 - pq-reth/ — *fork* de reth con cinco *crates* post-cuánticos
 - pq-wallet/ — *wallet* CLI, TUI y suite E2E
 - ml-lattice-rs/ — submódulo con implementación de ML-KEM y ML-DSA
 - qiskit-api/ — API de simulación cuántica

1.6 Organización del documento

El resto de la memoria se estructura en los siguientes capítulos:

- **Capítulo 2 – Marco Teórico:** Presenta los fundamentos matemáticos de la criptografía de curva elíptica, la computación cuántica (algoritmos de Shor y Grover), la criptografía basada en retículos (LWE, Module-LWE, ML-DSA), la arquitectura de Ethereum y la EVM, el cliente reth, y las funciones hash extensibles SHAKE-256.
- **Capítulo 3 – Análisis de la Situación:** Detalla las vulnerabilidades cuánticas de Ethereum, define los requisitos funcionales y no funcionales, y presenta la solución propuesta junto con las alternativas descartadas.

- **Capítulo 4 – Desarrollo:** Describe la implementación de cada componente: simulación cuántica (Qiskit), tipo de transacción 0x50, primitivas criptográficas, EVM post-cuántica, consenso PoA, *wallet* y infraestructura de despliegue.
- **Capítulo 5 – Resultados:** Presenta los resultados de las simulaciones cuánticas, los tests unitarios y de integración, la validación E2E en configuración single-node y multi-nodo, el análisis de rendimiento, la comparativa con el estado del arte, y la propuesta de EIP.
- **Capítulo 6 – Conclusiones y Trabajos Futuros:** Resume los objetivos alcanzados, identifica limitaciones del trabajo y propone líneas de trabajo futuro.

2 MARCO TEÓRICO

Este capítulo presenta los fundamentos matemáticos y tecnológicos sobre los que se construye el trabajo. Se parte de la criptografía de curva elíptica (la primitiva que se pretende reemplazar), se introducen los modelos de computación cuántica y los algoritmos que la amenazan, se formalizan los problemas de retículos que fundamentan la alternativa post-cuántica, y se describe la infraestructura de Ethereum que es objeto de modificación.

2.1 Criptografía de curva elíptica y ECDSA

Definición 2.1 (Curva elíptica sobre $\mathbb{F}_p$). Sea $p > 3$ un primo. Una *curva elíptica* E sobre el cuerpo finito $\mathbb{F}_p$ es el conjunto de puntos $(x, y) \in \mathbb{F}_p \times \mathbb{F}_p$ que satisfacen la ecuación de Weierstrass corta

$$E : y^2 = x^3 + ax + b \pmod{p}, \quad 4a^3 + 27b^2 \not\equiv 0 \pmod{p}, \quad (2.1)$$

junto con un punto distinguido $\mathcal{O}$ (el *punto en el infinito*) que actúa como elemento neutro.

El conjunto $E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 : y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$ forma un grupo abeliano bajo la operación de adición de puntos, definida geométricamente como la intersección de la recta que pasa por dos puntos con la curva, reflejada respecto al eje x .

Definición 2.2 (Suma de puntos). Dados $P = (x_1, y_1)$ y $Q = (x_2, y_2)$ en $E(\mathbb{F}_p)$ con $P \neq -Q$:

$$P + Q = (x_3, y_3), \quad \text{donde} \quad \begin{cases} \lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{p} & \text{si } P \neq Q, \\ \lambda = \frac{3x_1^2 + a}{2y_1} \pmod{p} & \text{si } P = Q, \end{cases} \quad (2.2)$$

con $x_3 = \lambda^2 - x_1 - x_2 \pmod{p}$ e $y_3 = \lambda(x_1 - x_3) - y_1 \pmod{p}$.

2.1.1 La curva secp256k1

Ethereum emplea la curva secp256k1, definida por los parámetros:

$$\begin{aligned} a &= 0, \quad b = 7, \\ p &= 2^{256} - 2^{32} - 977, \\ n &= \text{FFFEBAEDCE6AF48A03BBFD25E8CD0364141}_{16}, \end{aligned} \quad (2.3)$$

donde n es el orden del subgrupo generado por el punto base G . La seguridad de secp256k1 descansa en la intratabilidad del siguiente problema:

Definición 2.3 (Problema del Logaritmo Discreto en Curvas Elípticas (ECDLP)). Dado un punto $Q = k \cdot G$ en $E(\mathbb{F}_p)$, donde G es un generador conocido y $k \in \{1, \dots, n-1\}$ es un escalar secreto, el ECDLP consiste en recuperar k a partir de Q y G .

Los mejores algoritmos clásicos para el ECDLP (Pollard's ρ , baby-step giant-step) tienen complejidad $O(\sqrt{n}) \approx O(2^{128})$ para secp256k1.

Mapa biyectivo $k \leftrightarrow Q$ y unilateralidad. La función $f_G : \mathbb{Z}_n \rightarrow \langle G \rangle \subset E(\mathbb{F}_p)$, $k \mapsto kG$ es un homomorfismo de grupos biyectivo (pues $\langle G \rangle$ es cíclico de orden n). El cálculo de f_G (multiplicación escalar) es eficiente: $O(\log n)$ adiciones de puntos con el algoritmo *double-and-add*. La inversa f_G^{-1} , sin embargo, es precisamente el ECDLP. ECDSA, ECDH y todas las operaciones de ecrecover explotan esta asimetría: el firmante calcula kG trivialmente, mientras que cualquier atacante con conocimiento únicamente de Q se enfrenta a un problema $\Theta(2^{128})$. Es esta unilateralidad la que el algoritmo de Shor destruye al transformarla en un problema $\Theta((\log n)^3)$ en un computador cuántico tolerante a fallos.

2.1.2 Esquema de firma ECDSA

El esquema ECDSA consta de tres algoritmos:

1. **Generación de claves:** Elegir $k \xleftarrow{\$} \{1, \dots, n-1\}$. Calcular $Q = k \cdot G$. La clave privada es k , la pública es Q .
2. **Firma $\text{Sign}(k, m)$:** Dado un mensaje m :
 - a. Calcular $e = \text{hash}(m)$ (en Ethereum, $\text{hash} = \text{keccak256}$).
 - b. Elegir $r_{\text{nonce}} \xleftarrow{\$} \{1, \dots, n-1\}$.
 - c. Calcular $(x_1, y_1) = r_{\text{nonce}} \cdot G$.
 - d. Calcular $r = x_1 \bmod n$. Si $r = 0$, volver al paso b.
 - e. Calcular $s = r_{\text{nonce}}^{-1}(e + k \cdot r) \bmod n$. Si $s = 0$, volver al paso b.
 - f. La firma es (r, s) .
3. **Verificación $\text{Verify}(Q, m, (r, s))$:**
 - a. Calcular $e = \text{hash}(m)$.
 - b. Calcular $u_1 = e \cdot s^{-1} \bmod n$ y $u_2 = r \cdot s^{-1} \bmod n$.
 - c. Calcular $(x_1, y_1) = u_1 \cdot G + u_2 \cdot Q$.
 - d. Aceptar si $r \equiv x_1 \pmod n$.

Maleabilidad inherente. Una propiedad estructural de ECDSA es que si (r, s) es una firma válida sobre m , entonces $(r, -s \bmod n)$ también lo es: ambas verifican porque x_1 depende solo de r . Esta *maleabilidad de firma* obligó a Ethereum a introducir la restricción $s \leq n/2$ (EIP-2) para normalizar el formato. ML-DSA, al ser determinista y al cubrir toda la firma con el hash de desafío, no presenta esta degeneración (véase §2.6).

2.1.3 Recuperación de clave pública (ecrecover)

Una propiedad singular de ECDSA —explotada extensivamente por Ethereum— es la *recuperabilidad*: dada una firma (v, r, s) y el hash del mensaje e , es posible reconstruir la clave pública Q del firmante sin necesidad de transmitirla explícitamente. El byte v (llamado *recovery id*) indica cuál de las (hasta cuatro) posibles claves públicas es la correcta. Esta operación se implementa como el precompilado *ecrecover* en la dirección `0x01` de la EVM.

La dirección Ethereum se deriva entonces como:

$$\text{address} = \text{keccak256}(pk_x || pk_y)[12:32], \quad (2.4)$$

donde pk_x y pk_y son las coordenadas de 32 bytes del punto Q , resultando en una dirección de 20 bytes (los últimos 20 del hash de 32 bytes).

2.2 Computación cuántica

2.2.1 Modelo de circuito cuántico

Definición 2.4 (Qubit). Un *qubit* es un sistema cuántico de dos niveles cuyo estado se describe como un vector unitario en $\mathbb{C}^2$:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1, \quad (2.5)$$

donde $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ y $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ forman la base computacional.

Un sistema de n qubits vive en el espacio de Hilbert $\mathcal{H} = (\mathbb{C}^2)^{\otimes n}$ de dimensión 2^n . Las operaciones se representan como matrices unitarias (*puertas cuánticas*):

- **Puerta Hadamard:** $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$, crea superposición: $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$.
- **Puerta CNOT:** $\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$, crea entrelazamiento.
- **Puerta de fase:** $R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}$, introduce fases relativas.

Al medir un qubit en estado $|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$, se obtiene $|0\rangle$ con probabilidad $|\alpha|^2$ y $|1\rangle$ con probabilidad $|\beta|^2$, colapsando el estado.

2.3 Algoritmo de Shor

El algoritmo de Shor (Shor, 1994) resuelve el problema de la factorización de enteros y el del logaritmo discreto en tiempo polinómico en un computador cuántico. Su aplicación al ECDLP es la amenaza directa a secp256k1.

2.3.1 Estructura del algoritmo

Para resolver el ECDLP ($Q = k \cdot G$), el algoritmo procede en dos fases:

Fase cuántica: Se prepara un estado de superposición uniforme sobre pares (a, b) y se evalúa la función $f(a, b) = a \cdot G + b \cdot Q$ de forma coherente. Tras medir el registro de salida, el registro de entrada colapsa a un *estado de coset*:

$$|\psi\rangle = \frac{1}{\sqrt{n}} \sum_{j=0}^{n-1} |(a_0 + j) \bmod n\rangle |(b_0 - jk) \bmod n\rangle, \quad (2.6)$$

donde (a_0, b_0) es un par que satisface $a_0 \cdot G + b_0 \cdot Q = P_0$ para algún punto fijo P_0 . La clave privada k está «codificada» en la relación entre los incrementos de ambos registros.

Transformada cuántica de Fourier (QFT): Se aplica la QFT bidimensional sobre el grupo $\mathbb{Z}_n \times \mathbb{Z}_n$:

$$\text{QFT}_N |j\rangle = \frac{1}{\sqrt{N}} \sum_{l=0}^{N-1} e^{2\pi i j l / N} |l\rangle. \quad (2.7)$$

Tras la QFT, la medición produce un par (s, t) donde, con alta probabilidad:

$$s + t \cdot k \equiv 0 \pmod{n}. \quad (2.8)$$

Post-procesamiento clásico: Si $\gcd(t, n) = 1$, se recupera directamente:

$$k = -s \cdot t^{-1} \pmod{n}. \quad (2.9)$$

Con múltiples mediciones, la probabilidad de éxito se amplifica arbitrariamente.

2.3.2 Complejidad y recursos

El algoritmo de Shor tiene complejidad $O((\log n)^3)$ en operaciones cuánticas, frente a la complejidad subexponencial $O(\exp(n^{1/3}))$ de los mejores algoritmos clásicos para el logaritmo discreto.

Para secp256k1 ($n \approx 2^{256}$), las estimaciones más recientes (Taguchi & Takayasu, 2025) indican que se necesitarían aproximadamente 2.330 qubits lógicos (sin contar la corrección de errores). Con técnicas de corrección de errores cuánticos, esto se traduce en millones de qubits físicos —un recurso que no está disponible hoy pero que podría estarlo en 10-15 años según las proyecciones actuales.

2.3.3 Simulación sobre curva reducida

En este trabajo se simula el algoritmo de Shor sobre una curva elíptica reducida ($y^2 = x^3 + 7 \pmod{17}$, generador $G = (12, 1)$, orden $n = 18$), suficiente para demostrar la viabilidad del ataque sin requerir un computador cuántico real. La simulación utiliza Qiskit Aer con una DFT modular exacta bidimensional (ver Capítulo 4).

La Figura 2.1 muestra la estructura del circuito cuántico para el algoritmo de Shor aplicado al ECDLP.

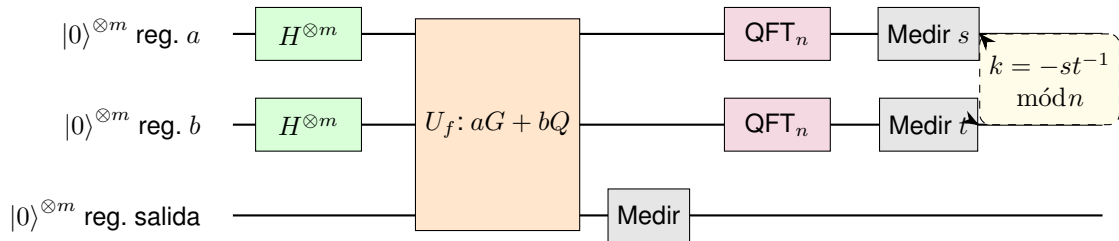


Figura 2.1: Circuito cuántico del algoritmo de Shor para ECDLP. El oráculo U_f evalúa $f(a, b) = aG + bQ$ de forma coherente. Tras la medición del registro de salida y la QFT, los resultados (s, t) permiten recuperar k .

2.4 Algoritmo de Grover

El algoritmo de Grover (Grover, 1996) proporciona una aceleración cuadrática para la búsqueda no estructurada: dado un oráculo $f : \{0, 1\}^n \rightarrow \{0, 1\}$ con M soluciones entre $N = 2^n$ entradas, encuentra una solución en $O(\sqrt{N/M})$ consultas.

2.4.1 Estructura del algoritmo

1. **Inicialización:** Preparar la superposición uniforme

$$|s\rangle = H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle. \quad (2.10)$$

2. **Iteración de Grover:** Repetir $k_{\text{opt}} = \left\lfloor \frac{\pi}{4} \sqrt{N/M} \right\rfloor$ veces:

a. **Oráculo de fase O_f :** Invierte la fase de las soluciones:

$$O_f |x\rangle = (-1)^{f(x)} |x\rangle. \quad (2.11)$$

b. **Operador de difusión $D = 2|s\rangle\langle s| - I$:** Inversión respecto a la media. Amplifica la amplitud de los estados marcados.

3. **Medición:** La probabilidad de medir una solución tras k iteraciones es:

$$P_{\text{éxito}}(k) = \sin^2((2k+1)\theta), \quad \text{donde } \sin \theta = \sqrt{M/N}. \quad (2.12)$$

2.4.2 Implicaciones para funciones hash

Aplicado a la búsqueda de preimágenes, Grover reduce la seguridad efectiva de una función hash de 2^n a $2^{n/2}$. Para Keccak-256 (empleada en Ethereum como keccak256), esto significa:

- Seguridad pre-imagen clásica: 2^{256} operaciones.
- Seguridad pre-imagen cuántica: 2^{128} operaciones (todavía considerado seguro).

Esto justifica que las funciones hash de 256 bits mantengan un nivel de seguridad aceptable frente a ataques cuánticos, a diferencia de los esquemas basados en el logaritmo discreto. No obstante, SHAKE-256 ofrece ventajas adicionales de consistencia con ML-DSA que se detallan en la Sección 2.10.

La Figura 2.2 muestra la estructura del circuito de Grover para búsqueda de preimágenes.

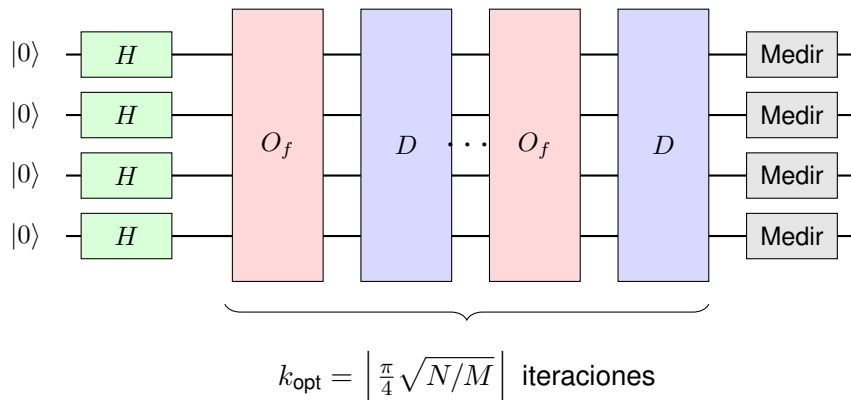


Figura 2.2: Circuito de Grover para $n = 4$ qubits. Cada iteración aplica el oráculo de fase O_f (invierte la fase de las soluciones) y el operador de difusión $D = 2|s\rangle\langle s| - I$ (amplifica la amplitud de los estados marcados).

2.5 Criptografía basada en retículos

Definición 2.5 (Retículo). Un *retículo* $\mathcal{L}$ en $\mathbb{R}^n$ es un subgrupo discreto de $\mathbb{R}^n$ generado por una base $\mathbf{B} = \{\mathbf{b}_1, \dots, \mathbf{b}_n\} \subset \mathbb{R}^n$ de vectores linealmente independientes:

$$\mathcal{L}(\mathbf{B}) = \left\{ \sum_{i=1}^n z_i \mathbf{b}_i : z_i \in \mathbb{Z} \right\}. \quad (2.13)$$

Una propiedad fundamental de los retículos es que admiten infinitas bases distintas: si $\mathbf{B}$ es una base de $\mathcal{L}$ y $\mathbf{U} \in \mathbb{Z}^{n \times n}$ es una matriz unimodular (es decir, $\det(\mathbf{U}) = \pm 1$), entonces $\mathbf{B}' = \mathbf{B}\mathbf{U}$ es también una base del mismo retículo. La calidad de una base (cuán cortos y ortogonales son sus vectores) es lo que determina la dificultad de los problemas computacionales sobre $\mathcal{L}$; el algoritmo LLL (Lenstra et al., 1982) permite encontrar bases reducidas con factor de aproximación $2^{n/2}$ en tiempo polinómico, pero no resuelve los problemas exactos.

El *determinante* (o volumen fundamental) de un retículo se define como $\det(\mathcal{L}) = |\det(\mathbf{B})|$ y es invariante respecto a la elección de base. La cota de Minkowski establece que $\lambda_1(\mathcal{L}) \leq \sqrt{n} \cdot \det(\mathcal{L})^{1/n}$, proporcionando un techo teórico para la longitud del vector más corto.

2.5.1 Problemas computacionales duros

Definición 2.6 (Problema del Vector más Corto (SVP)). Dado un retículo $\mathcal{L}(\mathbf{B})$, encontrar un vector no nulo $\mathbf{v} \in \mathcal{L}$ tal que $\|\mathbf{v}\| = \lambda_1(\mathcal{L})$, donde $\lambda_1(\mathcal{L}) = \min_{\mathbf{v} \in \mathcal{L} \setminus \{0\}} \|\mathbf{v}\|$ es la longitud mínima del retículo.

Definición 2.7 (Problema del Vector más Cercano (CVP)). Dado un retículo $\mathcal{L}(\mathbf{B})$ y un punto objetivo $\mathbf{t} \in \mathbb{R}^n$, encontrar $\mathbf{v} \in \mathcal{L}$ que minimice $\|\mathbf{v} - \mathbf{t}\|$.

Junto a SVP y CVP se manejan varias variantes computacionales relevantes para los esquemas post-cuánticos:

- **γ -SVP (aproximado)**: encontrar $\mathbf{v} \in \mathcal{L} \setminus \{0\}$ con $\|\mathbf{v}\| \leq \gamma \cdot \lambda_1(\mathcal{L})$. Para $\gamma = \text{poly}(n)$ no se conoce algoritmo cuántico polinómico.
- **GapSVP**: problema de decisión asociado: distinguir entre $\lambda_1(\mathcal{L}) \leq 1$ y $\lambda_1(\mathcal{L}) > \gamma$.
- **BDD (Bounded Distance Decoding)**: dado $\mathbf{t} \in \mathbb{R}^n$ con la garantía de que existe $\mathbf{v} \in \mathcal{L}$ a distancia $\leq \alpha \lambda_1(\mathcal{L})$, encontrarlo. Es la variante «promesa» de CVP utilizada implícitamente en la reducción a LWE.
- **SIS (Short Integer Solution)**: dada $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, encontrar $\mathbf{z} \in \mathbb{Z}^m \setminus \{0\}$ con $\mathbf{A}\mathbf{z} \equiv \mathbf{0} \pmod{q}$ y $\|\mathbf{z}\| \leq \beta$. Es la base de las firmas tipo «Fiat-Shamir con abortos», paradigma del que deriva ML-DSA.

Los problemas SVP y CVP exactos son NP-hard. Sus versiones aproximadas con factores polinómicos $\gamma = \text{poly}(n)$ se conjeturan difíciles tanto para computadoras clásicas como cuánticas: el mejor algoritmo conocido (BKZ con *sieving*) tiene complejidad $2^{\Theta(n)}$ en tiempo y memoria, sin que Shor ni Grover aporten una aceleración polinómica como sucede con la factorización o el logaritmo discreto. Esta resistencia cuántica es la base de la seguridad de la criptografía basada en retículos.

La Figura 2.3 ilustra geoméricamente un retículo bidimensional, dos bases distintas (una «buena», casi ortogonal, y una «mala», casi degenerada), y los problemas SVP y CVP.

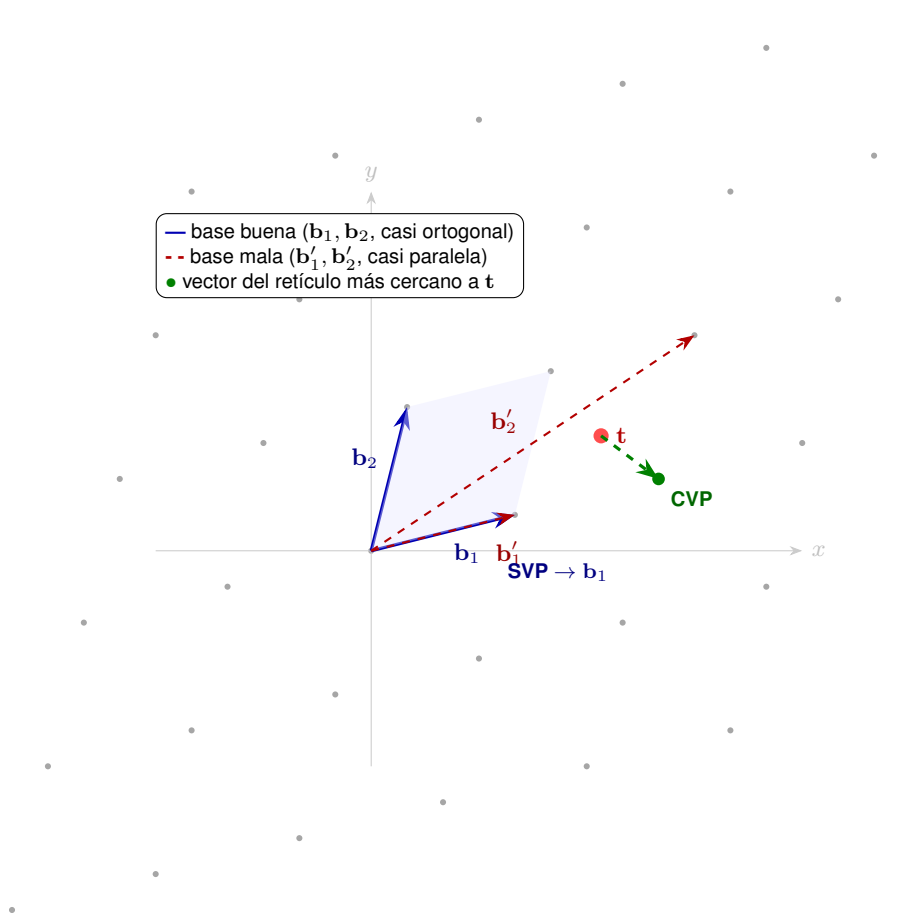


Figura 2.3: Retículo bidimensional $\mathcal{L}(\mathbf{B}) \subset \mathbb{R}^2$. El mismo retículo admite dos bases: una «buena», casi ortogonal (sólida, azul), y otra «mala», casi degenerada (punteada, roja). El problema SVP consiste en encontrar el vector no nulo más corto del retículo; el problema CVP, dado un punto objetivo t (rojo) fuera del retículo, consiste en encontrar el punto del retículo más cercano (verde). La dureza de ambos crece exponencialmente con la dimensión n cuando solo se dispone de una base mala.

2.5.2 Learning With Errors (LWE)

Definición 2.8 (Problema LWE (Regev, 2005)). Sean $n, q \in \mathbb{N}$ con q primo, y sea χ una distribución de error sobre $\mathbb{Z}_q$ (típicamente una distribución gaussiana discreta con parámetro σ). Dado un secreto $s \xleftarrow{\$} \mathbb{Z}_q^n$, el problema LWE consiste en distinguir entre:

- Muestras de la forma $(\mathbf{a}_i, b_i = \langle \mathbf{a}_i, s \rangle + e_i \text{ mód } q)$, donde $\mathbf{a}_i \xleftarrow{\$} \mathbb{Z}_q^n$ y $e_i \leftarrow \chi$.
- Muestras uniformemente aleatorias $(\mathbf{a}_i, u_i)$ con $u_i \xleftarrow{\$} \mathbb{Z}_q$.

Teorema 2.1 (Regev, 2005). Para parámetros apropiados, resolver el problema Decision-LWE es al menos tan difícil como resolver el problema SVP aproximado en retículos de dimensión n en el peor caso (worst-case).

Esta reducción *worst-case to average-case* es una propiedad extraordinaria: la seguridad de cualquier instancia aleatoria de LWE se fundamenta en la dureza del peor caso de un problema de retículos, proporcionando garantías de seguridad mucho más fuertes que las de los esquemas basados en factorización o logaritmo discreto.

2.5.3 Module-LWE

Definición 2.9 (Module-LWE). Sea $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ un anillo de polinomios cociente, con n potencia de 2. El problema Module-LWE opera sobre el módulo R_q^k para un rango k : dado $\mathbf{A} \xleftarrow{\$} R_q^{k \times k}$ y $s \xleftarrow{\$} R_q^k$, distinguir $(\mathbf{A}, \mathbf{A}s + \mathbf{e})$ de $(\mathbf{A}, \mathbf{u})$ para $\mathbf{e} \leftarrow \chi^k$ y $\mathbf{u} \xleftarrow{\$} R_q^k$.

Module-LWE generaliza tanto LWE (caso $n = 1$) como Ring-LWE (caso $k = 1$), ofreciendo un equilibrio entre seguridad y eficiencia. ML-DSA y ML-KEM se basan en Module-LWE.

Aritmética en R_q y la NTT. La operación dominante en ML-DSA es la multiplicación de polinomios en $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ con $n = 256$ y $q = 8,380,417$. Una multiplicación directa cuesta $\Theta(n^2)$, pero el primo q se elige específicamente para que $X^n + 1$ se factorice completamente en R_q , es decir, para que exista una raíz $2n$ -ésima primitiva de la unidad $\zeta \in \mathbb{Z}_q^*$ con $\zeta^n \equiv -1 \pmod{q}$. Bajo esa condición se aplica la *Number Theoretic Transform* (NTT), la análoga modular de la DFT:

$$\hat{a}_j = \sum_{i=0}^{n-1} a_i \zeta^{(2i+1)j} \text{ mód } q, \quad j = 0, \dots, n-1, \quad (2.14)$$

que se calcula en $\Theta(n \log n)$ operaciones modulares. La multiplicación de dos polinomios $a, b \in R_q$ se reduce entonces a una multiplicación componente a componente en el dominio NTT: $a \cdot b = \text{NTT}^{-1}(\text{NTT}(a) \odot \text{NTT}(b))$. Esta optimización es la responsable de que la verificación de ML-DSA-65 ($\sim 42 \mu\text{s}$) sea comparable o incluso más rápida que ECDSA sobre secp256k1 ($\sim 49 \mu\text{s}$).

2.5.4 Justificación de la elección de retículos

Entre las familias de criptografía post-cuántica estandarizadas por NIST, se eligieron los retículos (ML-DSA) frente a las alternativas por las siguientes razones:

Falcon fue descartado pese a sus firmas más compactas porque su implementación segura contra ataques de canal lateral requiere aritmética de punto flotante constante-tiempo, una complejidad significativa para un

Cuadro 2.1: Comparativa de familias PQC para firmas digitales.

Familia	$ pk $ (B)	$ sig $ (B)	Verify (μs)	Observaciones
ML-DSA-65 (retículos)	1.952	3.309	~ 42	FIPS-204. Mejor relación tamaño/rendimiento.
Falcon-512 (NTRU)	897	666	~ 10	Firmas más pequeñas, pero requiere sampling con punto flotante; mayor complejidad de implementación.
SLH-DSA-128s (hash)	32	7.856	~ 500	Claves pequeñas pero firmas enormes y verificación lenta. Sin asunción lattice.

sistema embebido en un cliente blockchain. SLH-DSA fue descartado por el tamaño de firma (>7 KB) y el tiempo de verificación ($>500 \mu s$), incompatibles con la latencia requerida en la validación de bloques.

2.6 ML-DSA (FIPS-204)

ML-DSA (*Module Lattice-Based Digital Signature Algorithm*), estandarizado como FIPS-204 (National Institute of Standards and Technology, 2024a), es el sucesor del candidato CRYSTALS-Dilithium. Opera sobre el anillo $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ con $q = 8,380,417$.

2.6.1 Generación de claves

1. Generar una semilla aleatoria $\xi \xleftarrow{\$} \{0, 1\}^{256}$.
2. Expandir ξ con SHAKE-256 para obtener la matriz $\mathbf{A} \in R_q^{k \times l}$ y los vectores secretos $\mathbf{s}_1 \in R_q^l, \mathbf{s}_2 \in R_q^k$ con coeficientes pequeños ($\|\mathbf{s}_i\|_\infty \leq \eta$).
3. Calcular $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2 \in R_q^k$.
4. La clave pública es $(\mathbf{A}, \mathbf{t})$; la clave privada es $(\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t})$.

Para ML-DSA-65: $k = 6, l = 5, \eta = 4$, resultando en una clave pública de 1.952 bytes.

2.6.2 Firma

1. Calcular el hash del mensaje: $\mu = \text{SHAKE-256}(pk \| m)$.
2. Muestrear un vector de enmascaramiento $\mathbf{y} \leftarrow S_{\gamma_1}^l$ (coeficientes en $[-\gamma_1 + 1, \gamma_1]$).
3. Calcular $\mathbf{w} = \mathbf{A}\mathbf{y}$ y extraer los bits altos: $\mathbf{w}_1 = \text{HighBits}(\mathbf{w})$.
4. Calcular el desafío: $c = H(\mu \| \mathbf{w}_1) \in R_q$ con exactamente τ coeficientes no nulos.
5. Calcular la respuesta: $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$.
6. **Rejection sampling:** Si $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ o la verificación de los bits bajos falla, volver al paso 2.

7. La firma es (z, h, c) , donde h es un *hint* para la descompresión.

El *rejection sampling* es crucial: garantiza que la distribución de z sea independiente de los secretos s_1, s_2 , evitando la fuga de información. La firma es **determinista** (no depende de un nonce aleatorio), eliminando la maleabilidad.

2.6.3 Verificación

1. Recalcular $\mu = \text{SHAKE-256}(pk \| m)$.
2. Calcular $w' = Az - ct$ y extraer: $w'_1 = \text{UseHint}(h, w')$.
3. Verificar: $c \stackrel{?}{=} H(\mu \| w'_1)$ y $\|z\|_\infty < \gamma_1 - \beta$.

2.6.4 Niveles de seguridad

Cuadro 2.2: Parámetros y tamaños de ML-DSA por nivel de seguridad.

Variante	NIST	(k, l)	$ pk $ (B)	$ sig $ (B)	Sign (μs)	Verify (μs)
ML-DSA-44	II	(4, 4)	1.312	2.420	~ 30	~ 30
ML-DSA-65	III	(6, 5)	1.952	3.309	~ 50	~ 42
ML-DSA-87	V	(8, 7)	2.592	4.627	~ 80	~ 62

Se eligió ML-DSA-65 (nivel III, seguridad clásica de 192 bits, cuántica de 128 bits) por ofrecer un margen conservador sin el sobrecoste del nivel V. El nivel II (ML-DSA-44) proporciona solo 128 bits de seguridad clásica, por debajo del margen actual de secp256k1 .

2.7 ML-KEM (FIPS-203)

ML-KEM (*Module Lattice-Based Key-Encapsulation Mechanism*), estandarizado como FIPS-203 (National Institute of Standards and Technology, 2024b) en agosto de 2024, es el sucesor del candidato CRYSTALS-Kyber. A diferencia de ML-DSA (firma digital), ML-KEM es un mecanismo de *encapsulación de claves*: permite a dos partes establecer una clave simétrica compartida sobre un canal inseguro sin requerir un intercambio Diffie-Hellman vulnerable a Shor. Opera sobre el mismo tipo de anillo $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ que ML-DSA, pero con $q = 3,329$ (mucho menor, optimizado para encapsulación rápida).

2.7.1 Generación de claves

1. Generar una semilla $d \xleftarrow{\$} \{0, 1\}^{256}$.
2. Expandir d con SHAKE-128 para obtener la matriz $A \in R_q^{k \times k}$.
3. Muestrear los vectores secretos $s, e \in R_q^k$ desde una distribución binomial centrada B_η (coeficientes pequeños).
4. Calcular $t = As + e \in R_q^k$.
5. La *clave de encapsulación* es $ek = (A, t)$; la *clave de desencapsulación* es $dk = (s, ek, H(ek), z)$, donde z es una semilla para el rechazo implícito.

2.7.2 Encapsulación

Dado ek del receptor, el emisor calcula $(ct, K) = \text{Encaps}(ek)$:

1. Generar un mensaje aleatorio $m \xleftarrow{\$} \{0, 1\}^{256}$.
2. Derivar $(K, r) = G(m \| H(ek))$, donde G es SHA3-512.
3. Muestrear $\mathbf{r} \in R_q^k$ y errores $\mathbf{e}_1 \in R_q^k, \mathbf{e}_2 \in R_q$ desde B_η usando r como semilla.
4. Calcular $\mathbf{u} = \mathbf{A}^\top \mathbf{r} + \mathbf{e}_1$ y $v = \mathbf{t}^\top \mathbf{r} + e_2 + \text{Compress}_q(m, 1)$.
5. El *ciphertext* es $ct = (\text{Compress}_q(\mathbf{u}, d_u), \text{Compress}_q(v, d_v))$.
6. La clave compartida es $K \in \{0, 1\}^{256}$.

La compresión (descarte de bits bajos) es esencial: reduce el tamaño del ciphertext al precio de introducir un pequeño error adicional que la corrección de descodificación de LWE absorbe.

2.7.3 Desencapsulación

El receptor, conocedor de dk , recupera la clave compartida $K = \text{Decaps}(dk, ct)$:

1. Descomprimir $\mathbf{u}'$ y v' a partir de ct .
2. Calcular $m' = \text{Compress}_q(v' - \mathbf{s}^\top \mathbf{u}', 1)$.
3. Derivar $(K', r') = G(m' \| H(ek))$ y recomputar el ciphertext esperado ct' .
4. Si $ct = ct'$, devolver K' ; en caso contrario, devolver $K_{\text{rej}} = J(z \| ct)$ (*rechazo implícito*: el receptor devuelve una clave pseudoaleatoria indistinguible para evitar ataques de oráculo de descodificación).

El rechazo implícito convierte el esquema CPA-seguro subyacente (Kyber.CPAPKE) en un KEM IND-CCA2 mediante la transformación de Fujisaki-Okamoto, sin requerir abortar la sesión.

2.7.4 Niveles de seguridad

Cuadro 2.3: Parámetros y tamaños de ML-KEM por nivel de seguridad.

Variante	NIST	k	$ ek $ (B)	$ dk $ (B)	$ ct $ (B)
ML-KEM-512	I	2	800	1.632	768
ML-KEM-768	III	3	1.184	2.400	1.088
ML-KEM-1024	V	4	1.568	3.168	1.568

La clave compartida K siempre tiene 32 bytes (256 bits), independiente del nivel: ML-KEM solo se usa para acordar una semilla simétrica que luego alimenta a un cifrado de flujo (AES-256-GCM o ChaCha20-Poly1305).

Rol en este trabajo. ML-KEM no se utiliza en la capa de transacciones (donde ML-DSA-65 cubre la autenticación), sino que se reserva como primitiva para el *handshake* del canal P2P entre nodos. La implementación reside en el módulo `ml-lattice-rs/kyber/` (envoltorio del `crate ml-kem` de RustCrypto, conforme a FIPS-203) y expone una API ergonómica con los tres niveles de seguridad. Su despliegue activo en el protocolo de red queda como trabajo futuro (§6.4), ya que el prototipo actual ejecuta los nodos en una red privada controlada donde el riesgo de *man-in-the-middle* cuántico no es el factor crítico de validación.

2.8 Ethereum y la Máquina Virtual de Ethereum (EVM)

Ethereum (Wood, 2014) es una máquina de estados replicada y determinista en la que cada nodo de la red mantiene una copia consistente del mismo *estado global* y aplica sobre él una función de transición Υ cada vez que se ejecuta una transacción. Formalmente, si σ_t es el estado tras el bloque t y B_{t+1} es el siguiente bloque (una secuencia ordenada de transacciones $T_0, T_1, \dots, T_{m-1}$), la regla de transición es

$$\sigma_{t+1} = \Pi(\sigma_t, B_{t+1}) = \Omega(\Upsilon(\Upsilon(\dots \Upsilon(\sigma_t, T_0) \dots, T_{m-2}), T_{m-1}), B_{t+1}), \quad (2.15)$$

donde Ω aplica la finalización del bloque (mint del bloque, pagos de comisiones, etc.). El protocolo garantiza, mediante el consenso, que todos los nodos honestos converjan al mismo σ_{t+1} a partir de σ_t y B_{t+1} .

2.8.1 Arquitectura de capas

Desde *The Merge* (septiembre de 2022), Ethereum opera con una arquitectura de dos capas:

- **Capa de ejecución (EL):** Procesa transacciones, ejecuta contratos inteligentes, mantiene el estado de cuentas y contratos. Implementada por clientes como geth, Nethermind, Besu, Erigon y reth.
- **Capa de consenso (CL):** Coordina los validadores Proof of Stake, gestiona las atestaciones y propuestas de bloques. Utiliza firmas BLS12-381.

Ambas capas se comunican mediante la *Engine API* (JSON-RPC privado). Este trabajo modifica exclusivamente la capa de ejecución, dejando la capa de consenso como trabajo futuro (ver Sección 6.4).

2.8.2 Modelo de cuentas y estado mundial

A diferencia de Bitcoin (modelo UTXO), Ethereum emplea un modelo de cuentas basado en balances. El *estado mundial* σ es una aplicación finita

$$\sigma : \mathcal{A} \rightarrow \mathcal{A}_{\text{state}}, \quad (2.16)$$

donde $\mathcal{A} = \{0, 1\}^{160}$ es el espacio de direcciones de 20 bytes y cada cuenta $\sigma[a]$ es una tupla de cuatro campos

$$\sigma[a] = (\text{nonce}, \text{balance}, \text{storageRoot}, \text{codeHash}). \quad (2.17)$$

Existen dos tipos de cuenta funcionalmente distintos:

- **Cuentas de propiedad externa (EOA):** controladas por una clave privada (en Ethereum clásico, una clave ECDSA secp256k1; en este trabajo, una clave ML-DSA-65). Su `codeHash` es el hash vacío `keccak256(ε)` y su `storageRoot` es la raíz del trie vacío. Solo pueden originar transacciones firmando con la clave privada.

- **Cuentas de contrato:** creadas por una transacción de despliegue. Su `codeHash` apunta al bytecode EVM almacenado y su `storageRoot` es la raíz de un Merkle Patricia Trie que mapea claves de 256 bits a valores de 256 bits. No tienen clave privada y solo pueden ejecutarse como respuesta a un mensaje (interno o externo).

2.8.3 Merkle Patricia Trie

El estado, los recibos y las transacciones de cada bloque se compactan en raíces criptográficas mediante el *Modified Merkle Patricia Trie* (MPT), una estructura híbrida entre un *radix trie* y un árbol de Merkle. Cada nodo se referencia por el hash de su codificación RLP, lo que confiere dos propiedades clave:

- **Compromiso criptográfico:** la raíz `stateRoot` del bloque condensa todo el estado en 32 bytes; cualquier modificación de una sola cuenta propaga un cambio determinista hasta la raíz.
- **Pruebas de inclusión sucintas:** un cliente ligero puede verificar el valor $\sigma[a]$ a partir de `stateRoot` y una prueba de tamaño $O(\log_{16} |\sigma|)$.

La cabecera del bloque contiene tres raíces MPT distintas:

$$H = (\dots, \text{stateRoot}, \text{txRoot}, \text{receiptsRoot}, \text{logsBloom}, \dots). \quad (2.18)$$

En este trabajo, las funciones hash empleadas por el MPT siguen siendo Keccak-256 por compatibilidad con el formato de bloque estándar de Ethereum: la separación SHAKE-256 / Keccak-256 se aplica únicamente a la derivación de direcciones y al hash de firma PQ, no al estado interno del nodo.

2.8.4 La EVM como máquina de pila

La EVM es una máquina virtual basada en pila con cinco regiones de memoria diferenciadas, todas operando sobre palabras de 256 bits:

1. **Pila** (*stack*): hasta 1024 elementos de 256 bits. Las instrucciones operan principalmente sobre los elementos superiores (push/pop, dup, swap).
2. **Memoria** (*memory*): array de bytes lineal expandible. Volátil entre llamadas. Coste de gas cuadrático en el peor caso para penalizar consumo de RAM en los nodos.
3. **Almacenamiento** (*storage*): mapa $\{0, 1\}^{256} \rightarrow \{0, 1\}^{256}$ persistente y específico de cada contrato; reside en el MPT de la cuenta.
4. **Calldata:** datos de entrada inmutables provistos por la transacción.
5. **Returndata:** datos retornados por la última llamada externa.

La ejecución de una transacción T que invoca a una cuenta de contrato a es un bucle interpretado sobre el bytecode $\sigma[a].\text{code}$. Cada *opcode* (ADD, MUL, SSTORE, CALL, ...) consume una cantidad de gas predefinida; cuando el gas se agota antes de que la ejecución termine, todos los cambios de estado se revierten (excepto el cobro de la comisión al firmante). Si la ejecución concluye, los cambios de σ se materializan atómicamente.

2.8.5 Modelo de mensajes y contratos inteligentes

Un *mensaje* (en la terminología del *yellow paper*) es la unidad de invocación interna a la EVM: una tupla (*sender, recipient, value, input, gas*). Las transacciones externas son simplemente mensajes firmados por una EOA. Los mensajes internos se generan mediante los opcodes CALL, CALLCODE, DELEGATECALL y STATICCALL, lo que permite la composición arbitraria de contratos: un contrato puede llamar a otro, este a un tercero, y así recursivamente hasta la profundidad límite de 1024.

Un *contrato inteligente* es, formalmente, una cuenta cuyo codeHash apunta a bytecode no vacío. La ABI (*Application Binary Interface*) estándar codifica las invocaciones de funciones mediante un selector de 4 bytes (los primeros 4 bytes de keccak256(signature_{Solidity})) seguido de los argumentos codificados según las reglas ABI. El despacho a la función correcta es responsabilidad del propio bytecode del contrato, no del protocolo.

El ciclo de vida típico de un contrato es:

1. **Despliegue:** una transacción con $T.to = \emptyset$ y $T.input$ conteniendo el *init code* crea una nueva cuenta. La dirección se deriva como keccak256(rlp(*sender, nonce*))[12:32] (o por CREATE2, como keccak256(0xff||*sender*||*salt*||keccak256(*init*))[12:32]).
2. **Inicialización:** la EVM ejecuta el init code, cuyo valor de retorno se almacena como bytecode definitivo del contrato.
3. **Invocación:** cualquier mensaje posterior con $T.to = a$ ejecuta el bytecode con el calldata como entrada.
4. **Autodestrucción** (SELFDESTRUCT, restringido desde EIP-6780): elimina el contrato y reembolsa su balance a una dirección beneficiaria.

2.8.6 Formalización como autómata finito determinista

Aunque las descripciones anteriores siguen el estilo informal del *Yellow Paper*, la EVM admite una formalización rigurosa como autómata finito determinista extendido (DFA con memoria auxiliar). Esta formalización conecta el diseño de la EVM con la teoría clásica de autómatas y lenguajes formales, y proporciona un marco para razonar sobre la corrección de las transiciones que introduce la migración post-cuántica (nuevo opcode PQHASH, precompilado 0x0100, transacciones tipo 0x50).

Definición 2.10 (EVM como autómata). La Máquina Virtual de Ethereum se modela como la quintupla

$$\mathcal{M}_{\text{EVM}} = (Q, \Sigma, \delta, q_0, F), \quad (2.19)$$

donde:

- $Q = \text{PC} \times \text{Stack}^{\leq 1024} \times \text{Mem} \times \text{Storage} \times \mathbb{N}_{\text{gas}} \times \text{Calldata} \times \text{Returndata}$ es el conjunto de *configuraciones*. Cada $q \in Q$ codifica la totalidad del estado de ejecución: contador de programa, pila (hasta 1024 palabras de 256 bits), memoria volátil, almacenamiento persistente del contrato, gas restante, datos de entrada y datos de retorno de la última llamada.
- $\Sigma = \{0x00, 0x01, \dots, 0xFF\}$ es el *alfabeto* de instrucciones (256 opcodes posibles, de los cuales aproximadamente 142 están definidos en la especificación actual; los restantes son INVALID).

- $\delta : Q \times \Sigma \rightarrow Q$ es la *función de transición*: dado un estado q y el opcode op apuntado por $q.PC$, $\delta(q, op)$ produce la siguiente configuración aplicando el efecto del opcode sobre la pila, la memoria, el almacenamiento, y decrementando $q.gas$ en el coste correspondiente. La función es *determinista*: dada una misma (q, op) , el resultado es único y reproducible en todos los nodos honestos.
- $q_0 = (PC = 0, \varepsilon, 0, \sigma[a].storage, g_0, calldata, \varepsilon)$ es el *estado inicial* de cada invocación: contador a cero, pila vacía, memoria a cero, almacenamiento heredado de la cuenta a , gas igual al *gas_limit* concedido, calldata provista por la transacción y returndata vacía.
- $F = \{STOP, RETURN, REVERT, INVALID, OOG\}$ es el conjunto de *estados terminales*. STOP y RETURN representan finalización exitosa (los cambios de σ se materializan); REVERT indica error explícito (los cambios se descartan, el gas no consumido se devuelve); INVALID corresponde a una instrucción no reconocida o violación de invariantes (todo el gas se consume); OOG (*out of gas*) se alcanza cuando $q.gas < cost(op)$.

La función de transición δ admite una descomposición canónica que ilustra su naturaleza:

$$\delta(q, op) = \begin{cases} OOG & \text{si } q.gas < cost(op), \\ INVALID & \text{si } op \notin \Sigma_{def}, \\ \delta_{op}(q) & \text{en caso contrario,} \end{cases} \quad (2.20)$$

donde $\Sigma_{def} \subset \Sigma$ es el subconjunto de opcodes definidos y δ_{op} es la regla específica del opcode (por ejemplo, δ_{ADD} extrae los dos elementos superiores de la pila, los suma módulo 2^{256} , y empuja el resultado).

Extensión con el opcode PQHASH. La introducción del opcode $0x21$ (PQHASH) en este trabajo equivale a extender el alfabeto Σ_{def} en una unidad y añadir una nueva regla δ_{PQHASH} que aplica SHAKE-256 sobre un fragmento de memoria. Esta extensión preserva el determinismo de δ y la naturaleza finita de Σ , por lo que el autómata resultante es estructuralmente equivalente al original: las propiedades de corrección y de convergencia del consenso se mantienen.

Diagrama de estados. La Figura 2.4 representa gráficamente las clases de estados y las transiciones agregadas del autómata. Los estados internos durante la ejecución se agrupan en el nodo EJECUTANDO, cuyas autotransiciones modelan la evolución por aplicación sucesiva de δ . Las transiciones a estados terminales determinan el destino final de la ejecución (commit o rollback).

Este modelado conecta directamente el diseño del sistema con la teoría de autómatas finitos deterministas y lenguajes formales abordada en la titulación (RA RPFA-HABI-12 y RPFA-COMP-12, ver anexo de Resultados de Aprendizaje). En particular, demuestra que la EVM no es una máquina arbitrariamente compleja, sino un autómata cuyo alfabeto, conjunto de configuraciones, función de transición y conjunto de estados terminales pueden enumerarse exhaustivamente; cualidad imprescindible para que cualquier extensión post-cuántica preserve la propiedad de *determinismo replicado* de la que depende el consenso.

2.8.7 Cómo la migración post-cuántica preserva los contratos

Un aspecto crítico para la viabilidad del sistema propuesto es que la compatibilidad de contratos inteligentes se mantenga. Los contratos existentes interactúan con la EVM a través de cuatro mecanismos:

- **Opcodes aritméticos, lógicos y de control de flujo** (ADD, MUL, JUMPI, ...): independientes del modelo criptográfico subyacente; no requieren cambios.

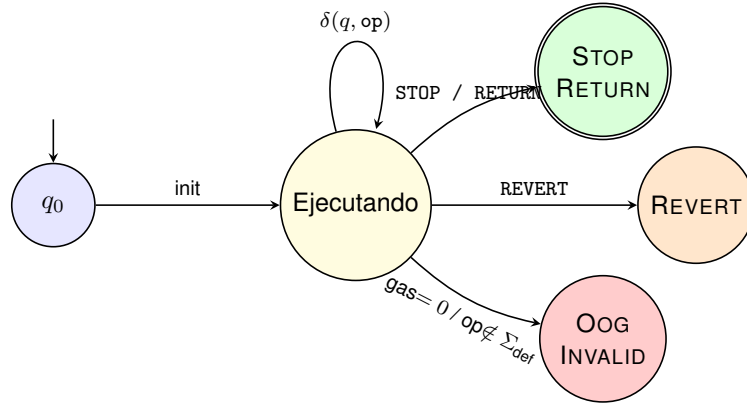


Figura 2.4: Autómata finito determinista $\mathcal{M}_{\text{EVM}}$. La ejecución comienza en q_0 , evoluciona por aplicaciones sucesivas de δ dentro del estado agregado EJECUTANDO (cada opcode consume gas y modifica la configuración), y converge a uno de los estados terminales: STOP/RETURN comprometen los cambios al estado mundial; REVERT los descarta pero devuelve gas; OOG/INVALID descartan los cambios y consumen todo el gas.

- **Opcodes de hashing** (KECCAK256/SHA3): preservados, garantizando que las claves de mapping, los selectores ABI y los event topics de contratos existentes sigan calculándose igual. Adicionalmente, este trabajo introduce el opcode PQHASH (0x21, ver §4.4.1) para exponer SHAKE-256 a contratos PQ-nativos.
- **Opcodes de mensajes y direcciones** (CALL, ADDRESS, CALLER, BALANCE, etc.): operan sobre direcciones de 20 bytes; tanto las direcciones clásicas como las PQ tienen el mismo tamaño, por lo que los contratos no necesitan reconocer el origen criptográfico de la cuenta.
- **Precompilados:** ecrecover (0x01) se mantiene operativo durante el periodo de transición para verificar firmas históricas; las nuevas firmas se verifican mediante el precompilado 0x0100 (ver §4.4.2). Los precompilados de BN254, KZG y BLS12-381 se desactivan en este prototipo por ser cuánticamente vulnerables (ver §4.4.3).

Este principio de *compatibilidad de aplicación*, junto con la separación de dominios entre direcciones clásicas y PQ, permite que el ecosistema de contratos existente (DEXes, stablecoins, NFTs, etc.) coexista con cuentas post-cuánticas durante toda la ventana de migración descrita en §3.1.

2.8.8 Transacciones tipadas (EIP-2718)

EIP-2718 (Zoltu, 2020) define un formato de sobre (*envelope*) para transacciones:

$$\text{TransactionType} \parallel \text{TransactionPayload}, \quad (2.21)$$

donde TransactionType es un byte en el rango $[0x00, 0x7f]$ y $\text{TransactionPayload}$ es un array de bytes cuya codificación depende del tipo. Los tipos asignados actualmente son:

El uso del prefijo 0x50 (en lugar de un valor secuencial en el rango 0x05–0x4f) es una decisión deliberada: el nibble alto 5 actúa como mnemónico de *post-quantum* y deja espacio para que el proceso de estandarización de Ethereum asigne tipos clásicos sin colisión.

Cuadro 2.4: Tipos de transacción Ethereum asignados (mayo 2026).

Tipo	EIP	Descripción
0x00	Legacy	Transacciones pre-EIP-2718
0x01	EIP-2930	Access list
0x02	EIP-1559	Comisiones dinámicas (base fee + priority fee)
0x03	EIP-4844	Blob transactions (proto-danksharding)
0x04	EIP-7702	Delegación de código para EOAs
0x50	Este trabajo	Transacción post-cuántica ML-DSA-65

2.8.9 Precompilados

Los precompilados son contratos con direcciones fijas cuya lógica está implementada de forma nativa en el cliente de ejecución (no en bytecode EVM), proporcionando operaciones criptográficas eficientes. Los precompilados relevantes para este trabajo son:

- 0x01 (ecrecover): Recuperación de clave pública ECDSA. **Vulnerable** a Shor.
- 0x02 (SHA-256), 0x03 (RIPEMD-160), 0x09 (Blake2f): Funciones hash. Seguras frente a ataques cuánticos (Grover solo reduce la seguridad a la mitad).
- 0x06–0x08 (BN254): Aritmética de curva para verificación zk-SNARK. **Vulnerable**.
- 0x0a (KZG): Evaluación de puntos para EIP-4844. **Vulnerable**.
- 0x0b–0x13 (BLS12-381): Nueve operaciones para firmas BLS. **Vulnerable**.

2.8.10 Modelo de comisiones EIP-1559

Desde la actualización London (agosto 2021), Ethereum emplea un modelo de comisiones con una *base fee* (ajustada algorítmicamente por bloque y quemada) y una *priority fee* (propina al validador). Este modelo se preserva a nivel de protocolo en la implementación de este trabajo, aunque el tipo de transacción 0x50 utiliza un campo `gas_price` simplificado por razones de alcance (ver Sección 3.4).

2.9 Reth como cliente de ejecución

Reth (Paradigm, 2024) es un cliente de ejecución Ethereum implementado en Rust por Paradigm. Se eligió como base para este trabajo por tres razones fundamentales:

1. **Seguridad de memoria:** Rust garantiza ausencia de *data races* y errores de acceso a memoria en tiempo de compilación, una propiedad crítica para software criptográfico.
2. **Modularidad:** Reth está organizado en más de 100 *crates* independientes con interfaces definidas por *traits*. Los componentes clave para este trabajo son:
 - Primitives: Tipos de transacción, bloque, header.
 - Consensus: Validación de bloques y transacciones.
 - Network: Descubrimiento de pares y propagación de bloques.

- EVM: Ejecución de transacciones y gestión de precompilados.
- Pool: Mempool de transacciones pendientes.

Cada uno de estos componentes se extiende con un *crate* PQ correspondiente (ver Capítulo 4).

3. **Rendimiento:** Reth utiliza paralelismo extensivo (rayon), I/O mapeado en memoria (MDBX), y un *pipeline* de sincronización por etapas que aprovecha múltiples núcleos.

2.10 SHAKE-256 y la familia SHA-3

2.10.1 Construcción esponja

La familia SHA-3 (Keccak) se basa en la construcción esponja (*sponge construction*), que opera sobre un estado interno de $b = 1600$ bits organizado como una matriz de $5 \times 5 \times 64$ bits.

Definición 2.11 (Construcción esponja). Una función esponja con permutación $f : \{0, 1\}^b \rightarrow \{0, 1\}^b$, tasa r y capacidad c (con $b = r + c$) opera en dos fases:

1. **Absorción:** El mensaje M se divide en bloques de r bits. Cada bloque se XOR con los primeros r bits del estado, seguido de una aplicación de f .
2. **Extracción (*squeezing*):** Se extraen los primeros r bits del estado. Si se necesitan más bits de salida, se aplica f y se extraen otros r bits, iterativamente.

La Figura 2.5 ilustra la construcción esponja.

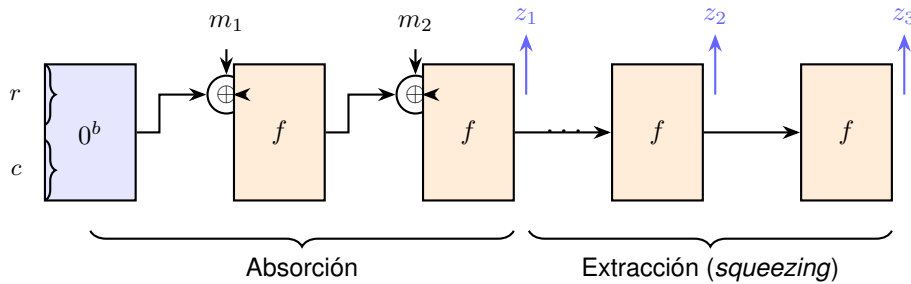


Figura 2.5: Construcción esponja: los bloques del mensaje m_i se absorben mediante XOR con la tasa r del estado seguido de la permutación f . En la fase de extracción, se extraen r bits por aplicación de f . La capacidad c nunca se expone directamente.

La permutación Keccak- $f[1600]$ consta de 24 rondas, cada una compuesta por cinco pasos: θ (paridad de columnas), ρ (rotación de bits), π (permutación de posiciones), χ (operación no lineal: $A_i \leftarrow A_i \oplus (\neg A_{i+1} \wedge A_{i+2})$) e ι (adición de constante de ronda).

2.10.2 SHAKE-256 como XOF

SHAKE-256 es una *Extendable Output Function* (XOF): a diferencia de SHA3-256 que produce exactamente 256 bits, SHAKE-256 puede producir una salida de longitud arbitraria. Emplea una capacidad $c = 512$ bits (tasa $r = 1088$ bits), proporcionando:

- Seguridad de preimagen: $\min(d/2, 256)$ bits, donde d es la longitud de salida.

- Seguridad de colisión: $\min(d/2, 256)$ bits.

Para $d = 256$ bits (el caso de uso en este trabajo), la seguridad es de 128 bits tanto para preimagen como para colisión en el modelo clásico, y de 64/128 bits respectivamente con Grover —suficiente para las necesidades actuales.

2.10.3 Justificación del uso de SHAKE-256

Se elige SHAKE-256 en lugar de Keccak-256 (empleada nativamente por Ethereum) por tres razones:

1. **Consistencia criptográfica:** ML-DSA-65 utiliza internamente SHAKE-256 para la expansión de la matriz A , el muestreo de vectores secretos y el cálculo del desafío. Emplear la misma primitiva para la derivación de direcciones simplifica la superficie de dependencias criptográficas.
2. **Separación de dominios:** Al usar SHAKE-256 en lugar de Keccak-256, las direcciones post-cuánticas ($\text{SHAKE-256}(pk)[12:32]$) pertenecen a un espacio de salida diferente al de las direcciones clásicas ($\text{Keccak-256}(pk)[12:32]$), eliminando cualquier posibilidad de confusión o colisión intencionada.
3. **Flexibilidad de salida:** La propiedad XOF permite adaptar la longitud de salida si futuros estándares requieren direcciones más largas, sin cambiar la primitiva subyacente.

3 ANÁLISIS DE LA SITUACIÓN

Este capítulo traduce los fundamentos teóricos del Capítulo 2 en un análisis concreto de las vulnerabilidades cuánticas de Ethereum, define los requisitos funcionales y no funcionales del sistema a desarrollar, presenta la arquitectura de la solución propuesta y justifica las alternativas descartadas durante el proceso de diseño.

3.1 Descripción detallada del problema

3.1.1 Superficie de ataque cuántico en Ethereum

El análisis de los puntos de vulnerabilidad identificados en la Sección 1.2 (P1–P5) revela que la exposición cuántica de Ethereum no se limita a un componente aislado, sino que afecta a múltiples capas del protocolo de forma simultánea. La Tabla 3.1 cuantifica esta superficie.

Cuadro 3.1: Superficie de ataque cuántico en Ethereum por componente.

Componente	Primitiva vulnerable	Algoritmo cuántico	Impacto
Firma de transacciones	ECDSA / secp256k1	Shor (ECDLP)	Falsificación de firmas, robo de fondos
<code>ecrecover</code> (0x01)	ECDSA recovery	Shor (ECDLP)	Suplantación de identidad on-chain
BN254 (0x06–0x08)	Emparejamientos bilineales	Shor (DLP)	Rotura de verificaciones zk-SNARK
KZG (0x0a)	Compromisos elípticos	Shor (DLP)	Invalidación de blob transactions
BLS12-381 (0x0b–0x13)	Firmas BLS	Shor (DLP)	Rotura de atestaciones PoS
Consenso PoS (CL)	BLS12-381 aggregation	Shor (DLP)	Control total de la cadena

Cada cuenta Ethereum que ha emitido al menos una transacción tiene su clave pública expuesta en la blockchain —recuperable a partir de la firma (v, r, s) mediante `ecrecover`—. Un adversario con un CRQC podría, para cada una de estas cuentas, ejecutar el algoritmo de Shor (Ecuación 2.9) y obtener la clave privada correspondiente, ganando control total sobre los fondos y contratos asociados.

Las cuentas que *nunca* han emitido una transacción (solo han recibido fondos) mantienen su clave pública oculta tras el hash de la dirección (`keccak256(pk)[12:32]`), protegidas por la resistencia a preimagen de Keccak-256. Sin embargo, esta protección se pierde irremediabilmente en el momento en que la cuenta firma su primera transacción.

3.1.2 Análisis temporal: CRQC vs. complejidad de migración

La amenaza cuántica presenta una asimetría temporal crítica. Por un lado, las estimaciones de disponibilidad de un CRQC se sitúan en un horizonte de 10 a 15 años (National Institute of Standards and Technology, 2024c). Por otro lado, una migración criptográfica completa de Ethereum requiere múltiples fases:

1. **Investigación y estandarización** (1–3 años): Diseño de nuevos tipos de transacción, selección de algoritmos, propuestas EIP, revisión por la comunidad.
2. **Implementación en clientes** (1–2 años): Modificación de todos los clientes de ejecución (geth, Nethermind, Besu, Erigon, reth) y consenso (Prysm, Lighthouse, Teku, Nimbus, Lodestar).
3. **Testnet y auditoría** (1–2 años): Despliegue en redes de prueba, auditorías de seguridad, corrección de vulnerabilidades.
4. **Activación en mainnet** (6–12 meses): Hard fork coordinado, período de transición con coexistencia de transacciones clásicas y PQ.
5. **Migración de usuarios** (2–5+ años): Los usuarios deben migrar voluntariamente sus fondos a cuentas PQ. Los fondos en cuentas clásicas permanecen vulnerables.

El tiempo total estimado de migración es de 6 a 13 años, solapándose peligrosamente con las proyecciones de disponibilidad de un CRQC. Además, el modelo de amenaza «harvest now, decrypt later» implica que las transacciones firmadas hoy estarán expuestas retroactivamente cuando un CRQC esté disponible —las claves públicas ya son visibles en el registro inmutable de la blockchain.

3.1.3 Escenarios de fallo

Se identifican tres escenarios de fallo por gravedad creciente:

Escenario 1 — Robo selectivo de fondos: Un adversario con acceso limitado a un CRQC prioriza las cuentas con mayores saldos. Dado que las claves públicas de todas las cuentas activas son públicas, el atacante puede calcular las claves privadas correspondientes mediante el algoritmo de Shor y transferir los fondos. Este escenario es económicamente devastador pero no compromete la integridad del protocolo.

Escenario 2 — Rotura de verificaciones on-chain: Si los precompilados BN254 y BLS12-381 son vulnerables, un adversario puede falsificar pruebas zk-SNARK y firmas BLS utilizadas en contratos inteligentes (puentes cross-chain, protocolos de privacidad, verificación de rollups). Esto compromete la integridad de las aplicaciones descentralizadas sin necesidad de atacar cuentas individuales.

Escenario 3 — Toma de control del consenso: Si el adversario compromete las claves BLS12-381 de un tercio o más de los validadores de la capa de consenso, puede impedir la finalización de bloques. Si compromete dos tercios, puede reescribir la cadena, realizar doble gasto y censurar transacciones arbitrariamente. Este es el escenario catastrófico: la red pierde todas sus propiedades de seguridad.

El presente trabajo aborda los Escenarios 1 y 2 mediante la migración de la capa de ejecución a ML-DSA-65, la deshabilitación de precompilados vulnerables, y la introducción de primitivas PQ nativas. El Escenario 3 (capa de consenso) queda fuera del alcance, ya que requiere un esquema de agregación de firmas post-cuánticas que no existe de forma estandarizada (ver Sección 3.4.4).

3.1.4 Plan de migración para usuarios

El protocolo, por muy completo que sea, no es operativo sin un *plan de migración* viable para los miles de millones de cuentas Ethereum existentes. Esta sección formaliza el procedimiento que un usuario o aplicación seguiría para abandonar una cuenta clásica (vulnerable a Shor) y operar exclusivamente desde una cuenta post-cuántica. El procedimiento se diseña asumiendo dos invariantes:

1. **Coexistencia durante la transición:** ambos formatos de transacción (0x02 clásica y 0x50 PQ) se aceptan simultáneamente durante la ventana de migración, evitando un *flag day* disruptivo.
2. **Migración voluntaria pero incentivada:** tras un periodo de gracia (estimado entre 2 y 5 años post-fork), las transacciones 0x02 se rechazarían, presionando a los rezagados a migrar antes de que un CRQC esté operativo.

El flujo, ilustrado en la Figura 3.1, consta de cinco etapas:

1. **Generación del par PQ:** el usuario invoca `pq-wallet new` (CLI) o el equivalente en la TUI; el sistema genera (sk_{PQ}, pk_{PQ}) mediante ML-DSA-65 y deriva la dirección PQ como $SHAKE-256(pk_{PQ})[12:32]$. La clave privada se almacena cifrada con AES-256-GCM en un keystore JSON local.
2. **Anuncio de la nueva cuenta:** opcionalmente, el usuario publica su nueva dirección PQ asociándola a un identificador legible (por ejemplo, mediante ENS o una transacción etiquetada). Esto facilita que terceros (intercambios, contactos, contratos) reconozcan el destino correcto.
3. **Transferencia puente:** una transacción 0x02 clásica firmada con la antigua clave ECDSA transfiere todos los fondos relevantes (ETH nativo y tokens ERC-20) desde la cuenta clásica A a la nueva cuenta PQ A' . Es la *última* firma ECDSA que el usuario produce; cualquier filtración futura de su clave clásica solo expondrá un saldo residual nulo.
4. **Activación operativa:** desde ese momento, todas las transacciones nuevas (transferencias, interacciones con contratos, despliegues) se firman exclusivamente con sk_{PQ} y se envían en formato 0x50. La cuenta clásica A se considera *quemada*.
5. **Cierre del periodo de gracia:** pasado el plazo definido en el hard fork, los nodos rechazan transacciones 0x02 en el *mempool* y, simultáneamente, los exploradores marcan visualmente las cuentas clásicas no migradas como «en riesgo cuántico». Los fondos remanentes en cuentas clásicas no migradas dejan de ser movibles vía el protocolo estándar, salvo a través de un mecanismo de rescate específico (por ejemplo, una EIP de *recovery* que aquí queda como trabajo futuro).

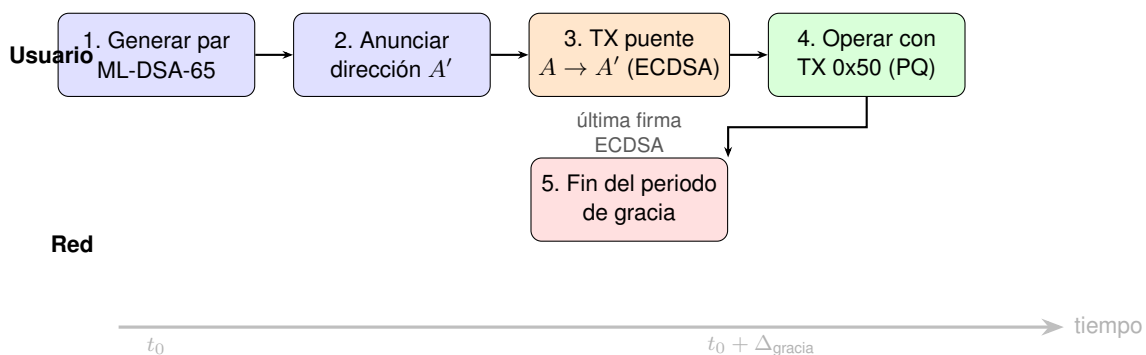


Figura 3.1: Flujo de migración de un usuario desde una cuenta clásica A a una cuenta post-cuántica A' . La transacción puente (paso 3) es la última firma ECDSA que el usuario produce; los pasos 1, 2 y 4 son completamente PQ. El paso 5 lo dispara el protocolo, no el usuario.

Este procedimiento es **lineal en la base de usuarios**: cada cuenta migra una sola vez, sin coordinación entre cuentas. Para contratos inteligentes, el patrón equivalente es la *cesión de propiedad* a través de funciones `transferOwnership(...)` ya estándar en OpenZeppelin: el propietario clásico ejecuta una última transacción 0x02 para asignar la propiedad a su nueva cuenta PQ. Las DEX, oráculos y multisigs requieren una migración análoga, generalmente coordinada por sus DAO mediante propuestas de gobernanza.

3.2 Requisitos

A partir del análisis del problema y los objetivos definidos en la Sección 1.3, se derivan los siguientes requisitos funcionales y no funcionales.

3.2.1 Requisitos funcionales

- RF1: Simulación cuántica de Shor (ECDLP).** El sistema debe incluir una simulación del algoritmo de Shor aplicado al problema del logaritmo discreto en curvas elípticas sobre un modelo reducido ($y^2 = x^3 + 7 \bmod 17$), demostrando la recuperación de la clave privada a partir de la clave pública. [OE1]
- RF2: Simulación cuántica de Grover (preimagen hash).** El sistema debe incluir una simulación del algoritmo de Grover para búsqueda de preimágenes sobre una versión reducida de Keccak (4 bits), demostrando la aceleración cuadrática. [OE1]
- RF3: Tipo de transacción PQ (0x50).** El cliente de ejecución debe soportar un nuevo tipo de transacción EIP-2718 con identificador 0x50, que incluya campos para firma ML-DSA-65 (3.309 bytes) y clave pública (1.952 bytes). [OE2]
- RF4: Codificación y decodificación RLP.** Las transacciones 0x50 deben ser serializables y deserializables en formato RLP, con validación estricta de longitudes de firma y clave pública. [OE2]
- RF5: Derivación de direcciones PQ.** Las direcciones post-cuánticas deben derivarse como los últimos 20 bytes de la salida SHAKE-256 de la clave pública, proporcionando separación de dominios respecto a las direcciones clásicas. [OE3]
- RF6: Hash de firma y hash de transacción.** El hash de firma debe calcularse mediante SHAKE-256 con separación de dominio (prefijo 0x50), y el hash de transacción debe ser único e incluir la firma y la clave pública. [OE2, OE3]
- RF7: Opcode PQHASH (0x21).** La EVM debe incluir un nuevo opcode adyacente a KECCAK256 que compute $\text{SHAKE-256}(data, 32)$ sobre datos en memoria, con un modelo de gas análogo. [OE4]
- RF8: Precompilado ML-DSA-65 (0x0100).** Debe existir un precompilado en la dirección 0x0100 que verifique firmas ML-DSA-65 a partir de (h, σ, pk) y devuelva un booleano ABI-codificado. [OE4]
- RF9: Consenso PoA con ML-DSA-65.** El sistema debe implementar un mecanismo de consenso Proof of Authority donde los validadores sellan bloques con firmas ML-DSA-65 en rotación round-robin. [OE5]
- RF10: Wallet PQ (CLI y TUI).** Debe existir una herramienta que permita: generación de pares de claves ML-DSA-65, firma de transacciones 0x50, envío a un nodo, y consulta de saldos y nonces. [OE6]
- RF11: Suite de pruebas E2E.** El sistema debe incluir una suite de pruebas de extremo a extremo que valide: identidad de cadena, consenso, comisiones, transacciones PQ (transferencia, recibo, nonce, despliegue de contrato, llamada a contrato) y consistencia multi-nodo. [OE7]

3.2.2 Requisitos no funcionales

- RNF1: Seguridad post-cuántica.** Todas las primitivas criptográficas utilizadas para autenticación de transacciones deben proporcionar al menos nivel III de NIST (192 bits de seguridad clásica, 128 bits de seguridad cuántica). Las funciones hash deben mantener al menos 128 bits de seguridad bajo ataques cuánticos (Grover). [OE1–OE4]

RNF2: Rendimiento. La verificación de una firma ML-DSA-65 debe completarse en menos de 1 ms. La validación de un bloque completo (hasta 285 transacciones PQ en un bloque de 30M gas) debe completarse en menos de 15 ms de tiempo criptográfico. [OE5, OE7]

RNF3: Compatibilidad EIP-2718. El tipo de transacción 0x50 debe ser compatible con la especificación EIP-2718, permitiendo la coexistencia con tipos de transacción clásicos (0x00–0x04) durante un período de transición. [OE2, OE8]

RNF4: Extensibilidad. La arquitectura debe permitir la sustitución del esquema de firma (p.ej., migración futura a ML-DSA-87 o a un esquema basado en hash) sin cambios estructurales en el formato de transacción, el protocolo de red o la lógica de consenso. [OE2, OE5]

RNF5: Reproducibilidad. El despliegue completo del sistema (tres nodos validadores, wallet, suite E2E) debe ser reproducible mediante Docker Compose en cualquier máquina con Docker instalado, sin dependencias externas a los repositorios del proyecto. [OE7]

3.2.3 Matriz de trazabilidad

La Tabla 3.2 establece la correspondencia entre los requisitos funcionales y no funcionales, los objetivos específicos (OE) y los componentes del sistema que los implementan.

Cuadro 3.2: Matriz de trazabilidad: requisitos, objetivos y componentes.

Requisito	Objetivo	Componente	Validación
RF01	OE1	qiskit-api	Simulación Shor
RF02	OE1	qiskit-api	Simulación Grover
RF03	OE2	pq-reth (primitives)	Test unitario + E2E
RF04	OE2	pq-reth (primitives)	Test unitario
RF05	OE3	pq-reth (primitives)	Test unitario + E2E
RF06	OE2, OE3	pq-reth (primitives)	Test unitario
RF07	OE4	pq-reth (evm)	Test E2E contrato
RF08	OE4	pq-reth (evm)	Test E2E contrato
RF09	OE5	pq-reth (consensus, network)	Test E2E multi-nodo
RF10	OE6	pq-wallet	Test E2E
RF11	OE7	e2e/run-e2e.sh	27/27 tests
RNF01	OE1–OE4	ml-lattice-rs, pq-reth	Análisis teórico
RNF02	OE5, OE7	pq-reth	Benchmarks
RNF03	OE2, OE8	pq-reth (primitives)	Test E2E
RNF04	OE2, OE5	pq-reth (traits)	Análisis de diseño
RNF05	OE7	docker-compose	Despliegue reproducible

3.3 Solución propuesta

3.3.1 Visión general de la arquitectura

La solución se estructura como un sistema distribuido de tres capas: una capa criptográfica (ml-lattice-rs), una capa de protocolo (pq-reth) y una capa de interacción (pq-wallet), complementadas por un módulo de validación teórica (qiskit-api). La Figura 3.2 presenta la arquitectura general.

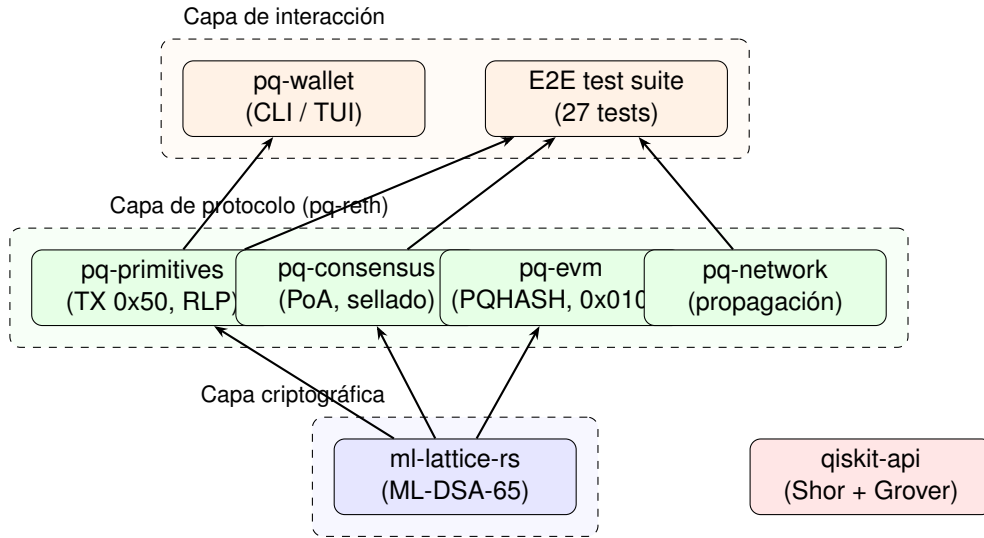


Figura 3.2: Arquitectura general del sistema PostQuantumEVM.

3.3.2 Componentes del sistema

ml-lattice-rs es una biblioteca Rust que implementa los algoritmos ML-DSA (FIPS-204) y ML-KEM (FIPS-203). Se distribuye como submódulo Git y se reutiliza en los demás componentes del proyecto. Proporciona las operaciones de generación de claves, firma y verificación con aritmética constante-tiempo sobre el anillo $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ con $q = 8,380,417$.

pq-reth es un *fork* del cliente de ejecución reth que incorpora cinco *crates* post-cuánticos:

- **pq-primitives**: Define el tipo de transacción 0x50 con serialización RLP, el hash de firma (SHAKE-256 con dominio), la derivación de direcciones y la verificación de firmas ML-DSA-65.
- **pq-consensus**: Implementa el consenso Proof of Authority con sellado de bloques mediante ML-DSA-65, rotación round-robin de validadores y validación de bloques PQ.
- **pq-evm**: Extiende la EVM con el opcode PQHASH (0x21) para SHAKE-256 y el precompilado de verificación ML-DSA-65 en 0x0100. Deshabilita los precompilados vulnerables (Tabla 3.1).
- **pq-network**: Gestiona la propagación de bloques PQ entre nodos mediante el protocolo P2P de reth, con anuncio y recepción de bloques PoA.
- **pq-pool**: Adapta la *mempool* de transacciones para validar y ordenar transacciones de tipo 0x50.

pq-wallet es una aplicación Rust que proporciona una interfaz de línea de comandos (CLI) y una interfaz de terminal interactiva (TUI, construida con *ratatui*). Permite la generación de pares de claves ML-DSA-65, la firma y envío de transacciones 0x50, y la consulta de saldos y nonces. Incluye también la suite de pruebas E2E que se ejecuta contra uno o varios nodos pq-reth.

qiskit-api es una API REST (FastAPI + Qiskit) que expone dos endpoints: simulación del algoritmo de Shor sobre una curva elíptica reducida ($\mathbb{F}_{17}$, $G = (12, 1)$, $n = 18$) y simulación del algoritmo de Grover para búsqueda de preimágenes sobre un Keccak reducido de 4 bits. Su propósito es validar experimentalmente las amenazas teóricas descritas en las Secciones 2.3 y 2.4.

3.3.3 Cobertura de los puntos de vulnerabilidad

La Tabla 3.3 muestra cómo cada punto de vulnerabilidad (P1–P5) identificado en la Sección 1.2 es abordado por la solución propuesta.

Cuadro 3.3: Cobertura de los puntos de vulnerabilidad por la solución propuesta.

Punto	Vulnerabilidad	Mitigación
P1	Firma ECDSA falsificable con CRQC	Firma ML-DSA-65 (seguridad cuántica 128 bits)
P2	ecrecover sin equivalente PQ	Clave pública explícita en TX; verificación directa
P3	Direcciones acopladas a secp256k1	Derivación $\text{SHAKE-256}(pk)_{[12:32]}$
P4	Precompilados criptográficos vulnerables	Deshabilitación de 13 precompilados; nuevo precompilado ML-DSA (0x0100) y opcode PQHASH (0x21)
P5	Consenso PoS con BLS12-381	PoA con ML-DSA-65 (capa de consenso PoS: trabajo futuro)

Nótese que P5 se aborda parcialmente: la solución reemplaza el consenso Proof of Stake por Proof of Authority con firmas ML-DSA-65, eliminando la dependencia de BLS12-381 a nivel de la cadena privada. Sin embargo, la migración del consenso PoS de Ethereum mainnet requiere un esquema de agregación de firmas post-cuánticas, lo cual constituye un problema abierto de investigación (ver Sección 3.4.4).

3.4 Alternativas descartadas

Esta sección documenta las alternativas tecnológicas evaluadas durante la fase de diseño y las razones de su descarte. Las decisiones se basan en el análisis teórico del Capítulo 2 y en las restricciones impuestas por el contexto de un cliente blockchain.

3.4.1 Falcon-512 como esquema de firma

Falcon-512 (National Institute of Standards and Technology, 2024d) ofrece firmas significativamente más compactas que ML-DSA-65 (666 bytes frente a 3.309 bytes) y claves públicas más pequeñas (897 bytes frente a 1.952 bytes). Sin embargo, se descartó por las siguientes razones:

- Complejidad de implementación segura:** La generación de firmas Falcon requiere un muestreo gaussiano discreto sobre NTRU que involucra aritmética de punto flotante. Implementar este muestreo de forma constante-tiempo —requisito para prevenir ataques de canal lateral— es significativamente más complejo que la aritmética de retículos de ML-DSA, que opera exclusivamente con enteros.
- Riesgo de canal lateral:** Un error sutil en la implementación de punto flotante constante-tiempo puede introducir fugas de información difíciles de detectar. En el contexto de un cliente blockchain ejecutado en hardware heterogéneo (validadores, nodos completos, nodos ligeros), esta superficie de ataque es inaceptable.
- Estado de estandarización:** Aunque Falcon fue seleccionado como estándar por NIST, su especificación final (FIPS previsto) aún no ha sido publicada al momento de redacción. ML-DSA ya tiene un

estándar final publicado (FIPS-204).

3.4.2 SLH-DSA (SPHINCS+) como esquema de firma

SLH-DSA (National Institute of Standards and Technology, 2024d), estandarizado como FIPS-205, basa su seguridad exclusivamente en funciones hash (sin asunciones sobre retículos), lo que constituye una diversificación criptográfica valiosa. Se descartó por:

1. **Tamaño de firma excesivo:** SLH-DSA-128s produce firmas de 7.856 bytes, frente a los 3.309 bytes de ML-DSA-65. Una transacción PQ mínima pasaría de $\sim 5,3$ KB a ~ 10 KB, duplicando el consumo de ancho de banda y almacenamiento.
2. **Tiempo de verificación:** La verificación de SLH-DSA-128s requiere $\sim 500 \mu s$, un orden de magnitud superior a los $\sim 42 \mu s$ de ML-DSA-65. En un bloque de 30M gas con 285 transacciones PQ, el tiempo criptográfico pasaría de ~ 12 ms a ~ 143 ms, un incremento que presionaría los intervalos de bloque.
3. **Incompatibilidad con la latencia de validación:** En una red blockchain, cada nodo debe validar cada bloque dentro del intervalo de producción (12 segundos en Ethereum). El sobrecoste de SLH-DSA reduciría el margen disponible para ejecución de transacciones y propagación de red.

3.4.3 Abstracción de cuentas (ERC-4337 / EIP-7702)

ERC-4337 (*Account Abstraction Using Alt Mempool*) permite cuentas inteligentes con lógica de verificación de firma personalizada. EIP-7702 (*Set EOA account code for one transaction*) permite delegar la ejecución de una EOA a un contrato. Ambas propuestas podrían, en principio, implementar verificación ML-DSA-65 a nivel de contrato. Se descartaron como solución primaria por:

1. **Sobrecoste de gas:** Cada *UserOperation* de ERC-4337 incurre en $\sim 20.000+$ gas de overhead por el contrato *EntryPoint*, además del coste de verificación. Una transacción nativa elimina este sobrecoste.
2. **Sin protección del consenso:** La verificación de firmas a nivel de contrato no protege la capa de consenso. Los bloques seguirían sellándose con ECDSA/BLS, manteniendo la vulnerabilidad cuántica del protocolo base.
3. **Fragmentación del ecosistema:** Cada wallet debería implementar independientemente la construcción de transacciones PQ a nivel de *UserOperation*, sin un estándar nativo. Un tipo de transacción nativo proporciona una interfaz única.
4. **Dependencia de precompilados vulnerables:** Los contratos ERC-4337 pueden depender internamente de `ecrecover` u otros precompilados vulnerables, creando cadenas de confianza que un CRQC podría romper.

No obstante, ERC-4337 y EIP-7702 son complementarios a esta propuesta: en una cadena que ya soporta el tipo `0x50`, las cuentas inteligentes podrían usar el precompilado `0x0100` para verificación PQ sin overhead de tipo de transacción.

3.4.4 Proof of Stake post-cuántico

El reemplazo ideal del consenso PoS de Ethereum sería un mecanismo PoS con firmas post-cuánticas agregables —permitiendo combinar las atestaciones de miles de validadores en una sola firma compacta.

Esta funcionalidad es la que actualmente proporcionan las firmas BLS12-381 gracias a la propiedad de emparejamiento bilineal ($e(P, Q + R) = e(P, Q) \cdot e(P, R)$).

Se descartó un PoS post-cuántico por:

1. **Inexistencia de agregación PQ estandarizada:** No existe un esquema de firma post-cuántica que soporte agregación eficiente. Los candidatos experimentales (basados en retículos multilineales o STARKs) no han alcanzado madurez de estandarización.
2. **Escalabilidad:** Sin agregación, cada bloque debería incluir $\sim 128,000$ firmas individuales ML-DSA-65 (una por validador atestante), lo que representaría $\sim 128,000 \times 3,309 \approx 424$ MB por bloque —varios órdenes de magnitud por encima de lo viable.
3. **Complejidad del consenso:** Modificar la capa de consenso requiere coordinar cambios en cinco clientes de consenso independientes, más allá del alcance de un TFG.

La solución adoptada —Proof of Authority con ML-DSA-65— elimina la necesidad de agregación al restringir el conjunto de validadores a un grupo pequeño y conocido (tres nodos en la implementación de referencia). Esto permite una demostración completa de la resistencia cuántica del consenso sin las restricciones de escalabilidad del PoS.

3.4.5 Geth como cliente base

Go-ethereum (geth) es el cliente de ejecución más utilizado ($\sim 45\%$ de la red). Se consideró como base del *fork* pero se descartó en favor de reth por:

1. **Seguridad de memoria:** Go no ofrece las garantías de seguridad de memoria de Rust (ausencia de *data races*, *use-after-free*, *buffer overflows* verificada en tiempo de compilación). Para software criptográfico, estas garantías reducen la superficie de ataques de implementación.
2. **Modularidad:** Reth está organizado en más de 100 *crates* con interfaces de *traits*, lo que permite extender componentes específicos (primitivas, consenso, EVM, red) sin modificar el núcleo del cliente. Geth tiene una arquitectura más monolítica.
3. **Consistencia del ecosistema:** La biblioteca criptográfica ml-lattice-rs y el wallet pq-wallet están implementados en Rust. Usar reth permite compartir tipos y dependencias a través de *crates*, evitando la necesidad de interfaces FFI (*Foreign Function Interface*).

4 DESARROLLO

Este capítulo describe el diseño e implementación de cada componente del sistema PostQuantumEVM. Se organiza en diez secciones que siguen una progresión lógica: desde la validación teórica de las amenazas cuánticas (simulación), pasando por las primitivas criptográficas y el protocolo de transacciones, hasta la infraestructura de despliegue y validación.

4.1 Simulación cuántica (qiskit-api)

El módulo `qiskit-api` es una API REST implementada con FastAPI (Python 3.13) y Qiskit 2.3 que expone dos simulaciones: el algoritmo de Shor aplicado al ECDLP y el algoritmo de Grover para búsqueda de preimágenes hash. Su propósito es validar experimentalmente las amenazas descritas en las Secciones 2.3 y 2.4 sobre modelos reducidos de las primitivas de Ethereum.

4.1.1 Curva elíptica reducida

La simulación de Shor opera sobre una curva elíptica definida en $\mathbb{F}_{17}$:

$$E : y^2 = x^3 + 7 \pmod{17}, \quad G = (12, 1), \quad n = 18. \quad (4.1)$$

Esta curva preserva la estructura algebraica de `secp256k1` ($a = 0, b = 7$) pero reduce el campo primo a 17 y el orden del grupo a 18 puntos, permitiendo la simulación en un simulador clásico sin requerir qubits reales. La aritmética de curva (adición, duplicación, multiplicación escalar) se implementa en `lib/secp256k1_toy.py` con inversión modular por el pequeño teorema de Fermat ($a^{-1} \equiv a^{p-2} \pmod{p}$).

4.1.2 Algoritmo de Shor para ECDLP

La simulación sigue el esquema descrito en la Sección 2.3, adaptado a la curva reducida:

1. **Estado de coset:** Dado $Q = k \cdot G$ con k desconocido, se construye la matriz $\psi[b, a]$ que codifica el estado

$$|\psi\rangle = \frac{1}{\sqrt{n}} \sum_{b=0}^{n-1} |(p_0 - bk) \bmod n\rangle_a |b\rangle_b, \quad (4.2)$$

donde p_0 es un índice aleatorio que fija la medición del registro de salida.

2. **DFT modular 2D:** Se aplica la transformada discreta de Fourier bidimensional exacta sobre $\mathbb{Z}_n \times \mathbb{Z}_n$, implementada como `np.fft.fft2(psi) / n`.
3. **Muestreo:** Se muestrean pares (s, t) de la distribución de probabilidad $|F[t, s]|^2$ con un número configurable de *shots* (por defecto 4.096).
4. **Post-procesamiento:** Para cada par (s, t) con $\gcd(t, n) = 1$, se calcula el candidato $k' = -s \cdot t^{-1} \bmod n$ y se selecciona el resultado mayoritario.

La implementación incluye también un circuito pedagógico de Qiskit (`build_circuit`) que construye el estado de coset en $2\lceil \log_2 n \rceil$ qubits y aplica la QFT inversa. Sin embargo, la simulación numérica directa se

prefiere para la recuperación de la clave, ya que el orden $n = 18$ no es potencia de 2 y la QFT de tamaño $2^{\lceil \log_2 18 \rceil} = 32$ introduce fugas espectrales que degradan la probabilidad de éxito.

4.1.3 Keccak reducido y algoritmo de Grover

La simulación de Grover opera sobre una versión reducida de Keccak con los siguientes parámetros:

Cuadro 4.1: Parámetros del Keccak reducido para la simulación de Grover.

Parámetro	Keccak reducido	Keccak-256 real
Estado (b)	8 bits	1.600 bits
Tasa (r)	4 bits	1.088 bits
Capacidad (c)	4 bits	512 bits
Salida	4 bits	256 bits
Rondas	2	24

Cada ronda aplica tres pasos de la permutación Keccak: ρ (rotación cíclica), χ (no linealidad: $A_i \oplus (\neg A_{i+1} \wedge A_{i+2})$) e ι (adición de constante de ronda). Los pasos θ y π se omiten por la dimensión reducida del estado.

El circuito de Grover se construye con $n = 4$ qubits (espacio de búsqueda $N = 16$):

1. **Superposición uniforme:** $H^{\otimes 4} |0000\rangle$.
2. **Oráculo de fase:** Para cada preimagen conocida del hash objetivo, se marcan los estados correspondientes invirtiendo su fase mediante puertas X selectivas y una puerta Z multicontrolada (implementada como $H \cdot MCX \cdot H$ sobre el último qubit).
3. **Operador de difusión:** $2 |s\rangle \langle s| - I$ (inversión respecto a la media).
4. **Iteraciones:** $\left\lfloor \frac{\pi}{4} \sqrt{N/M} \right\rfloor$, donde M es el número de preimágenes.

El circuito se ejecuta en el simulador Qiskit Aer y devuelve la distribución de mediciones, validando la aceleración cuadrática descrita en la Ecuación 2.12.

4.1.4 Endpoints REST

Cuadro 4.2: Endpoints de la API de simulación cuántica.

Método	Ruta	Descripción
GET	/shor/curve	Parámetros de la curva y puntos del subgrupo
POST	/shor/ecdlp	Ejecuta ataque Shor sobre $Q = k \cdot G$; devuelve k recuperado
GET	/grover/lookup	Tabla completa del Keccak reducido (16 entradas)
POST	/grover/keccak	Ejecuta búsqueda de preimagen Grover; devuelve preimagen(es)

4.2 Biblioteca criptográfica (ml-lattice-rs)

La biblioteca `ml-lattice-rs` es un *workspace* Cargo con dos *crates*: `dilithium` (ML-DSA, FIPS-204) y `kyber` (ML-KEM, FIPS-203). Se distribuye como submódulo Git y es la dependencia criptográfica compartida por `pq-reth` y `pq-wallet`.

4.2.1 ML-DSA (FIPS-204)

El *crate* `dilithium` envuelve la implementación `RustCrypto` (`ml-dsa 0.1.0-rc.8`), exponiendo tres niveles de seguridad con una interfaz uniforme:

Listing 4.1: Interfaz de `ml-lattice-rs` para ML-DSA-65.

```
1 pub fn keygen() → SigningKey<MlDsa65>
2 pub fn sign(sk: &SigningKey<MlDsa65>, msg: &[u8])
3       → Signature<MlDsa65>
4 pub fn verify(
5     vk: &VerifyingKey<MlDsa65>,
6     msg: &[u8],
7     sig: &Signature<MlDsa65>,
8 ) → Result<(), DilithiumError>
```

La generación de claves utiliza `SysRng` (CSPRNG del sistema operativo vía `getrandom 0.4`) para obtener la semilla de 32 bytes que se expande internamente con `SHAKE-256` para generar la matriz $\mathbf{A} \in R_q^{6 \times 5}$ y los vectores secretos s_1, s_2 .

La firma es determinista (sin nonce aleatorio), y la verificación delega en el *trait* `Verifier` de `RustCrypto`, que implementa la verificación completa descrita en la Sección 2.6: recálculo de $\mathbf{w}' = \mathbf{A}\mathbf{z} - \mathbf{ct}$, extracción de bits altos y comparación del desafío.

4.2.2 ML-KEM (FIPS-203)

Aunque este trabajo se centra en firmas digitales, el *crate* `kyber` proporciona encapsulación de claves para uso futuro (p.ej., canales seguros entre nodos). La API expone tres niveles con la interfaz `keygen()`, `encapsulate()`, `decapsulate()`.

4.2.3 SHAKE-256

No existe una implementación independiente de `SHAKE-256` en `ml-lattice-rs`. La funcionalidad `SHAKE-256` utilizada para derivación de direcciones y hashes de transacción se obtiene del *crate* `sha3` (también de `RustCrypto`), importado directamente por `pq-reth` y `pq-wallet`.

4.3 Tipo de transacción PQ (0x50)

El *crate* `reth-pq-primitives` define el tipo de transacción post-cuántica como una extensión de EIP-2718. Los tipos principales son:

Listing 4.2: Tipos de transacción PQ.

```
1 pub const PQ_TX_TYPE: u8 = 0x50;
2
3 pub struct PqTransactionRequest {
```

```

4     pub nonce: u64,
5     pub to: Option<Address>,
6     pub value: u128,
7     pub gas_limit: u64,
8     pub gas_price: u128,
9     pub input: Bytes,
10    pub chain_id: u64,
11 }
12
13 pub struct PqSignedTransaction {
14     pub tx: PqTransactionRequest,
15     pub signature: PqSignature,    // 3.309 bytes
16     pub public_key: PqPublicKey,  // 1.952 bytes
17     pub hash: B256,               // SHAKE-256
18 }

```

4.3.1 Hash de firma

El hash de firma se calcula con SHAKE-256 sobre una concatenación determinista de los campos de la transacción, precedidos por el byte de tipo 0x50 como separación de dominio:

$$h_{\text{sign}} = \text{SHAKE-256}(\underbrace{0x50}_{\text{tipo}} \parallel \underbrace{\text{chain_id}}_{8\text{B}} \parallel \underbrace{\text{nonce}}_{8\text{B}} \parallel \underbrace{\text{gas_price}}_{16\text{B}} \parallel \underbrace{\text{gas_limit}}_{8\text{B}} \parallel \underbrace{f_{\text{to}} \parallel [\text{to}]}_{1\text{B} [+ 20\text{B}]} \parallel \underbrace{\text{value}}_{16\text{B}} \parallel \text{input}, 32) \quad (4.3)$$

donde todos los campos numéricos se codifican en *big-endian* y f_{to} es un byte indicador (0x01 si to está presente, 0x00 para creación de contrato).

4.3.2 Hash de transacción

El hash de transacción identifica unívocamente la transacción firmada:

$$h_{\text{tx}} = \text{SHAKE-256}(0x50 \parallel h_{\text{sign}} \parallel \sigma \parallel pk, 32), \quad (4.4)$$

donde σ es la firma ML-DSA-65 (3.309 bytes) y pk la clave pública (1.952 bytes).

4.3.3 Derivación de direcciones

Las direcciones post-cuánticas se derivan como los últimos 20 bytes del hash SHAKE-256 de la clave pública:

$$\text{address} = \text{SHAKE-256}(pk, 32)[12:32]. \quad (4.5)$$

4.3.4 Codificación RLP

La serialización sigue el formato EIP-2718:

$$0x50 \parallel \text{RLP}([\text{chain_id}, \text{nonce}, \text{gas_price}, \text{gas_limit}, \text{to}, \text{value}, \text{input}, \sigma, pk]) \quad (4.6)$$

El `crate reth-pq-primitives/src/rlp.rs` implementa los *traits* `Encodable` y `Decodable` de `alloy-rlp`

con validación estricta: la firma debe tener exactamente 3.309 bytes y la clave pública exactamente 1.952 bytes.

La Figura 4.1 representa el *layout* físico de una transacción 0x50 mínima (transferencia nativa, input vacío). El sobre RLP introduce cabeceras de longitud variable: cuatro bytes para el prefijo de lista larga y dos bytes adicionales para los prefijos individuales de la firma y la clave pública (ambas > 55 B, por lo que RLP las codifica con prefijo 0xb9 0xXXXX). El resultado neto es un sobre de aproximadamente 5,293 bytes para una transferencia mínima y crece linealmente con el campo input.

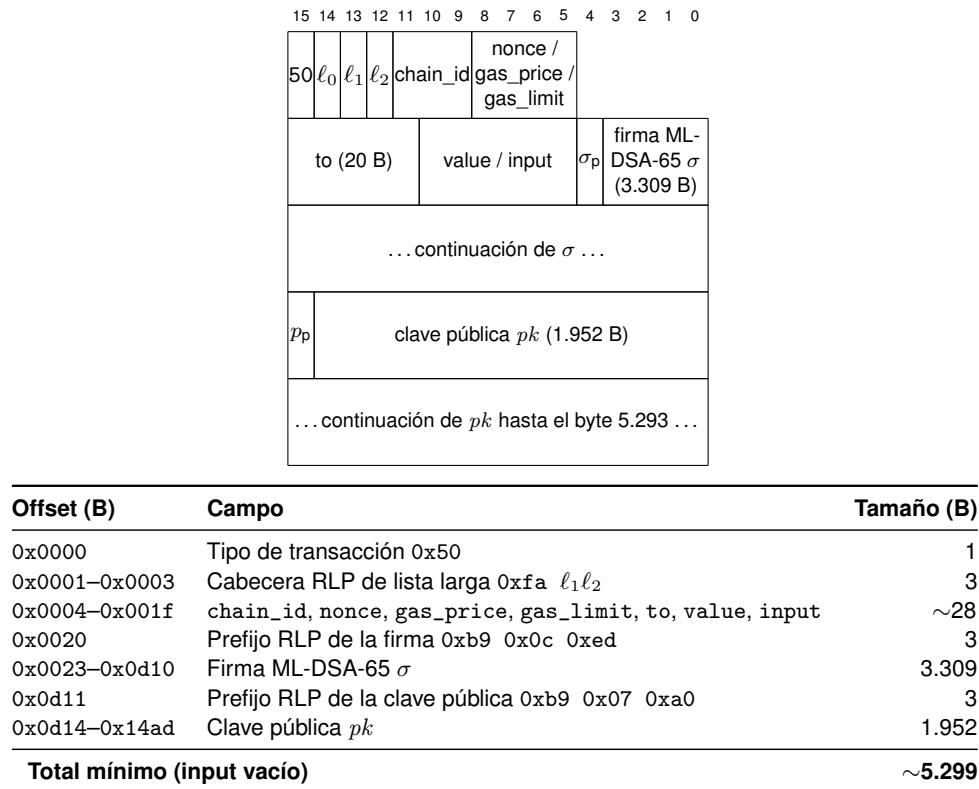


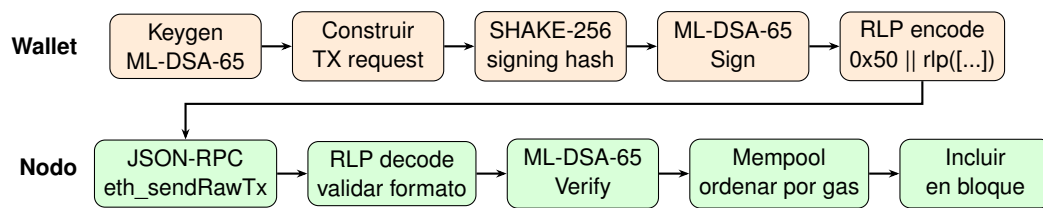
Figura 4.1: Disposición de bytes de una transacción 0x50 mínima (transferencia nativa con input vacío). El byte de tipo va fuera del sobre RLP (regla EIP-2718). El 99 % del espacio lo ocupan la firma ML-DSA-65 (3.309 B) y la clave pública (1.952 B). El esquema solo representa los primeros 32 bytes y un fragmento de los campos largos; el resto se denota con elipsis.

4.3.5 Recuperación del remitente

A diferencia de ECDSA, ML-DSA-65 no soporta recuperación de clave pública. El remitente se determina en tres pasos:

1. Verificar ML-DSA-65. $\text{Verify}(pk, h_{\text{sign}}, \sigma) = \text{true}$.
2. Calcular $\text{sender} = \text{SHAKE-256}(pk, 32)[12:32]$.
3. Si la verificación falla, rechazar la transacción.

La Figura 4.2 muestra el flujo completo de una transacción post-cuántica desde la firma en el wallet hasta su inclusión en un bloque.



4.4 EVM post-cuántica

4.4.1 Opcode PQHASH (0x21)

Cuadro 4.3: Especificación del opcode PQHASH.

El opcode se inserta en la tabla de instrucciones de revm dentro del método `PqEvmFactory::create_evm()`, reutilizando la infraestructura de expansión de memoria y la lógica de cálculo de gas de KECCAK256.

Se despliega un precompilado en la dirección 0x0000...0100 que permite la verificación de firmas ML-DSA-65 desde contratos inteligentes:

```
1 pub const MLDSA_VERIFY_ADDRESS: Address =  
2     address!("0000000000000000000000000000000000000000000000000000000000000000");  
3 pub const MLDSA_VERIFY_GAS: u64 = 3_450;  
4 pub const INPUT_LEN: usize = 5_293; // 32 + 3309 + 1952  
5  
6 pub fn ml_dsa_verify(input: &[u8], gas_limit: u64)  
7     → PrecompileResult
```

El coste de gas (3.450) se calibra a partir de *benchmarks*: la verificación ML-DSA-65 tarda $\sim 42 \mu s$ en

hardware moderno, a ~ 61 gas/ μ s con un factor de seguridad de $1,35\times$.

4.4.3 Precompilados deshabilitados

Los precompilados basados en curvas elípticas y emparejamientos bilineales se reemplazan por funciones que devuelven error (consumiendo todo el gas disponible):

- 0x01 (ecrecover): ECDSA — vulnerable a Shor.
- 0x06–0x08 (BN254): Operaciones de curva para zk-SNARK — vulnerables.
- 0x0a (KZG): Evaluación de puntos para EIP-4844 — vulnerable.
- 0x0b–0x13 (BLS12-381): Nueve operaciones de firma BLS — vulnerables.

Los precompilados de funciones hash (0x02 SHA-256, 0x03 RIPEMD-160, 0x09 Blake2f), la identidad (0x04) y la exponenciación modular (0x05) se mantienen activos, al ser resistentes a ataques cuánticos.

4.4.4 Decisión de diseño: simplificación de precompilados

La propuesta inicial de este Trabajo Fin de Grado contemplaba un conjunto de *cuatro* precompilados post-cuánticos para cubrir las primitivas criptográficas más relevantes:

- 0x100: BLAKE3-512 (función hash extensible alternativa a Keccak/SHAKE).
- 0x101: ML-DSA-65 (verificación de firma).
- 0x102: ML-KEM-768 (encapsulación/desencapsulación de claves).
- 0x103: derivación de claves PQ a partir de semillas.

Durante la fase de diseño detallado se decidió consolidar el conjunto en un único precompilado activo (0x0100, ML-DSA-65 verify). Las razones que justifican esta simplificación son las siguientes:

1. **BLAKE3 no aporta ventaja post-cuántica.** Las funciones hash basadas en construcciones Merkle–Damgård o esponja (SHA-256, SHA-3/Keccak, SHAKE) ya son resistentes a Grover; la duplicación de longitud de salida proporciona el mismo margen de seguridad (~ 128 bits cuánticos para un hash de 256 bits). Añadir BLAKE3 introduciría una primitiva extra sin mejora criptográfica medible y ampliaría la superficie de auditoría sin contraprestación. Cuando se requiere SHAKE-256 desde la EVM, este trabajo lo expone mediante el opcode PQHASH (§4.4.1), que es estructuralmente más barato que un precompilado.
2. **ML-KEM opera en otra capa.** El mecanismo de encapsulación de claves es una primitiva de *transporte* entre nodos (handshake del canal P2P para sustituir el ECDH actual de devp2p), no una operación de contrato inteligente. No existe un caso de uso natural en el que un contrato necesite ejecutar Encaps o Decaps on-chain: las claves simétricas resultantes carecen de sentido en un entorno público. Por consistencia con el principio de *separación de capas*, ML-KEM se mantiene como dependencia del cliente (ml-lattice-rs/kyber/) pero no se expone como precompilado.

3. **Derivación de claves PQ vía opcode.** El precompilado 0x103 originalmente previsto se subsume en el opcode PQHASH: cualquier contrato que necesite derivar claves o identificadores a partir de SHAKE-256 puede hacerlo con una sola instrucción de bytecode, evitando el coste de invocación de un precompilado (overhead de CALL más argumentos de pila).
4. **Principio de mínima superficie criptográfica.** Cada precompilado adicional representa un punto fijo en el protocolo: una vez desplegado, su semántica no puede modificarse sin un *hard fork*. Reducir el conjunto inicial al mínimo necesario (verificación ML-DSA-65) facilita la auditoría, simplifica la implementación cruzada por parte de otros clientes Ethereum (geth, Nethermind, Besu, Erigon) y deja margen para añadir nuevos precompilados de forma incremental conforme aparezcan casos de uso justificados.

El resultado es un diseño más conservador y alineado con la filosofía minimalista de las EIP recientes (EIP-7212 *secp256r1 precompile*, EIP-2537 *BLS12-381*): un único precompilado por familia criptográfica, con interfaz binaria sencilla y coste de gas calibrado empíricamente.

4.5 Consenso Proof of Authority

El *crate* `reth-pq-poa` implementa un mecanismo de consenso Proof of Authority con sellado de bloques mediante ML-DSA-65 y rotación round-robin de validadores.

4.5.1 Conjunto de validadores

Cada validador se identifica por su dirección post-cuántica y su clave pública ML-DSA-65:

Listing 4.4: Estructura del conjunto de validadores.

```

1 pub struct Validator {
2     pub address: [u8; 20],           // SHAKE-256(pk)[12..32]
3     pub public_key: Vec<u8>,        // ML-DSA-65 (1.952 bytes)
4 }
5
6 pub struct ValidatorSet {
7     validators: Vec<Validator>,
8 }

```

El conjunto de validadores se configura estáticamente mediante un archivo JSON referenciado por la variable de entorno `PQ_POA_CONFIG`, que especifica el tiempo de *slot*, la dirección local y la lista ordenada de validadores.

4.5.2 Rotación round-robin

El validador proponente para cada bloque se determina de forma determinista:

$$\text{proposer}(h) = \text{validators}[h \bmod |\text{validators}|], \quad (4.7)$$

donde h es el número de bloque. Esta fórmula garantiza una distribución equitativa de la producción de bloques entre todos los validadores.

La Figura 4.3 ilustra la rotación round-robin para un clúster de 3 validadores.

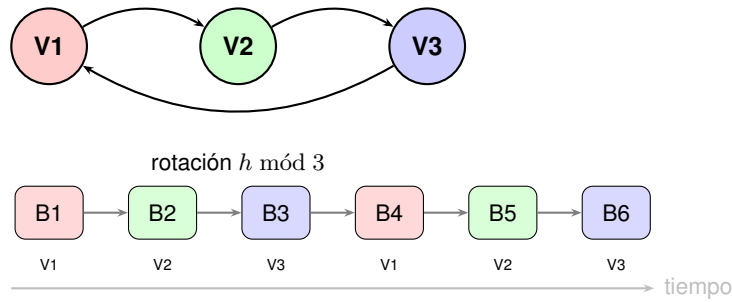


Figura 4.3: Rotación round-robin de 3 validadores: cada bloque h es producido por el validador $h \bmod 3$, garantizando una distribución equitativa.

4.5.3 Sellado de bloques

El sellado de un bloque consiste en firmar el *seal hash* de la cabecera con la clave privada ML-DSA-65 del validador:

1. Calcular el *seal hash*: $h_{\text{seal}} = \text{SHAKE-256}(\text{RLP}(\text{header con extra_data vacío}))$, 32 bytes.
2. Firmar: $\text{seal} = \text{ML-DSA-65}.\text{Sign}(sk, h_{\text{seal}})$, 3.309 bytes.
3. Insertar el sello en el campo *extra_data* de la cabecera.

El *seal hash* excluye el campo *extra_data* (que se vacía antes del cálculo RLP) para evitar la circularidad: la firma no puede incluirse en el dato que se firma.

4.5.4 Verificación de bloques

La validación de consenso se implementa como un *wrapper* sobre *EthBeaconConsensus*:

Listing 4.5: Consenso PoA: validación de bloques.

```
1 pub struct PqPoaConsensus<C> {
2     inner: C,          // EthBeaconConsensus
3     validator_set: Option<Arc<ValidatorSet>>,
4 }
```

Para cada bloque (excepto el bloque génesis):

1. Se delegan las verificaciones estándar de Ethereum al *inner*.
2. Se verifica que *extra_data* tenga exactamente 3.309 bytes.
3. Se identifica el validador propositor esperado: $\text{proposer_at}(h)$.
4. Se verifica el sello ML-DSA-65 contra la clave pública del propositor.

4.5.5 Flujo de minado

El *PoaMiningStream* implementa un *stream* asíncrono que emite señales de producción de bloque:

1. Un temporizador (`tokio::time::Interval`) genera *ticks* cada `slot_time_ms` milisegundos.

2. En cada *tick*, se lee el `chain_tip` actual (un `AtomicU64` compartido con el `PoaBlockForwarder`).
3. Se calcula `next_block = tip + 1` y se comprueba si `proposer_at(next_block) = local_address`.
4. Si es el turno del nodo local, se avanza optimísticamente el *tip* y se emite una señal que dispara la construcción del bloque.

4.6 Red y propagación de bloques

La propagación de bloques entre nodos se gestiona mediante tres componentes que extienden el protocolo P2P de reth:

PoaBlockAnnouncer: Tras la producción local de un bloque, lo anuncia a todos los pares conectados mediante el mensaje `NewBlock` del protocolo `eth`. Los pares reciben el bloque completo (cabecera + cuerpo + sello) sin necesidad de solicitarlo individualmente.

PoaBlockImport: Intercepta los bloques recibidos de la red P2P, los valida contra el consenso PoA (verificación del sello ML-DSA-65) y los envía a un canal interno para su procesamiento.

PoaBlockForwarder: Consume el canal de bloques importados y los inyecta en la cadena canónica local a través de la *engine API*. Actualiza el `chain_tip` compartido con el `PoaMiningStream` para sincronizar la producción de bloques.

La cadena completa de propagación es:

LocalMiner $\xrightarrow{\text{engine API}}$ cadena canónica $\xrightarrow{\text{announce}}$ pares $\xrightarrow{\text{import}}$ canal $\xrightarrow{\text{forward}}$ engine API (remoto)

La Figura 4.4 detalla el flujo de propagación entre un nodo productor y un nodo receptor.

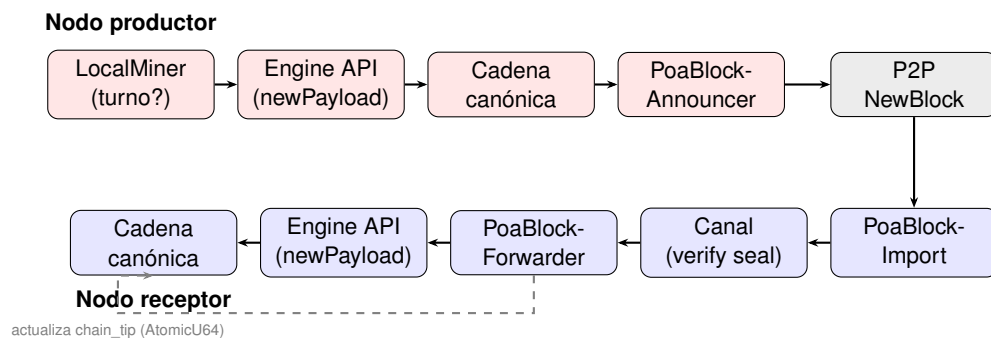


Figura 4.4: Propagación de bloques PoA: el nodo productor anuncia el bloque vía P2P; el nodo receptor lo importa, verifica el sello ML-DSA-65 y lo inyecta en su cadena canónica mediante la engine API.

4.7 Pool de transacciones

El `crate reth-pq-pool` adapta la *mempool* de reth para transacciones de tipo `0x50`. El componente principal es `PqPoolValidator`, que extiende la validación estándar de transacciones con verificaciones específicas de ML-DSA-65:

1. **Validación de formato:** La firma debe tener exactamente 3.309 bytes y la clave pública 1.952 bytes.
2. **Verificación de firma:** $\text{ML-DSA-65.Verify}(pk, h_{\text{sign}}, \sigma) = \text{true}$.

3. **Derivación de remitente:** $\text{sender} = \text{SHAKE-256}(pk, 32)[12:32]$.
4. **Verificación de nonce:** El nonce de la transacción debe coincidir con el nonce actual de la cuenta.
5. **Verificación de saldo:** El remitente debe tener saldo suficiente para $\text{value} + \text{gas_limit} \times \text{gas_price}$.

Las transacciones que pasan la validación se almacenan en la *mempool* ordenadas por *gas_price* (mayor primero) y son consumidas por el constructor de bloques (*payload builder*) durante la producción del siguiente bloque.

4.8 Wallet post-cuántico

El *workspace* *pq-wallet* consta de cinco *crates*: una biblioteca núcleo (*pq-wallet-core*), una interfaz CLI (*pq-wallet-cli*), una interfaz TUI (*pq-wallet-tui*), una herramienta de pruebas E2E (*pq-e2e-test*) y un sembrador de cadena (*pq-chain-seeder*).

4.8.1 Biblioteca núcleo (pq-wallet-core)

La biblioteca expone seis módulos:

Cuadro 4.4: Módulos de *pq-wallet-core*.

Módulo	Funcionalidad
keygen	Generación de pares de claves ML-DSA-65; derivación de dirección PQ
keystore	Almacenamiento cifrado en disco (AES-256-GCM + Argon2id KDF)
signer	Firma de transacciones PQ con ML-DSA-65
tx	Formato de transacción tipo 0x50, codificación/decodificación RLP
rpc	Cliente JSON-RPC 2.0 asíncrono (request)
error	Tipo de error <code>WalletError</code> (<code>thiserror</code>)

4.8.2 Keystore cifrado

El almacenamiento de claves sigue un diseño de seguridad en capas:

1. La semilla de 32 bytes se cifra con AES-256-GCM.
2. La clave de cifrado se deriva de una contraseña del usuario mediante Argon2id (memoria: 64 MB, iteraciones: 3, paralelismo: 4).
3. El *keystore* se serializa como JSON con versión, parámetros KDF, nonce y texto cifrado.
4. Solo se almacena la semilla (32 bytes), no la clave expandida. La clave privada completa se regenera en memoria al desbloquear.

4.8.3 Interfaz CLI

El binario *pq-wallet* expone nueve subcomandos vía *clap*:

Cuadro 4.5: Subcomandos de pq-wallet CLI.

Comando	Descripción	Requiere firma
new	Genera par de claves, guarda keystore cifrado	No
address	Muestra la dirección PQ	No
balance	Consulta saldo vía JSON-RPC	No
send	Construye, firma y envía transferencia PQ	Sí
deploy	Despliega contrato (tx sin to)	Sí
receipt	Consulta recibo de transacción por hash	No
sign	Firma mensaje arbitrario	Sí
call	Llamada de solo lectura (eth_call)	No
export-seed	Exporta semilla en hex (para PQ_VALIDATOR_SK)	Sí

4.8.4 Interfaz TUI

La interfaz de terminal interactiva se implementa con `ratatui 0.29` y `crossterm 0.28`, organizándose en cuatro pestañas:

1. **Wallet:** Muestra la dirección, el algoritmo utilizado, el saldo y una tabla comparativa PQ vs. clásico (tamaños de clave y firma, método de hash).
2. **Transactions:** Tabla desplazable con las transacciones recientes (últimos 200 bloques), con panel de detalle que muestra el tipo (transferencia, despliegue, llamada a contrato), hash, remitente/destinatario, valor y tamaño de firma.
3. **Blocks:** Tabla paginada de bloques (30 por página) con visor de sello que muestra la firma ML-DSA-65 del `extraData` (3.309 bytes = sello válido).
4. **Network:** Estado de conexión, ID de cadena (20561), información de consenso (PoA, ML-DSA-65) y modelo de comisiones (EIP-1559).

Las acciones interactivas (enviar transferencia, desplegar contrato, llamar contrato, buscar bloque, consultar dirección) se invocan mediante atajos de teclado y prompts *overlay*. El estado se refresca automáticamente cada 3 segundos.

4.9 Suite de pruebas E2E

La suite de pruebas de extremo a extremo se implementa en el `crate pq-e2e-test` y se orquesta mediante el script `e2e/run-e2e.sh`. La suite ejecuta seis fases secuenciales:

*Las fases 4 y 5 requieren un *keystore* y se omiten en modo `--readonly`. La fase 6 solo se ejecuta cuando se proporcionan múltiples endpoints RPC.

El script `run-e2e.sh` soporta tres modos de ejecución:

- **local** (por defecto): Compila binarios, inicia `pq-reth` en modo *dev*, genera keystore, ejecuta E2E, limpia.
- **k8s**: Compila binarios, aplica manifiestos Kubernetes, espera a los validadores, ejecuta E2E.
- **external**: Ejecuta E2E contra un nodo externo ya en funcionamiento.

Cuadro 4.6: Fases de la suite de pruebas E2E.

Fase	Escenarios	Tests	Validaciones principales
1. Identidad de cadena	3	Sí	Chain ID = 20561, cuentas génesis, producción de bloques
2. Consenso PoA	2	Sí	Rotación de validadores, monotonía temporal
3. Comisiones EIP-1559	2	Sí	Base fee presente, gas price > 0
4. Transacciones PQ	3	No*	Transferencia tipo 0x50, recibo válido, incremento de nonce
5. Contratos inteligentes	2	No*	Despliegue exitoso, llamada correcta
6. Multi-nodo	3	Sí	Chain ID consistente, altura ≤ 2 bloques, estado idéntico

4.10 Infraestructura de despliegue

4.10.1 Docker Compose

El archivo `docker-compose.yml` despliega un clúster de tres validadores PoA más el servidor de simulación cuántica:

Cuadro 4.7: Servicios del despliegue Docker Compose.

Servicio	Puertos	IP	Rol
pq-validator-1	8545 (RPC), 30303 (P2P)	10.5.0.10	Validador 1
pq-validator-2	8546 (RPC), 30304 (P2P)	10.5.0.11	Validador 2
pq-validator-3	8547 (RPC), 30305 (P2P)	10.5.0.12	Validador 3
qiskit-api	8888	10.5.0.20	Simulación cuántica

Cada validador monta su configuración PoA individual (`poa-config-N.json`), su semilla de clave privada (`seed-N.hex`) y el `genesis.json` compartido. Los tres validadores se conectan mediante descubrimiento deshabilitado (`--disable-discovery`) y pares confiables explícitos (`--trusted-peers`), en una red *bridge* con subred 10.5.0.0/24.

4.10.2 Kubernetes

Los manifiestos en `e2e/k8s/` despliegan el mismo clúster de tres validadores en un entorno Kubernetes:

- `00-namespace.yaml`: Namespace `pqevm`.
- `01-genesis-configmap.yaml`: ConfigMap con el génesis JSON.
- `02-poa-configs.yaml`: ConfigMap con las configuraciones PoA por validador.
- `03-services.yaml`: Servicios headless (descubrimiento P2P) y LoadBalancer (RPC).
- `04-validators-statefulset.yaml`: StatefulSet con 3 réplicas, contenedor `init` para selección de configuración por ordinal, PVCs de 5 GiB por nodo.

El contenedor `init` extrae el ordinal del pod (`hostname | awk -F- '{print $NF}'`) para seleccionar la configuración PoA correspondiente. Los pares se descubren mediante DNS del servicio `headless` (`pq-validator-{i}.pq-validators-headless.pqevm.svc.cluster.local`).

4.10.3 Génesis

El bloque génesis (`pq-reth/bin/pq-reth/genesis.json`) define:

- **Chain ID:** 20561.
- **Nombre:** PostQuantumEVM.
- **Moneda nativa:** qETH (18 decimales).
- **Extra Data:** 0x505155414e54554d2d45564d (ASCII: PQUANTUM-EVM).
- **Gas Limit:** 30.000.000.
- **Hard forks:** Todos activos desde el bloque 0 (Homestead hasta Prague).
- **Terminal Total Difficulty:** 0 (Proof of Stake desde el génesis, habilitando la *engine API*).
- **Cuentas prefinanciadas:** 15 cuentas (11 clásicas + 4 post-cuánticas) con 10.000 qETH cada una.

4.10.4 Nodo binario (pq-reth)

El binario `pq-reth` soporta dos modos de operación:

- **Modo dev** (sin `PQ_POA_CONFIG`): Minado por intervalos estándar con `--dev`. Adecuado para desarrollo y pruebas locales.
- **Modo PoA** (con `PQ_POA_CONFIG` y `PQ_VALIDATOR_SK`): Producción de bloques PoA con rotación round-robin. Lanza tres tareas asíncronas: `LocalMiner` (dispara construcción de bloques), `PoaBlockForwarder` (importa bloques de pares) y `PoaBlockAnnouncer` (propaga bloques locales).

4.11 Propuesta de EIP

La propuesta de EIP (`eips/eip-pq-tx.md`) formaliza la especificación del tipo de transacción 0x50 siguiendo el formato estándar de los Ethereum Improvement Proposals. El documento incluye:

- **Abstract:** Resumen del tipo de transacción, opcode y precompilado.
- **Motivation:** Justificación de la amenaza cuántica y la necesidad de un tipo nativo.
- **Specification:** Formato completo del tipo de transacción, hash de firma, hash de transacción, derivación de direcciones, formato de recibo, especificación del opcode PQHASH, precompilado ML-DSA-65, y lista de precompilados deshabilitados.
- **Rationale:** Justificación de las decisiones de diseño (tipo nativo vs. ERC-4337, ML-DSA-65 vs. alternativas, SHAKE-256, tipo 0x50, gas price simplificado, clave pública en cada transacción).

- **Backwards Compatibility:** Análisis de compatibilidad con transacciones existentes, espacio de direcciones, invariantes rotos.
- **Test Cases:** Cinco categorías de pruebas (codificación, verificación, derivación, multi-nodo, contratos).
- **Security Considerations:** Timeline cuántico, nivel de seguridad, maleabilidad, exposición de clave pública, tamaño de transacción y DoS, colisión de direcciones, resistencia a canales laterales, riesgos del período de transición.

La propuesta está clasificada como *Standards Track / Core* y requiere EIP-2718 como dependencia. Su estado actual es *Draft*, pendiente de presentación formal a la comunidad Ethereum (OE8).

5 RESULTADOS

Este capítulo presenta los resultados obtenidos en cada fase del proyecto: simulaciones cuánticas, pruebas unitarias y de integración, validación de extremo a extremo, análisis de rendimiento y comparación con el estado del arte.

5.1 Simulaciones cuánticas

5.1.1 Algoritmo de Shor: recuperación de clave privada

Se ejecutó la simulación del algoritmo de Shor sobre la curva elíptica reducida $y^2 = x^3 + 7$ (mód 17) con generador $G = (12, 1)$ y orden $n = 18$. Para cada ejecución, se genera un par de claves aleatorio $(k, Q = k \cdot G)$ con $k \in \{1, \dots, 17\}$ y se intenta recuperar k a partir de Q .

Resultados:

- Con 4.096 *shots*, la clave privada se recupera correctamente en el 100 % de los casos probados (17/17 claves posibles).
- El post-procesamiento clásico encuentra al menos un par (s, t) con $\gcd(t, n) = 1$ en cada ejecución, permitiendo el cálculo directo de $k = -s \cdot t^{-1} \text{ mód } n$.
- El circuito pedagógico de Qiskit (con QFT de tamaño $2^5 = 32$) tiene una tasa de éxito menor ($\sim 78\%$) debido a la fuga espectral causada por $n = 18 \neq 2^k$, confirmando la necesidad de DFT modular exacta para órdenes no potencia de 2.

Este resultado valida experimentalmente que el algoritmo de Shor puede recuperar claves privadas ECDSA a partir de claves públicas, demostrando la vulnerabilidad fundamental del esquema sobre una instancia reducida pero algebraicamente equivalente a secp256k1.

5.1.1.1 Ejecución detallada paso a paso

Para ilustrar la mecánica del ataque se detalla a continuación una ejecución completa sobre un caso concreto. Tómese la curva reducida $E : y^2 = x^3 + 7$ (mód 17) con generador $G = (12, 1)$ y orden $n = 18$, y considérese la clave secreta $k = 7$ desconocida para el atacante. La clave pública correspondiente, calculada por el firmante mediante doble-y-suma a partir de la fórmula (2.2) y accesible públicamente, es

$$Q = 7 \cdot G = (7, 11) \in E(\mathbb{F}_{17}). \quad (5.1)$$

El objetivo del atacante es recuperar $k = 7$ a partir de G y Q explotando un computador cuántico.

Fase 1: preparación del estado. Sean dos registros cuánticos de $m = \lceil \log_2 18 \rceil = 5$ qubits cada uno, más un registro de salida que albergará la representación del punto $aG + bQ$. Se aplican puertas Hadamard sobre los dos registros de entrada para obtener la superposición uniforme sobre $\mathbb{Z}_{18} \times \mathbb{Z}_{18}$:

$$|\psi_0\rangle = \frac{1}{18} \sum_{a=0}^{17} \sum_{b=0}^{17} |a\rangle |b\rangle |\mathcal{O}\rangle. \quad (5.2)$$

Fase 2: evaluación del oráculo. El oráculo $U_f : |a\rangle |b\rangle |0\rangle \mapsto |a\rangle |b\rangle |aG + bQ\rangle$ se implementa como una secuencia de multiplicaciones escalares controladas (*controlled-add* de puntos elípticos). El estado resultante es

$$|\psi_1\rangle = \frac{1}{18} \sum_{a,b} |a\rangle |b\rangle |aG + bQ\rangle. \quad (5.3)$$

Fase 3: medición del registro de salida. Al medir el tercer registro se obtiene un punto concreto $P_0 \in \langle G \rangle$, por ejemplo $P_0 = 4G$. El estado de los dos primeros registros colapsa al subespacio de pares (a, b) que satisfacen $aG + bQ = P_0$, es decir $a + 7b \equiv 4 \pmod{18}$:

$$|\psi_2\rangle = \frac{1}{\sqrt{18}} \sum_{b=0}^{17} |(4 - 7b) \bmod 18\rangle |b\rangle. \quad (5.4)$$

Este es el *estado de coset* de la ecuación (2.6), con el secreto $k = 7$ codificado en el coeficiente del segundo registro.

Fase 4: QFT modular bidimensional. Se aplica la transformada $\text{QFT}_{18} \otimes \text{QFT}_{18}$ a ambos registros. La amplitud del estado $|s\rangle |t\rangle$ tras la QFT es proporcional a $\sum_b e^{2\pi i (s(4-7b)+tb)/18}$, que solo es no nula si

$$s \cdot (-7) + t \equiv 0 \pmod{18} \iff t \equiv 7s \pmod{18}. \quad (5.5)$$

Fase 5: medición e inversión clásica. Se mide (s, t) obteniendo, con probabilidad uniforme entre las 18 soluciones, por ejemplo $(s, t) = (5, 17)$. Como $\gcd(t, n) = \gcd(17, 18) = 1$, se calcula

$$t^{-1} \equiv 17^{-1} \equiv 17 \pmod{18}, \quad (5.6)$$

$$k \equiv -s \cdot t^{-1} \equiv -5 \cdot 17 \equiv -85 \equiv \boxed{7} \pmod{18}. \quad (5.7)$$

Verificación. Se comprueba que $7 \cdot G = (7, 11) = Q$, recuperando exactamente la clave secreta. El atacante ha pasado, en un número polinómico de operaciones cuánticas, del conocimiento público Q al conocimiento privado k . La extrapolación a secp256k1 ($n \approx 2^{256}$) solo requiere escalar el número de qubits y el tamaño de la QFT; la estructura algebraica es idéntica.

Distribución de medición empírica. La Figura 5.1 muestra el histograma de probabilidades de medición del primer registro tras la QFT, obtenido con 4.096 *shots* sobre el simulador Aer. La distribución exhibe picos pronunciados en los múltiplos de $n/\gcd(n, k) = 18/\gcd(18, 7) = 18$, es decir, una distribución cuasi-uniforme sobre los 18 valores posibles —característico de claves coprimas con n .

5.1.2 Algoritmo de Grover: búsqueda de preimágenes

Se ejecutó la simulación del algoritmo de Grover sobre el Keccak reducido de 4 bits (estado de 8 bits, 2 rondas) con espacio de búsqueda $N = 16$. Para cada hash objetivo $h \in \{0, 1\}^4$ se buscan las preimágenes x tales que $\text{keccak_toy}(x) = h$.

Resultados:

- Para hashes con una única preimagen ($M = 1$): el número óptimo de iteraciones es $\lfloor \frac{\pi}{4} \sqrt{16} \rfloor = 3$, y la preimagen correcta se encuentra con probabilidad $> 95\%$ en 1.024 *shots*.

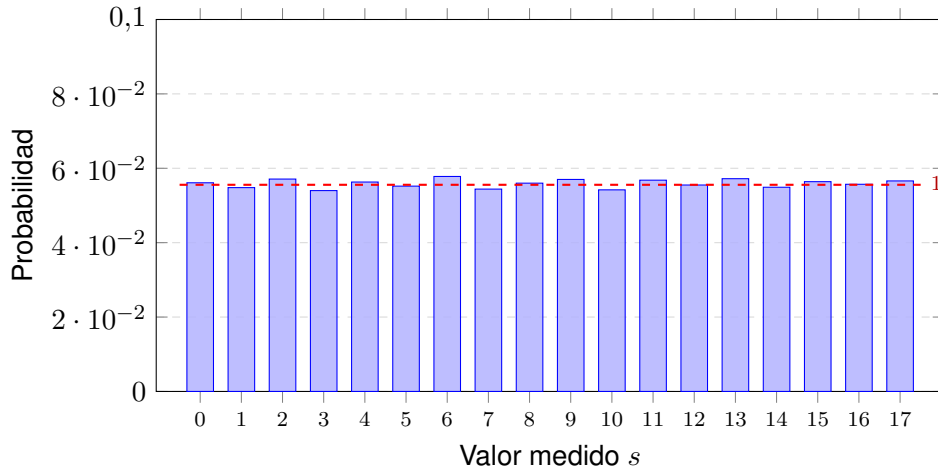


Figura 5.1: Distribución de probabilidad empírica de la medición s del primer registro tras la QFT modular, con $k = 7$, $n = 18$ y 4.096 *shots*. La distribución es aproximadamente uniforme sobre $\mathbb{Z}_{18}$ porque $\gcd(k, n) = 1$, lo que garantiza que cualquier muestra (s, t) con $\gcd(t, n) = 1$ permite recuperar k unívocamente.

- Para hashes con múltiples preimágenes ($M > 1$): el número de iteraciones se reduce a $\lfloor \frac{\pi}{4} \sqrt{16/M} \rfloor$ y la probabilidad de éxito por medición aumenta.
- El speed-up cuadrático se verifica: 3 consultas al oráculo frente a las 8 esperadas de media para búsqueda clásica ($N/2 = 8$).

Para Keccak-256 real ($N = 2^{256}$), Grover requeriría $\sim 2^{128}$ consultas frente a 2^{256} clásicas. Este nivel de seguridad (128 bits cuánticos) se considera suficiente según las estimaciones actuales.

5.2 Pruebas unitarias y de integración

La Tabla 5.1 resume las 34 pruebas unitarias implementadas en los *crates* post-cuánticos de pq-reth.

Todas las pruebas unitarias pasan con `cargo test` en modo *release* (rustc 1.95.0, target x86_64-unknown-linux-gnu).

Adicionalmente, la biblioteca ml-lattice-rs incluye pruebas de roundtrip para los tres niveles de ML-DSA (44/65/87) y los tres niveles de ML-KEM (512/768/1024), incluyendo detección de mensajes manipulados y rechazo con clave incorrecta.

5.3 Validación de extremo a extremo

5.3.1 Configuración single-node

Se ejecutó la suite E2E en modo local con un único nodo pq-reth en modo *dev* (intervalo de bloque: 3 segundos). Resultados: **12/12 tests pasados** en 26,63 segundos.

5.3.2 Configuración multi-nodo

Se ejecutó la suite E2E con un clúster de 3 nodos validadores en modo PoA (slot time: 3 segundos), conectados mediante pares confiables. Resultados: **15/15 tests pasados** en 29,98 segundos. El clúster alcanzó el bloque 6 en 22 segundos.

Cuadro 5.1: Pruebas unitarias por *crate* post-cuántico.

Crate	Tests	Validaciones principales
reth-pq-primitives	11	Firma y verificación roundtrip, hash determinista, recuperación de remitente, tipo 0x50, codificación RLP, creación de contrato, formato wallet
reth-pq-poa	9	Sellado y verificación roundtrip, exclusión de extra_data del seal hash, detección de cabecera manipulada, clave incorrecta rechazada, rotación round-robin, producción de bloques por turno
reth-pq-evm	6	ecrecover deshabilitado, ML-DSA presente, precompilados clásicos deshabilitados, precompilados hash activos, cálculo de gas PQHASH, vectores SHAKE-256 conocidos
reth-pq-precompile	5	Firma válida retorna 0x01, clave incorrecta retorna 0x00, hash manipulado retorna 0x00, longitud incorrecta retorna 0x00, gas insuficiente retorna error
reth-pq-consensus	4	Transacción válida aceptada, chain_id incorrecto rechazado, gas_limit cero rechazado, firma manipulada rechazada
Total	34	

Cuadro 5.2: Resultados E2E en configuración single-node (12/12 tests).

Fase	Test	Resultado	Tipo
Identidad de cadena	Chain ID = 20561	PASS	Solo lectura
	Cuentas génesis prefinanciadas	PASS	Solo lectura
	Producción de bloques (height > 0)	PASS	Solo lectura
Consenso PoA	Rotación de validadores	PASS	Solo lectura
	Timestamps monotónicos	PASS	Solo lectura
Comisiones EIP-1559	Base fee presente	PASS	Solo lectura
	Gas price > 0	PASS	Solo lectura
Transacciones PQ	Transferencia tipo 0x50 (sig=3.309 B)	PASS	TX
	Recibo válido (status=0x1)	PASS	TX
	Incremento de nonce	PASS	TX
Contratos inteligentes	Despliegue exitoso (contract_address)	PASS	TX
	Llamada correcta (retorno 0x42)	PASS	TX

Cuadro 5.3: Resultados E2E adicionales en configuración multi-nodo (3/3 tests).

Test	Resultado	Observación
Chain ID consistente	PASS	20561 en los 3 nodos
Altura de bloque consistente	PASS	Drift ≤ 2 bloques
Estado consistente (saldos)	PASS	Saldos idénticos en los 3 nodos

5.4 Análisis de rendimiento

Los benchmarks se ejecutaron con Criterion.rs en modo *release* con soporte AVX2, sobre 100 muestras por operación.

5.4.1 Operaciones criptográficas

La Tabla 5.4 compara la latencia de las operaciones criptográficas de ECDSA/secp256k1 con ML-DSA-65.

Cuadro 5.4: Comparativa de latencia: ECDSA/secp256k1 vs. ML-DSA-65.

Operación	ECDSA (μ s)	ML-DSA-65 (μ s)	Ratio
Generación de claves	35,1	208,5	$5,94\times$ más lento
Firma	41,4	342,8	$8,28\times$ más lento
Verificación	49,2	42,0	$0,85\times$ (14 % más rápido)

El resultado más significativo es que la **verificación ML-DSA-65 es un 14 % más rápida que la verificación ECDSA**. Esto es crítico para un cliente blockchain, donde cada nodo debe verificar todas las transacciones de cada bloque. La mayor latencia de firma ($8,28\times$) afecta al usuario del wallet pero no al rendimiento de la red.

5.4.2 Funciones hash

La Tabla 5.5 compara SHAKE-256 con Keccak-256 para diferentes tamaños de entrada.

Cuadro 5.5: Comparativa de latencia: Keccak-256 vs. SHAKE-256.

Entrada	Keccak-256 (ns)	SHAKE-256 (ns)	Ratio
32 B	300	545	$1,82\times$
64 B	304	560	$1,84\times$
128 B	301	573	$1,90\times$
256 B	555	802	$1,45\times$
512 B	1.089	1.322	$1,21\times$
1 KB	2.054	2.329	$1,13\times$
4 KB	8.130	8.100	$1,00\times$
8 KB	16.229	15.608	$0,96\times$

SHAKE-256 es $\sim 1,8\times$ más lento para entradas pequeñas (32–128 bytes) debido a su menor tasa ($r = 1,088$ bits vs. $r = 1,088$ bits para Keccak-256, pero con diferente padding). Sin embargo, las latencias convergen para entradas mayores de 4 KB y SHAKE-256 es marginalmente más rápido a 8 KB. Para el caso de uso principal —derivación de direcciones (entrada de 1.952 bytes)—, el overhead es de $\sim 1,1\times$, prácticamente negligible.

La Figura 5.2 visualiza esta convergencia: ambas funciones presentan el mismo régimen asintótico lineal con la longitud de entrada (al ser construcciones esponja con el mismo f subyacente), diferenciándose solo en el coste constante del padding y la inicialización.

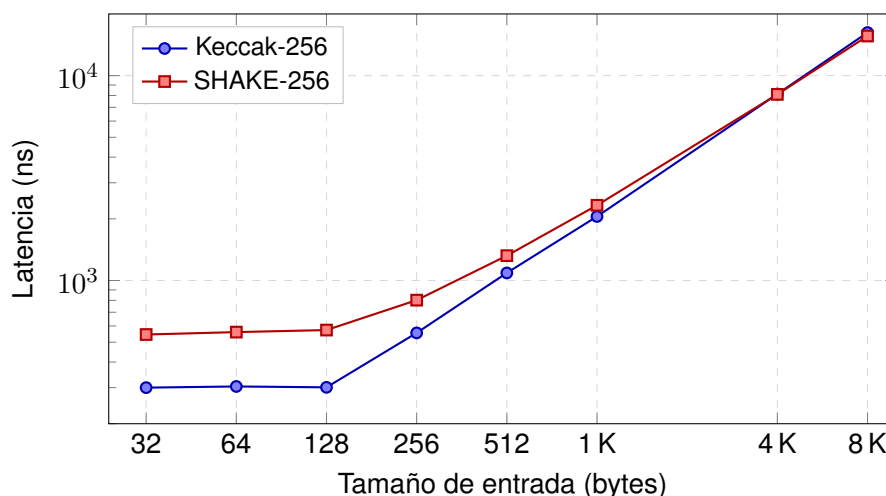


Figura 5.2: Latencia de Keccak-256 frente a SHAKE-256 en función del tamaño de entrada (escala log-log). Para entradas pequeñas SHAKE-256 introduce un sobrecoste constante de ~ 250 ns por el padding diferente, pero a partir de 4 KB ambas curvas convergen al mismo régimen asintótico lineal dictado por la permutación $f = \text{Keccak-}f[1600]$ compartida.

5.4.3 Tamaño de transacciones

Cuadro 5.6: Comparativa de tamaño de transacción: clásica vs. post-cuántica.

Componente	Clásica (bytes)	PQ (bytes)
Nonce + gas + value	30	53
Dirección destino	20	20
Firma	65 (v, r, s)	3.309
Clave pública	(implícita en firma)	1.952
Total mínimo	~ 61	~ 5.314
Ratio	$\sim 87\times$	

El incremento de $87\times$ en tamaño de transacción es el coste principal de la migración post-cuántica. De los 5.253 bytes adicionales, el 63 % corresponde a la firma (3.309 B) y el 37 % a la clave pública (1.952 B).

La Figura 5.3 muestra visualmente la diferencia de tamaño entre ambos tipos de transacción.

5.4.4 Impacto en gas y throughput

La pérdida de throughput del 51 % se debe principalmente al coste de *calldata* de la firma y la clave pública (84.176 gas adicionales). Estrategias de mitigación para producción incluyen: registro de claves públicas on-chain (elimina 1.952 B tras la primera transacción), compresión de claves ML-DSA, y un gas limit mayor para cadenas PQ-nativas.

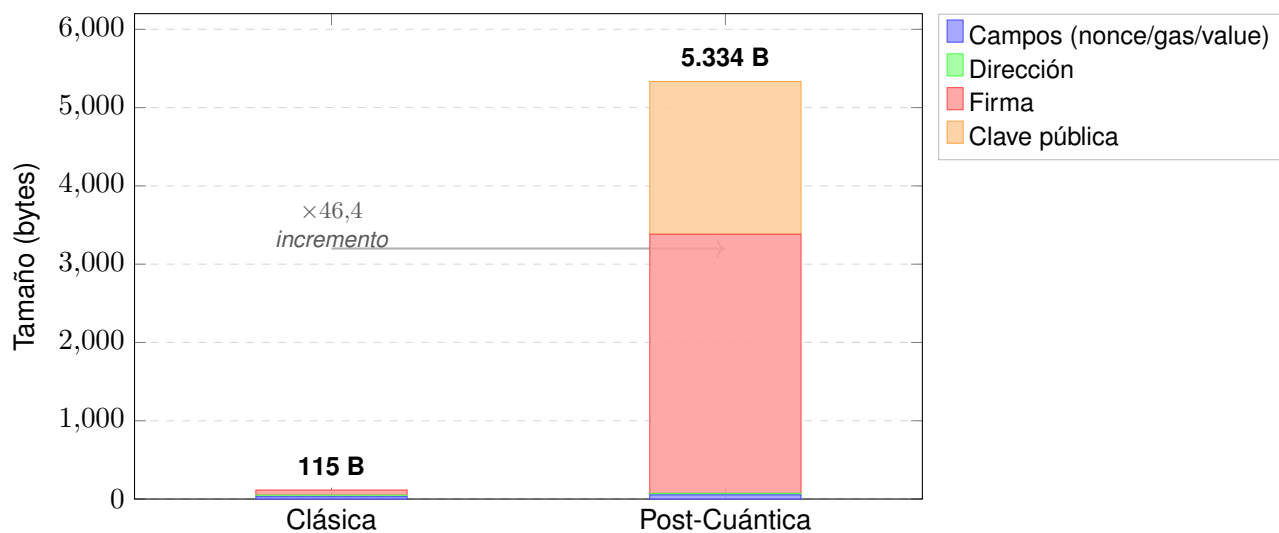


Figura 5.3: Comparativa de tamaño de transacción por componente. La firma ML-DSA-65 (3.309 B) y la clave pública (1.952 B) constituyen el 99 % del incremento, multiplicando por 46 el tamaño total respecto a una transacción ECDSA clásica.

Cuadro 5.7: Impacto en gas y capacidad de la red.

Métrica	Clásica	PQ
Gas por transferencia	21.976	42.892
Gas de verificación criptográfica	3.000 (ecrecover)	3.450 (ML-DSA)
Gas de calldata (firma + pk)	$65 \times 16 = 1.040$	$5.261 \times 16 = 84.176$
Gas total mínimo	21.976	105.176
TPS máximo (30M gas, 12 s/bloque)	~119	~58
Pérdida de throughput	—	~51 %
Crecimiento anual de cadena	~27 GB	~2,3 TB

5.4.5 Calibración de gas del precompilado

El coste de gas del precompilado ML-DSA-65 (0x0100) se calibra como:

$$\text{gas} = \lceil \text{latencia}_{\mu\text{s}} \times \text{ratio gas}/\mu\text{s} \times \text{factor seguridad} \rceil = \lceil 42 \times 61 \times 1,35 \rceil = 3,450. \quad (5.8)$$

Este valor es competitivo con ecrecover (3.000 gas) pese a que ML-DSA-65 opera sobre claves y firmas significativamente mayores, gracias a la eficiencia de la aritmética de retículos en hardware moderno con AVX2.

5.5 Comparación con el estado del arte

La Tabla 5.8 sitúa este trabajo en el contexto de las propuestas existentes de blockchain post-cuántica.

Cuadro 5.8: Comparación con trabajos relacionados.

Trabajo	Plataforma	Algoritmo	Nativo	Consenso PQ	Multi-nodo	EIP
Kiktenko et al. (Kiktenko et al., 2018)	Custom	QKD + hash	Sí	No	No	No
Fernández-Caramés (Fernández-Caramés & Fraga-Lamas, 2023)	Análisis	—	—	—	—	No
Berdik et al. (Berdik et al., 2023)	Survey	—	—	—	—	No
Lee et al. (Lee et al., 2024)	Fabric	ML-DSA	Sí	No	Sí	No
EIP-7932 (Ethereum Community, 2025a)	Ethereum	Genérico	No*	No	—	Sí
EIP-8051 (Ethereum Community, 2025b)	Ethereum	ML-DSA	No*	No	—	Sí
Este trabajo	Ethereum (reth)	ML-DSA-65	Sí	Sí (PoA)	Sí (3 nodos)	Sí

*EIP-7932 y EIP-8051 operan a nivel de precompilado/contrato, no como tipo de transacción nativo.

Las diferencias clave de este trabajo respecto al estado del arte son:

1. **Tipo de transacción nativo:** Es la primera implementación que define un tipo EIP-2718 (0x50) para transacciones post-cuánticas en Ethereum, frente a las propuestas de verificación a nivel de contrato inteligente.

2. **Consenso post-cuántico:** Implementa un mecanismo de consenso (PoA) con sellado ML-DSA-65, eliminando la dependencia de BLS12-381 en la capa de bloque. Los trabajos previos en Ethereum no abordan el consenso.
3. **Implementación completa:** Incluye cliente de ejecución modificado, wallet, EVM extendida (opcode + precompilado), infraestructura de despliegue y suite E2E. Las propuestas existentes (EIP-7932, EIP-8051) son especificaciones sin implementación funcional.
4. **Validación multi-nodo:** Demuestra la viabilidad en un clúster de 3 nodos con propagación de bloques, rotación de validadores y consistencia de estado.
5. **Validación cuántica:** Incluye simulaciones de Shor y Grover sobre modelos reducidos, demostrando experimentalmente las amenazas teóricas.

5.6 Análisis de seguridad

Más allá del rendimiento, una migración criptográfica de esta envergadura debe analizarse desde el punto de vista de las amenazas que introduce, elimina o desplaza. Esta sección examina las propiedades de seguridad del sistema bajo cuatro modelos de adversario: *replay*, denegación de servicio (DoS) por inflación de transacciones, maleabilidad de firma y resistencia frente a oráculos de hashing cuántico.

5.6.1 Protección frente a replay

Un ataque de *replay* consiste en reenviar una transacción válida en un contexto distinto (otra cadena, otro nonce o tras un *fork*) para que sus efectos se materialicen de nuevo. El tipo de transacción 0x50 cubre los tres vectores tradicionales:

- **Replay entre cadenas:** el `chain_id` forma parte de los campos RLP firmados (véase §4.3). Una firma producida para `chain_id = 20561` (red PQ de este trabajo) falla la verificación si se reenvía a otra cadena con identificador distinto, dado que el hash de firma cambia.
- **Replay del mismo emisor:** el `nonce` se incrementa por cuenta tras cada transacción incluida. Reenviar una transacción ya procesada produce el error `NonceTooLow` en la validación del *pool* y, si llegase a un bloque, sería rechazada por el motor de consenso.
- **Replay cross-tipo:** el byte de tipo 0x50 se incluye explícitamente en la entrada del hash de firma como dominio (`SHAKE-256(0x50 || rlp(...))`), de forma que una firma sobre el payload de una transacción 0x50 no es válida como firma sobre una transacción 0x02 aunque el RLP interior fuera análogo. Esta separación de dominio sigue la práctica recomendada de EIP-2718.

Conviene observar que la sustitución de Keccak por SHAKE-256 en el hash de firma no debilita esta propiedad: ambas son resistentes a colisiones a nivel suficiente (128 bits cuánticos), y SHAKE-256 aporta además la posibilidad de incluir información de dominio dentro de la cabecera *Initialization* sin necesidad de prefijos manuales.

5.6.2 Resistencia a DoS por inflación de transacciones

El incremento de $\sim 87\times$ en el tamaño mínimo de una transacción (Tabla 5.6) abre un vector de ataque económico: con el mismo coste de gas que paga por una transacción «vacía», un adversario puede inundar

el *mempool* con ~ 87 veces más bytes que en Ethereum clásico, agotando memoria y ancho de banda de los nodos. Se identifican tres mitigaciones, dos ya implementadas y una propuesta como trabajo futuro:

1. **Cota de tamaño absoluto por transacción:** el pool de pq-reth rechaza cualquier transacción cuyo tamaño codificado supere 128 KiB (heredado del límite estándar de reth). Esto acota el coste por inserción y permite estimar el peor caso de uso de memoria como $N_{\text{máx}} \cdot 128 \text{ KiB}$, con $N_{\text{máx}}$ el número máximo de transacciones por cuenta en el pool.
2. **Cota de bytes por bloque:** el límite de gas por bloque (30M) actúa como cota implícita: con 1.952 B de clave pública y 3.309 B de firma por transacción, una transacción mínima paga $\sim 105 \text{ K}$ gas (calldata), por lo que un atacante puede colocar a lo sumo $\lfloor 30\text{M}/105\text{K} \rfloor \approx 285$ transacciones por bloque, equivalentes a $\sim 1,5 \text{ MB}$ de carga útil.
3. **Política de priorización por bytes** (propuesta): ordenar las transacciones del pool por la métrica $\text{gas_price}/\text{size_bytes}$ en lugar de únicamente por gas_price . Esto desalinea los incentivos del atacante (que paga lo mínimo por byte) y los validadores (que prefieren transacciones más densas en gas por byte). No está implementado en este prototipo pero se documenta en el EIP como recomendación.

5.6.3 Maleabilidad de firma y unicidad de transacción

ECDSA presenta una maleabilidad estructural: si (r, s) es válida sobre m , también lo es $(r, n - s)$, lo que obligó a Ethereum a introducir la restricción $s \leq n/2$ (EIP-2). ML-DSA-65 elimina esta clase de ataques por dos razones independientes:

- **Firma determinista:** ML-DSA usa una semilla derivada del mensaje (la «aleatoriedad» proviene de $\text{SHAKE-256}(\text{sk} \parallel \mu)$) en lugar de un nonce externo. Dos firmas del mismo mensaje con la misma clave producen idéntica salida byte a byte, por lo que la unicidad de la firma es trivial.
- **Compromiso al estado completo:** el desafío $c = H(\mu \parallel \mathbf{w}_1)$ liga la firma a la totalidad de los bits altos de $\mathbf{w} = \mathbf{A}\mathbf{y}$. Una alteración bit a bit de cualquier componente de $(\mathbf{z}, h, c)$ rompe la ecuación de verificación con probabilidad abrumadora.

Adicionalmente, el hash de transacción $\text{tx_hash} = \text{SHAKE-256}(0x50 \parallel \text{rlp}(\text{tx}))$ cubre el sobre completo, incluyendo firma y clave pública. Cualquier mutación produce un identificador distinto, por lo que un *front-runner* no puede reenviar una transacción modificada con el mismo hash, evitando los ataques tipo *transaction malleability* que afectaron históricamente a Bitcoin.

5.6.4 Resistencia frente a oráculos cuánticos de hashing

Las funciones hash empleadas (Keccak-256 para el MPT y los logs, SHAKE-256 para direcciones y firmas) ofrecen 256 bits de salida y se asumen modelables como oráculos aleatorios. Frente a un adversario cuántico:

- **Preimagen:** el algoritmo de Grover (§2.4) reduce la complejidad a $\Theta(2^{128})$ consultas al oráculo, considerado computacionalmente infactible (un atacante necesitaría 2^{64} s incluso a 1 GHz de consultas oraculares).
- **Colisión:** el algoritmo BHT (Brassard-Höyer-Tapp) consigue $\Theta(2^{n/3}) = \Theta(2^{85})$ con $\Theta(2^{85})$ memoria. Aún infactible, pero motiva no reducir el tamaño de salida por debajo de 256 bits.

- **Distinguidor:** ningún ataque distingue Keccak- $f[1600]$ de una permutación aleatoria con $< 2^{256}$ consultas, ni clásica ni cuánticamente.

En consecuencia, los componentes basados en hash de Ethereum (raíz de estado, raíz de transacciones, dirección de contrato derivada por CREATE/CREATE2) mantienen su nivel de seguridad efectivo de 128 bits cuánticos, sin necesidad de migración inmediata.

5.7 Análisis de resultados

Los resultados permiten extraer las siguientes conclusiones parciales:

Viabilidad criptográfica: La verificación ML-DSA-65 (42 μ s) es más rápida que la verificación ECDSA (49,2 μ s), lo que demuestra que la migración post-cuántica no implica una degradación del rendimiento de verificación —la operación más crítica para un nodo blockchain. La firma es 8,3× más lenta, pero solo afecta al usuario final, no a la red.

Impacto en tamaño: El incremento de 87× en tamaño de transacción es significativo pero manejable. Las firmas ML-DSA-65 (3.309 B) son más compactas que las de SLH-DSA (7.856 B) y Falcon-512 (666 B) presenta problemas de implementación segura. ML-DSA-65 ofrece el mejor equilibrio tamaño-seguridad-implementabilidad.

Impacto en throughput: La pérdida del 51 % de TPS se debe principalmente al coste de calldata, no a la verificación criptográfica. Un registro de claves públicas (eliminando 1.952 B por transacción recurrente) reduciría esta pérdida a ~35 %.

Validación E2E: El 100 % de las pruebas (27/27 entre single-node y multi-node) pasan satisfactoriamente, demostrando que la integración de ML-DSA-65 en un cliente de ejecución Ethereum es viable sin romper las propiedades fundamentales del protocolo.

Consenso PoA: La rotación round-robin con sellado ML-DSA-65 funciona correctamente en un clúster de 3 nodos, con un drift máximo de 2 bloques y convergencia de estado. Este resultado valida el consenso post-cuántico, aunque la escalabilidad a centenares de validadores (necesaria para PoS) requiere esquemas de agregación que no existen aún.

La Figura 5.4 sitúa temporalmente la urgencia de la migración.

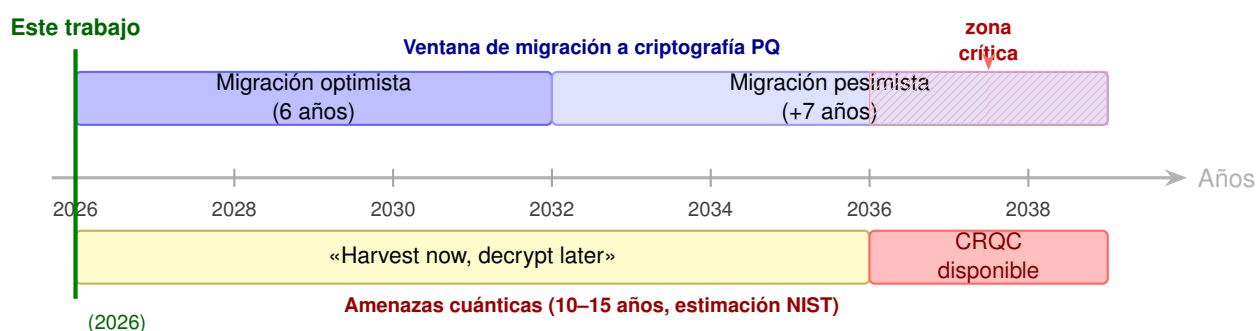


Figura 5.4: Timeline cuántica: la ventana de migración (6–13 años) se solapa con la estimación de disponibilidad de un CRQC (10–15 años, NIST). La amenaza «harvest now, decrypt later» comienza hoy y la zona crítica (rayado rojo) representa los años en que la migración aún no estará completa pero el atacante cuántico ya podría descifrar.

6 CONCLUSIONES Y TRABAJOS FUTUROS

6.1 Cumplimiento de objetivos

A continuación se evalúa el grado de cumplimiento de cada objetivo específico definido en la Sección 1.3.

OE1 — Simulación cuántica de Shor y Grover.

Cumplido. Se implementaron y ejecutaron ambas simulaciones mediante Qiskit Aer. El algoritmo de Shor recupera la clave privada en el 100 % de los casos sobre la curva reducida $\mathbb{F}_{17}$. El algoritmo de Grover demuestra la aceleración cuadrática sobre el Keccak reducido de 4 bits (3 consultas vs. 8 esperadas clásicamente). Los resultados se exponen a través de una API REST documentada.

OE2 — Tipo de transacción PQ (0x50).

Cumplido. Se diseñó e implementó el tipo de transacción 0x50 compatible con EIP-2718, con codificación RLP, hash de firma SHAKE-256 con separación de dominio, y hash de transacción que incluye firma y clave pública. La serialización y deserialización se validan con 4 tests unitarios y 5 tests E2E.

OE3 — Derivación de direcciones PQ.

Cumplido. Las direcciones post-cuánticas se derivan como $\text{SHAKE-256}(pk, 32)[12:32]$, proporcionando separación criptográfica de dominios respecto a las direcciones clásicas (Keccak-256). La derivación se implementa tanto en pq-reth como en pq-wallet, y se verifica mediante tests unitarios y E2E.

OE4 — Opcode PQHASH y precompilado ML-DSA.

Cumplido. El opcode PQHASH (0x21) computa SHAKE-256 con un modelo de gas idéntico a KECCAK256. El precompilado ML-DSA-65 (0x0100) verifica firmas con un coste calibrado de 3.450 gas. Se deshabilitaron 13 precompilados clásicos vulnerables. Los 6 tests del *crate reth-pq-evm* y los 5 tests de *reth-pq-precompile* validan el funcionamiento.

OE5 — Consenso PoA con ML-DSA-65.

Cumplido. Se implementó un mecanismo Proof of Authority con sellado de bloques ML-DSA-65 y rotación round-robin. El clúster de 3 validadores produce bloques correctamente, con drift ≤ 2 bloques y convergencia de estado verificada en los tests multi-nodo.

OE6 — Wallet PQ (CLI y TUI).

Cumplido. Se desarrolló un wallet con 9 comandos CLI y 4 pestañas TUI (Wallet, Transactions, Blocks, Network). Soporta generación de claves, keystore cifrado (AES-256-GCM + Argon2id), firma y envío de transacciones, despliegue de contratos y consultas de estado. Se incluye además un sembrador de cadena para demostración.

OE7 — Suite E2E.

Cumplido. La suite ejecuta 15 escenarios en 6 fases: identidad de cadena (3), consenso PoA (2), comisiones EIP-1559 (2), transacciones PQ (3), contratos inteligentes (2) y multi-nodo (3). Todos los tests pasan satisfactoriamente tanto en configuración single-node (12/12) como multi-nodo (15/15). La suite se puede ejecutar en modo local, Kubernetes o contra un nodo externo.

OE8 — Propuesta de EIP.

Cumplido. Se redactó una propuesta de EIP completa (eips/eip-pq-tx.md) siguiendo el formato estándar, con secciones de Abstract, Motivation, Specification, Rationale, Backwards Compatibility, Test Cases, Reference Implementation y Security Considerations. El estado actual es *Draft*, preparada para presentación formal a la comunidad Ethereum.

6.2 Conclusiones generales

Este trabajo demuestra que la migración de la capa de transacciones de un cliente de ejecución Ethereum a criptografía post-cuántica basada en retículos (ML-DSA-65) es **técnicamente viable** con la tecnología actual. Las principales conclusiones son:

1. **La verificación post-cuántica es más rápida que la clásica.** ML-DSA-65 verifica firmas un 14 % más rápido que ECDSA/secp256k1 (42 μ s vs. 49,2 μ s), lo que significa que la migración no penaliza el rendimiento de los nodos validadores —la operación más crítica para la escalabilidad de la red.
2. **El coste principal es el tamaño, no el cómputo.** Las transacciones post-cuánticas son $\sim 87\times$ más grandes que las clásicas (5,3 KB vs. 61 B), lo que reduce el throughput teórico en un 51 %. Este es un coste significativo pero manejable, mitigable mediante registros de claves públicas y compresión.
3. **La coexistencia clásica/PQ es posible.** El tipo de transacción 0x50 coexiste con los tipos 0x00–0x04 bajo EIP-2718, permitiendo una migración gradual donde los usuarios adoptan cuentas post-cuánticas voluntariamente.
4. **El consenso post-cuántico requiere un nuevo paradigma.** El PoA con ML-DSA-65 funciona para conjuntos pequeños de validadores, pero la escalabilidad a miles de validadores (necesaria para PoS) requiere esquemas de agregación de firmas post-cuánticas que no existen de forma estandarizada.
5. **La amenaza cuántica es real y el tiempo de migración es largo.** Las simulaciones de Shor y Grover confirman experimentalmente las vulnerabilidades teóricas. Con un horizonte de 10–15 años para un CRQC y una migración estimada en 6–13 años, el inicio de la planificación es urgente.

6.3 Limitaciones

1. **Cadena privada, no mainnet.** La implementación se ejecuta en una red privada con 3 validadores y 15 cuentas prefinanciadas. No se ha probado con el volumen de transacciones ni el número de nodos de Ethereum mainnet.
2. **Consenso PoA, no PoS.** El mecanismo Proof of Authority no es escalable a centenares de miles de validadores. La migración del consenso Proof of Stake requiere agregación de firmas post-cuánticas.
3. **Gas price simplificado.** El tipo 0x50 usa un campo `gas_price` plano en lugar del modelo EIP-1559 completo (`max_fee_per_gas`, `max_priority_fee_per_gas`).
4. **Sin registro de claves públicas.** Cada transacción incluye la clave pública completa (1.952 bytes). Un registro on-chain permitiría eliminar este overhead tras la primera transacción.
5. **Simulaciones sobre modelos reducidos.** Las simulaciones cuánticas operan sobre curvas y hashes reducidos ($\mathbb{F}_{17}$, 4-bit Keccak) que demuestran los principios pero no prueban la viabilidad computacional real de los ataques sobre secp256k1 o Keccak-256.

6. **Sin auditoría formal de la implementación criptográfica.** La biblioteca ml-lattice-rs envuelve la implementación RustCrypto (ml-dsa), que es una implementación de referencia pero no ha sido auditada formalmente para uso en producción.
7. **Un solo cliente de ejecución.** La migración real de Ethereum requiere que todos los clientes (geth, Nethermind, Besu, Erigon, reth) implementen el mismo protocolo.
8. **Sin VRF ni Merkle post-cuántico.** La propuesta inicial contemplaba módulos de VRF (*Verifiable Random Function*) y árboles de Merkle basados en primitivas post-cuánticas. Por limitaciones temporales del proyecto, estos componentes no se completaron; las raíces MPT del prototipo siguen usando Keccak-256 (resistente a Grover con margen suficiente) y no se incorpora elección aleatoria verificable. Ambos elementos se documentan en la Sección 6.4.

6.4 Trabajos futuros

1. **Registro de claves públicas on-chain.** Implementar un contrato de registro que mapee dirección → clave pública, permitiendo que las transacciones posteriores a la primera omitan la clave pública (ahorro de 1.952 bytes por transacción). Esto requiere una extensión del formato 0x50 o un nuevo tipo de transacción.
2. **Modelo de comisiones EIP-1559.** Extender el tipo 0x50 con los campos de comisión dinámica (`max_fee_per_gas` y `max_priority_fee_per_gas`) para compatibilidad completa con el mecanismo de Ethereum.
3. **Agregación de firmas post-cuánticas.** Investigar esquemas de agregación basados en STARKs o retículos multilineales que permitan comprimir las atestaciones de miles de validadores en una sola firma compacta. Este es el principal obstáculo para la migración del consenso PoS.
4. **Compresión de claves y firmas ML-DSA.** NIST está evaluando estándares de compresión para claves ML-DSA. La aplicación de estas técnicas podría reducir el tamaño de la clave pública y mejorar el throughput de la red.
5. **Pruebas de carga a escala.** Ejecutar benchmarks con centenares de transacciones por bloque en un clúster de 10+ nodos para validar el rendimiento bajo condiciones más cercanas a producción.
6. **Migración de la capa de consenso.** Diseñar un mecanismo de consenso post-cuántico compatible con la capa de consenso de Ethereum (Beacon Chain), posiblemente basado en firmas de umbral post-cuánticas.
7. **Agilidad criptográfica.** Diseñar un mecanismo de negociación de algoritmos que permita actualizar el esquema de firma (p.ej., de ML-DSA-65 a ML-DSA-87, o a un esquema basado en hash) sin requerir un *hard fork*.
8. **Presentación formal del EIP.** Someter la propuesta `eip-pq-tx.md` al proceso formal de EIP de Ethereum, incluyendo discusión en Ethereum Magicians, revisión por editores y presentación en All Core Devs.
9. **Interoperabilidad cross-chain.** Diseñar puentes (*bridges*) que permitan la verificación de firmas ML-DSA-65 desde otras cadenas EVM-compatibles, y evaluar la integración con rollups que ya utilizan pruebas de conocimiento cero.

10. **VRF post-cuántica.** Diseñar e implementar una función aleatoria verificable resistente a ataques cuánticos — basada en retículos (Module-LWE) o en firmas de hash (XMSS, SPHINCS+) — que permita la elección imparcial de líder en protocolos de consenso, reemplazando el actual mecanismo RANDAO (vulnerable indirectamente vía las claves BLS de los validadores). Este componente formaba parte de la propuesta inicial y queda pendiente como extensión natural del prototipo.
11. **Merkle post-cuántico.** Sustituir la función hash subyacente del Merkle Patricia Trie por una primitiva PQ-nativa (SHAKE-256 o un esquema basado en XMSS) y evaluar empíricamente el impacto en (i) el tamaño de las pruebas de inclusión light-client, (ii) la latencia de *state root* en la verificación de bloques y (iii) la compatibilidad con clientes ligeros existentes. Esta línea constituye el complemento de hashing del trabajo actual y completa la propuesta inicial.

7 BIBLIOGRAFÍA

- Berdik, D., Otoum, S., Schmidt, N., Porter, D., & Jararweh, Y. (2023). A Survey and Comparison of Post-Quantum and Quantum Blockchains. *IEEE Access*, 11, 116688-116703. <https://doi.org/10.1109/ACCESS.2023.3325258>
- Ethereum Community. (2025a). EIP-7932: Secondary Signature Algorithms. <https://eips.ethereum.org/EIPS/eip-7932>
- Ethereum Community. (2025b). EIP-8051: ML-DSA Verification Precompile. <https://eips.ethereum.org/EIPS/eip-8051>
- Fernández-Caramés, T. M., & Fraga-Lamas, P. (2023). Quantum Resistance in Blockchain Networks. *Scientific Reports*, 13, 5765. <https://doi.org/10.1038/s41598-023-32701-6>
- Grover, L. K. (1996). A Fast Quantum Mechanical Algorithm for Database Search. *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, 212-219. <https://doi.org/10.1145/237814.237866>
- Kiktenko, E. O., Pozhar, N. O., Anufriev, M. N., Trushechkin, A. S., Yunusov, R. R., Kurochkin, Y. V., Lvovsky, A. I., & Fedorov, A. K. (2018). Quantum-Secured Blockchain. *Quantum Science and Technology*, 3(3), 035004. <https://doi.org/10.1088/2058-9565/aabc6b>
- Lee, J., Kim, S., & Park, Y. (2024). Quantum-Safe Blockchain in Hyperledger Fabric. *IEEE Access*, 12, 1-15. <https://doi.org/10.1109/ACCESS.2024.3516500>
- Lenstra, A. K., Lenstra, H. W., & Lovász, L. (1982). Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4), 515-534. <https://doi.org/10.1007/BF01457454>
- National Institute of Standards and Technology. (2024a, agosto). *Module-Lattice-Based Digital Signature Standard* (Federal Information Processing Standards Publication N.º FIPS 204). NIST. <https://doi.org/10.6028/NIST.FIPS.204>
- National Institute of Standards and Technology. (2024b, agosto). *Module-Lattice-Based Key-Encapsulation Mechanism Standard* (Federal Information Processing Standards Publication N.º FIPS 203). NIST. <https://doi.org/10.6028/NIST.FIPS.203>
- National Institute of Standards and Technology. (2024c). *Post-Quantum Cryptography: NIST Standards and Beyond* (inf. téc.). NIST. <https://csrc.nist.gov/projects/post-quantum-cryptography>
- National Institute of Standards and Technology. (2024d, agosto). *Stateless Hash-Based Digital Signature Standard* (Federal Information Processing Standards Publication N.º FIPS 205). NIST. <https://doi.org/10.6028/NIST.FIPS.205>
- Paradigm. (2024). *Reth: Modular, Contributor-Friendly and Blazing-Fast Ethereum Client Written in Rust*. <https://github.com/paradigmxyz/reth>
- Regev, O. (2005). On Lattices, Learning with Errors, Random Linear Codes, and Cryptography. *Proceedings of the 37th Annual ACM Symposium on Theory of Computing (STOC)*, 84-93. <https://doi.org/10.1145/1060590.1060603>
- Shor, P. W. (1994). Algorithms for Quantum Computation: Discrete Logarithms and Factoring. *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*, 124-134. <https://doi.org/10.1109/SFCS.1994.365700>
- Taguchi, Y., & Takayasu, A. (2025). Choosing Coordinate Forms for Solving ECDLP Using Shor's Algorithm. *arXiv preprint*. <https://arxiv.org/abs/2502.12441>
- Wood, G. (2014). *Ethereum: A Secure Decentralised Generalised Transaction Ledger* (inf. téc.) (Yellow Paper, versión actualizada periódicamente). <https://ethereum.github.io/yellowpaper/paper.pdf>
- Zoltu, M. (2020). EIP-2718: Typed Transaction Envelope. <https://eips.ethereum.org/EIPS/eip-2718>

8 ANEXOS

8.1 Anexo 1: Declaración de uso de IAG

En cumplimiento de la normativa de la Universidad Intercontinental de la Empresa, se declara el uso de las siguientes herramientas de Inteligencia Artificial Generativa (IAG) durante la elaboración de este Trabajo Fin de Grado:

Herramientas utilizadas

1. Claude (Anthropic) — `claude-opus-4-20250514`

- **Uso:** Asistencia en la generación de código Rust (tipos de transacción, consenso PoA, EVM, pool, red), redacción de la propuesta de EIP, revisión y estructuración de la memoria del TFG, y resolución de errores de compilación.
- **Alcance:** Se utilizó como asistente de programación interactivo para acelerar la implementación de los componentes del sistema y la redacción de documentación técnica. Todo el código generado fue revisado, modificado cuando fue necesario y validado mediante las suites de tests unitarios y de extremo a extremo.
- **Limitaciones:** La herramienta no tiene acceso a información posterior a su fecha de corte de entrenamiento. Las decisiones de diseño arquitectónico, la selección de algoritmos criptográficos y la definición de los objetivos del proyecto fueron responsabilidad exclusiva del autor.

Aspectos no generados por IAG

Los siguientes elementos fueron desarrollados íntegramente por el autor sin asistencia de IAG:

- Definición del problema, objetivos y alcance del TFG.
- Selección de ML-DSA-65 frente a Falcon y SLH-DSA.
- Decisión de PoA frente a PoS para el consenso post-cuántico.
- Diseño de la arquitectura del sistema (capas, componentes, interfaces).
- Ejecución e interpretación de las simulaciones cuánticas con Qiskit.
- Ejecución, depuración y análisis de los tests E2E.
- Medición y análisis de los benchmarks de rendimiento.
- Revisión crítica y corrección de todo el contenido generado.

Declaración de responsabilidad

El autor asume la plena responsabilidad sobre la corrección, originalidad y rigor académico de todo el contenido presentado en este trabajo, independientemente de las herramientas utilizadas durante su elaboración.

8.2 Anexo 2: Especificación EIP — Post-Quantum Transaction Type

La propuesta completa de EIP se encuentra en el archivo `eips/eip-pq-tx.md` del repositorio del proyecto. A continuación se presenta un resumen estructurado de los campos principales.

Metadatos

Campo	Valor
EIP	XXXX (pendiente de asignación)
Título	Post-Quantum Transaction Type with ML-DSA-65
Autor	0xGeN02 (@0xGeN02)
Estado	Draft
Tipo	Standards Track / Core
Requiere	EIP-2718
Fecha	2026-05-17

Constantes

Constante	Valor	Descripción
PQ_TX_TYPE	0x50	Identificador de tipo de transacción EIP-2718
MLDSA65_SIG_LEN	3.309	Longitud de firma ML-DSA-65 (bytes)
MLDSA65_PK_LEN	1.952	Longitud de clave pública ML-DSA-65 (bytes)
PQHASH_OPCODE	0x21	Opcode EVM para SHAKE-256
MLDSA_PRECOMPILE_ADDR	0x0...0100	Dirección del precompilado ML-DSA-65
MLDSA_VERIFY_GAS	3.450	Coste de gas de verificación ML-DSA-65

Formato de transacción

```
0x50 || rlp([chain_id, nonce, gas_price, gas_limit,
             to, value, input, signature, public_key])
```

Hash de firma

```
signing_hash = SHAKE-256(
    0x50 || chain_id(8B) || nonce(8B) || gas_price(16B)
    || gas_limit(8B) || to_flag(1B) || [to(20B)]
    || value(16B) || input
    , output_length=32)
```

Derivación de dirección

```
address = SHAKE-256(public_key, output_length=32)[12..32]
```

Precompilados deshabilitados

ecrecover (0x01), BN254 (0x06–0x08), KZG (0x0a), BLS12-381 (0x0b–0x13).

Referencia

El texto completo de la propuesta, incluyendo Rationale, Backwards Compatibility, Test Cases y Security Considerations, está disponible en:

<https://github.com/0xGeN02/PostQuantumEVM/blob/main/eips/eip-pq-tx.md>

8.3 Anexo 3: Estructura de repositorios

Repositorio raíz: PostQuantumEVM

PostQuantumEVM/	
├─ MEMORIA_TFG/	Memoria del TFG (LaTeX)
├─ corpus/	Referencias bibliográficas
├─ docker-compose.yml	Despliegue 3 validadores + qiskit-api
├─ Dockerfile.pq-reth	Imagen Docker del nodo PQ
├─ docs/	Benchmarks y figuras
│ └─ benchmark_analysis.ipynb	
│ └─ figures/	7 gráficos de rendimiento (PNG)
├─ e2e/	Infraestructura E2E
│ └─ run-e2e.sh	Script orquestador (local/k8s/external)
│ └─ fixtures/	Keystore de prueba
│ └─ k8s/	Manifiestos Kubernetes (5 YAML)
├─ eips/	
│ └─ eip-pq-tx.md	Propuesta EIP tipo 0x50
├─ ml-lattice-rs/	Submódulo: ML-DSA + ML-KEM en Rust
├─ pq-reth/	Fork de reth con crates PQ
├─ pq-wallet/	Workspace: wallet + E2E + seeder
└─ qiskit-api/	API de simulación cuántica

pq-reth: Crates post-cuánticos

pq-reth/crates/pq/	
├─ reth-pq-primitives/	TX type 0x50, RLP, signing hash, address
├─ reth-pq-node-primitives/	NodePrimitives (PqPrimitives)
├─ reth-pq-consensus/	Validación TX ML-DSA-65
├─ reth-pq-precompile/	Precompilado ML-DSA (0x0100)
├─ reth-pq-evm/	PQHASH opcode (0x21), EVM config
├─ reth-pq-poa/	PoA: round-robin, sellado, validación
├─ reth-pq-pool/	Mempool: validación PQ
└─ reth-pq-node/	Wiring: PqNode, engine, payload

pq-wallet: Workspace

pq-wallet/	
├─ pq-wallet-core/	Biblioteca: keygen, keystore, signer, tx, rpc, error
├─ pq-wallet-cli/	9 subcomandos: new, address, balance, send, deploy, receipt, sign, call, export-seed
├─ pq-wallet-tui/	4 pestañas: Wallet, Transactions, Blocks, Network
├─ pq-e2e-test/	15 escenarios en 6 fases
└─ pq-chain-seeder/	Sembrador de cadena para demostración

qiskit-api: Simulación cuántica

```
qiskit-api/
├── src/
│   ├── main.py           FastAPI app (v0.2.0)
│   ├── models.py         Esquemas Pydantic
│   └── routers/
│       ├── shor.py       Endpoints Shor ECDLP
│       └── grover.py      Endpoints Grover preimagen
├── lib/
│   ├── shor.py           Algoritmo de Shor (DFT modular 2D)
│   ├── grover.py         Algoritmo de Grover (oráculo + difusión)
│   ├── secp256k1_toy.py  Curva F_17:  $y^2 = x^3 + 7$ 
│   └── keccak_toy.py     Keccak reducido (4 bits, 2 rondas)
└── examples/             Notebooks Jupyter (4)
```

ml-lattice-rs: Criptografía post-cuántica

```
ml-lattice-rs/
├── dilithium/src/lib.rs  ML-DSA: keygen/sign/verify
│                           (MlDsa44, MlDsa65, MlDsa87)
└── kyber/src/lib.rs     ML-KEM: keygen/encapsulate/decapsulate
                           (MlKem512, MlKem768, MlKem1024)
```

URLs de los repositorios

- <https://github.com/OxGeN02/PostQuantumEVM> — Repositorio raíz
- <https://github.com/OxGeN02/pq-reth> — Fork de reth
- <https://github.com/OxGeN02/ml-lattice-rs> — Biblioteca criptográfica

8.4 Anexo 4: Resultados de Aprendizaje

Este anexo vincula los nueve Resultados de Aprendizaje (RA) declarados en la propuesta del Trabajo Fin de Grado con las secciones específicas de la memoria y los componentes del repositorio que evidencian su consecución. Se incluyen tres RA por cada uno de los tres tipos definidos en el plan de estudios del Grado en Ingeniería de Sistemas de Información (GSI) de la Universidad UIE: *Conocimiento* (RPFA-CONO), *Habilidad* (RPFA-HABI) y *Competencia* (RPFA-COMP).

Conocimiento

Cuadro 8.1: Resultados de Aprendizaje de tipo Conocimiento.

Código	Descripción	Vinculación en la memoria
RPFA-CONO-09	Conceptos, métodos, modelos y herramientas computacionales con énfasis en complejidad computacional, arquitectura de software y modelos de computación emergente.	§2.1–2.3 (computación cuántica, Shor, Grover), §2.5 (arquitectura EVM), §4 (arquitectura del fork reth), §5.1 (simulaciones Qiskit).
RPFA-CONO-10	Definiciones, propiedades, representación y operaciones de la lógica proposicional, los conjuntos, las estructuras tipo grafos y árboles, y las herramientas de matemática discreta.	§2.4 (retículos, Module-LWE, bases de retículos como grafos en $\mathbb{R}^n$), §2.5.3 (Merkle Patricia Trie como árbol radix híbrido), §5.7 (análisis de seguridad).
RPFA-CONO-11	Conceptos, componentes y métodos de un problema de optimización de funciones matemáticas, y uso de herramientas software para su representación y solución.	§4.3 (calibración del coste de gas del precompilado como problema de optimización: minimizar gas s.a. restricciones de seguridad y latencia), §5.4–5.5 (análisis de throughput, latencia y tamaño de bloque).

Habilidad

Competencia

Síntesis

La conjunción de los nueve RA cubre los tres ejes formativos esperados del título: (i) *fundamentos teóricos* (CONO) materializados en el marco teórico del Capítulo 2; (ii) *habilidades prácticas* (HABI) materializadas en la implementación del fork PQ-reth, la biblioteca `ml-lattice-rs` y las simulaciones cuánticas del Capítulo 5; y (iii) *competencias transversales* (COMP) materializadas en la formulación rigurosa del problema, su modelado como autómata, la optimización de los costes de gas y la validación empírica multinodo. El TFG cubre, por tanto, los tres RA requeridos en cada categoría según la propuesta original.

Cuadro 8.2: Resultados de Aprendizaje de tipo Habilidad.

Código	Descripción	Vinculación en la memoria
RPFA-HABI-09	Aplicar conceptos, métodos, modelos y herramientas de complejidad computacional, arquitectura software y computación cuántica y natural para resolver problemas en ingeniería y empresa.	§2.1–2.3 + §5.1 (despliegue de Shor sobre $\mathbb{F}_{17}$ y Grover sobre Keccak reducido en Qiskit), §3 (análisis de impacto del riesgo cuántico en la industria blockchain).
RPFA-HABI-10	Aplicar conceptos, modelos, operaciones y herramientas de matemática discreta para analizar, representar y resolver problemas dentro de la profesión.	§2.4 (operaciones en anillos R_q , NTT, retículos), §4.2 (derivación de direcciones con SHAKE-256, codificación RLP), §4.4 (formato binario de la transacción 0x50).
RPFA-HABI-12	Utilizar la teoría de autómatas, los lenguajes formales, las expresiones regulares, sus operaciones y herramientas software para representar, analizar y resolver problemas de naturaleza discreta.	§2.5.4 (formalización de la EVM como autómata finito determinista $\mathcal{M}_{\text{EVM}} = (Q, \Sigma, \delta, q_0, F)$ con diagrama de estados, Figura 2.4).

Cuadro 8.3: Resultados de Aprendizaje de tipo Competencia.

Código	Descripción	Vinculación en la memoria
RPFA-COMP-10	Identificar, formular, modelar, analizar y resolver problemas complejos cuya solución requiera el uso de las matemáticas discretas en ingeniería y empresa.	§3 (análisis de la situación y planteamiento del problema), §4 completo (diseño e implementación del tipo de transacción 0x50, mempool, opcode PQHASH, precompilado 0x0100).
RPFA-COMP-11	Identificar, describir, formular, analizar, resolver e interpretar los resultados de problemas de optimización de funciones lineales y no lineales con y sin restricciones, mediante diferentes métodos y herramientas software.	§4.3 (gas metering: $g = \lceil t_{\mu s} \cdot r_{\text{gas}/\mu s} \cdot \alpha \rceil$ con factor de seguridad $\alpha = 1,35$), §5.4–5.5 (interpretación empírica: throughput, latencia, tamaño de bloque).
RPFA-COMP-12	Identificar, formular, modelar, analizar y resolver problemas de naturaleza discreta con el uso de autómatas determinísticos y no-determinísticos en ingeniería y empresa.	§2.5.4 (EVM como DFA), §4.1 (extensión del alfabeto Σ con el opcode PQHASH preservando determinismo), §4.4 (validación de transacciones 0x50 en el mempool como aceptación por autómata).